東南大學 软件工程研究所

INSTITUTE OF SOFTWARE ENGIEERING,SOUTHEAST UNIVERSITY

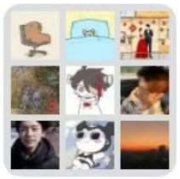
软件全方位缺陷检测

李必信

东南大学计算机科学与工程学院

东南大学软件工程研究所

2025年2月21日



群聊：软件质量保障课程群2025

该二维码7天内(3月7日前)有效，重新进入将更新

东南大学2024—2025学年校历

说 明

一、教职工上班：8月16日

二、本科生暑期学校：
8月19日至9月15日
（研究生教学由导师或课题组安排）

三、在校生注册：8月19日至23日

四、新生报到注册
本科新生：8月24日
研究生新生：9月12日

五、本科新生军训：8月25日至9月14日

六、校学位评定委员会会议：9月4日

七、新生开学典礼：9月14日

八、中秋节：9月15日至17日放假调休，9月14日上班，学生教学活动另行通知安排。

九、秋季学期上课：9月18日

十、国庆节：10月1日至7日放假调休，9月29日和10月12日分别补10月4日和7日的课/班。

十一、校运会：11月1日至2日，停课

十二、校学位评定委员会会议：12月11日

十三、停课复习考试：1月6日至19日

十四、学生寒假：1月20日至2月16日

十五、教职工轮休：1月20日至2月12日

2024-2025学年暑期学校（2024.8.19—2024.9.15）

周次	月份	星期 一	二	三	四	五	六	日
	八月	12	13	14	15	16	17	18
1		19	20	21	22	23	24	25
2		26	27	28	29	30	31	1
3	九月	2	3	4	5	6	7	8
4		9	10	11	12	13	14	15

2024-2025学年秋季学期（2024.9.16—2025.1.19）

周次	月份	星期 一	二	三	四	五	六	日
1	九月	16	17	18	19	20	21	22
2		23	24	25	26	27	28	29
3		30	1	2	3	4	5	6
4	十月	7	8	9	10	11	12	13
5		14	15	16	17	18	19	20
6		21	22	23	24	25	26	27
7		28	29	30	31	1	2	3
8	十一月	4	5	6	7	8	9	10
9		11	12	13	14	15	16	17
10		18	19	20	21	22	23	24
11		25	26	27	28	29	30	1
12	十二月	2	3	4	5	6	7	8
13		9	10	11	12	13	14	15
14		16	17	18	19	20	21	22
15		23	24	25	26	27	28	29
16		30	31	1	2	3	4	5
17	一月	6	7	8	9	10	11	12
18		13	14	15	16	17	18	19

2024-2025学年寒假（2025.1.20-2025.2.16）

周次	月份	星期 一	二	三	四	五	六	日
1	一月	20	21	22	23	24	25	26
2		27	28	29	30	31	1	2
3	二月	3	4	5	6	7	8	9
4		10	11	12	13	14	15	16

2024-2025学年春季学期（2025.2.17—2025.6.22）

周次	月份	星期 一	二	三	四	五	六	日
	二月	10	11	12	13	14	15	16
1		17	18	19	20	21	22	23
2		24	25	26	27	28	1	2
3	三月	3	4	5	6	7	8	9
4		10	11	12	13	14	15	16
5		17	18	19	20	21	22	23
6		24	25	26	27	28	29	30
7		31	1	2	3	4	5	6
8	四月	7	8	9	10	11	12	13
9		14	15	16	17	18	19	20
10		21	22	23	24	25	26	27
11		28	29	30	1	2	3	4
12	五月	5	6	7	8	9	10	11
13		12	13	14	15	16	17	18
14		19	20	21	22	23	24	25
15		26	27	28	29	30	31	1
16	六月	2	3	4	5	6	7	8
17		9	10	11	12	13	14	15
18		16	17	18	19	20	21	22

2024-2025学年暑假（2025.6.23-2025.8.24）

周次	月份	星期 一	二	三	四	五	六	日
1	六月	23	24	25	26	27	28	29
2		30	1	2	3	4	5	6
3	七月	7	8	9	10	11	12	13
4		14	15	16	17	18	19	20
5		21	22	23	24	25	26	27
6		28	29	30	31	1	2	3
7	八月	4	5	6	7	8	9	10
8		11	12	13	14	15	16	17
9		18	19	20	21	22	23	24

说 明

一、教职工上班：2月13日

二、春季学期上课：2月17日

三、在校生注册：2月17日至21日

四、2025级春博报到：2月26日
上课：3月3日

五、校学位评定委员会会议：3月12日

六、停课复习考试：6月9日至22日

七、毕业离校手续
本科生：6月14日至20日
研究生：毕业之日起两周内，6月批次6月20日前

八、校学位评定委员会会议：6月18日

九、毕业典礼：6月19日至20日

十、学生暑假：6月23日至8月24日

十一、教职工轮休：6月30日至8月19日

十二、教职工上班：8月20日

2025年放假安排待国务院办公厅公布2025年节假日安排后另行通知。学校相关专项工作时间由有关部门另行通知安排。

《软件质量》课程简介

- 简介：48+16学时， 3学分
- 时间和地点：周五3-5节（1-16周）教3-402
- 理论课**48**学时：李必信/助教:王煜钧
- 实验课**16**学时：陈浩,谢瑞麟,徐志豪[第5/9/11/13周的周六3-5节]
- 教材:《软件全方位缺陷检测技术》（讲义）
- 参考书
  - 《开发者测试》, 王兴亚 王智钢 赵源 陈振宇 编著，机械工业出版社，2019.
  - 《全程软件测试(第三版)》, 朱少民 编著, 人民邮电出版社，2019.
- 考勤：每次上课需要签到
- 作业：思考题+课程作业40%
- 考核：考勤10%+作业40%+考试50%
- 课程作业：（1）+（3）或者（2）+（3）

课程作业

任务内容:（1）基于AI的软件质量保障方法[AI for SQA]

基本要求：

（1）自行组队，每组4人。

（2）参考一篇论文[中英文都可以]，完整地介绍一种基于AI的软件质量保障方法，包括例子、实验情况，鼓励自己动手做实验。

（3）准备PPT，进行课堂汇报10-15分钟, 组号顺序就是汇报顺序，时间是5月1日之后。

（4）备注参考论文完整信息[作者、论文题目、发表刊物、发表时间等]。

（5）同一小组的所有成员课程作业成绩相同。

（6） 这里AI是技术手段：有/无监督学习、联邦学习、深度学习、大模型、…

（7）软件质量保障方法：AI+[评审、分析、测试、验证、度量、监控、…]

课程作业(续)

任务内容:（2）面向AI的软件质量保障方法[SQA for AI]

基本要求:

（1）自行组队，每组4人。

（2）参考一篇论文[中英文都可以]，完整地介绍一种基于AI的软件质量保障方法，包括例子、实验情况，鼓励自己动手做实验。

（3）准备PPT，进行课堂汇报10-15分钟, 组号顺序就是汇报顺序，时间是5月1日之后。

（4）备注参考论文完整信息[作者、论文题目、发表刊物、发表时间等]。

（5）同一小组的所有成员课程作业成绩相同。

（6）这里AI是保障对象，即保障AI的质量：GNN/GCN、大模型、机器人、自动驾驶等

（7）软件质量保障方法[可以不是AI技术]：评审、分析、测试、验证、度量、监控、仿真、…

课程作业(续)

任务内容:（3）《软件质量保障》知识图谱

基本要求:

（1）自行组队，每组4人。

（2）梳理《软件质量保障》的知识点及其之间的关系，用Neo4j、Apache、Jena Dgraph、Protégé等画图工具生成知识图谱。

（3）从管理角度、从技术角度、以及从全局的角度构建3张知识图谱；分别于3月底、4月底、5月底之前提交就可以。电子版提交到：bx.li@seu.edu.cn

（4）同一小组的所有成员课程作业成绩相同。

《软件质量保障》课程定位

① 软件(software)无处不在，软件定义一切(Software-Defined Everything, SDE/SDx)

② 软件质量(Software Quality)是任何软件赖以生存的基础，质量差的软件注定是没用的！

③ 软件质量保障(Software Quality Assurance)：管理+技术! 先验方法+后验方法! 管理：**ISO、CMMI、4P Model、…；** 技术：分析、测试、监控、模拟、…

④ 软件缺陷(Software Defect)是导致软件质量差的罪魁祸首！但是软件存在缺陷又是不可避免的。

⑤ 软件全方位缺陷检测技术(Comprehensive Software Defect Detection Technology)是保障软件全生命周期质量的有效**方法库、工具包和百宝箱**！软件工程学科（其他学科也差不多）没有银弹(仙丹, No Silver Bullet)！

⑥ 软件测试(Software Testing)是软件全方位缺陷检测技术中最有效、最广泛使用的技术！大概70%左右的软件缺陷是由软件测试[静态+动态]发现的。

⑦ 《软件质量保障》前期基础课程是《编译原理》《程序设计》《软件工程》等；和《软件测试》《软件安全》等是兄弟姐妹课程。

《软件全方位缺陷检测》提纲

一．什么是软件？
二．什么是软件质量？
三．软件全生命周期质量保障问题
四．软件全方位缺陷检测主流方法
五．软件全方位缺陷预测主流方法
六．SHC研究进展汇报

思考题(2025.2.21)

1. 什么是软件？
2. 什么是软件质量？
3. 什么是软件缺陷？
4. 软件缺陷都有哪些类型？
5. 为什么需要全方位的缺陷检测？

一 什么是软件？

- 什么是软件？[内涵]
  - 软件（Software）是一系列按照特定顺序组织的计算机指令和数据的集合。
  - 简单来说：软件=程序+数据+文档

程序：计算机指令/语言代码【加工厂】
数据：程序的加工处理对象【原材料：数字、文字、模型、图表、图形、图像、音频、视频】
文档：记录怎么开发、测试、使用、运维、注释等的文字、模型和图表

- 软件的三种类型[外延]
  - 编程语言
    - C、C++、Java、Python、C#、Visual Basic.NET、PHP、Javascript、SQL、R、Ruby

用于编写其他软件的软件

- 基础软件
  - 操作系统：Windows、Unix、Linux、 Sybain、Andriod、IOS、 Windows CE、Windows Mobile、 Palm OS
  - 开发平台或环境：Visual Studio（VS）、 NetBeans、PyCharm、 IntelliJ IDEA、Eclipse、 Code:: Blocks、Aptana Studio 3、 CodeLite、 Xcode、 Komodo
  - 数据库：常见的关系型数bai据库有Mysql 、SQL Server、Oracle、Sybase、DB2
  - 工具软件：建模工具、测试工具、管理工具、调试工具

用于开发其他软件或者支撑其他软件运维的软件

- 应用软件: 特定领域软件
  - 按应用领域划分：财务软件、图书管理软件、各种控制软件、嵌入式软件、办公自动化软件、安全监控软件、智能制造软件、海洋生态健康检测和分析软件、物流管理软件、供应链管理软件、网上拍卖软件系统、竞选投票软件、...

用于处理和管理应用领域业务的软件

● 核心关注点

- 用户和开发人员关心的永远是这几个方面：
  - 软件产品能力（capability）：能做什么，是否有新功能
  - 软件产品质量（quality）：有多好，例如，性能（performance）如何？是否易维护？等
  - 软件开发和运维成本（cost）：需要多少钱，多少人力
  - 软件开发效率（efficiency）：开发时间需要多久，使用有多方便？

- 研究新语言、新方法、新技术、新模型、以及如何培养新人才，便于我们更好地满足用户和开发人员需求：
  - 软件产品能力需求：能力越强越好|数据处理、文档处理、语音处理、图像处理、视频处理…
  - **软件产品质量需求：质量越高越好|功能正确性和完整性、性能、安全、可靠、可信…**
  - 软件开发效率需求：速度越快越好|开发效率、维护效率、管理效率、测试效率、…
  - 软件运维成本需求：成本越低越好|开发成本、运行成本、维护成本、更新换代成本、…

二 什么是软件质量？

- 内涵：1）概括地说，软件质量就是“软件与明确地和隐含地定义的需求相一致的程度”。2）具体地说，软件质量是软件与明确叙述的功能和性能需求、文档中明确描述的开发标准以及任何专业开发的软件产品都应该具有的隐含特征相一致的程度。
- 外延：
  - 1）软件的各种质量：产品质量（数据、代码、文档、模型、算法等）、过程质量、客户满意度等。
  - 2）各种软件的质量：基础软件质量（编程语言，操作系统,数据库,工具软件,平台软件、中间件等…）、应用软件质量（嵌入式软件、智能控制软件、管理软件…）。

- 如何评估(和保障)软件质量？

质量评估模型

Jim McCall 软件质量模型（1977 年）
Barry W. Boehm 软件质量模型（1978 年）
FURPS/FURPS+ 软件质量模型
R. Geoff Dromey 软件质量模型
ISO/IEC 9126 软件质量模型（1993 年）
ISO/IEC 25010 软件质量模型（2011）

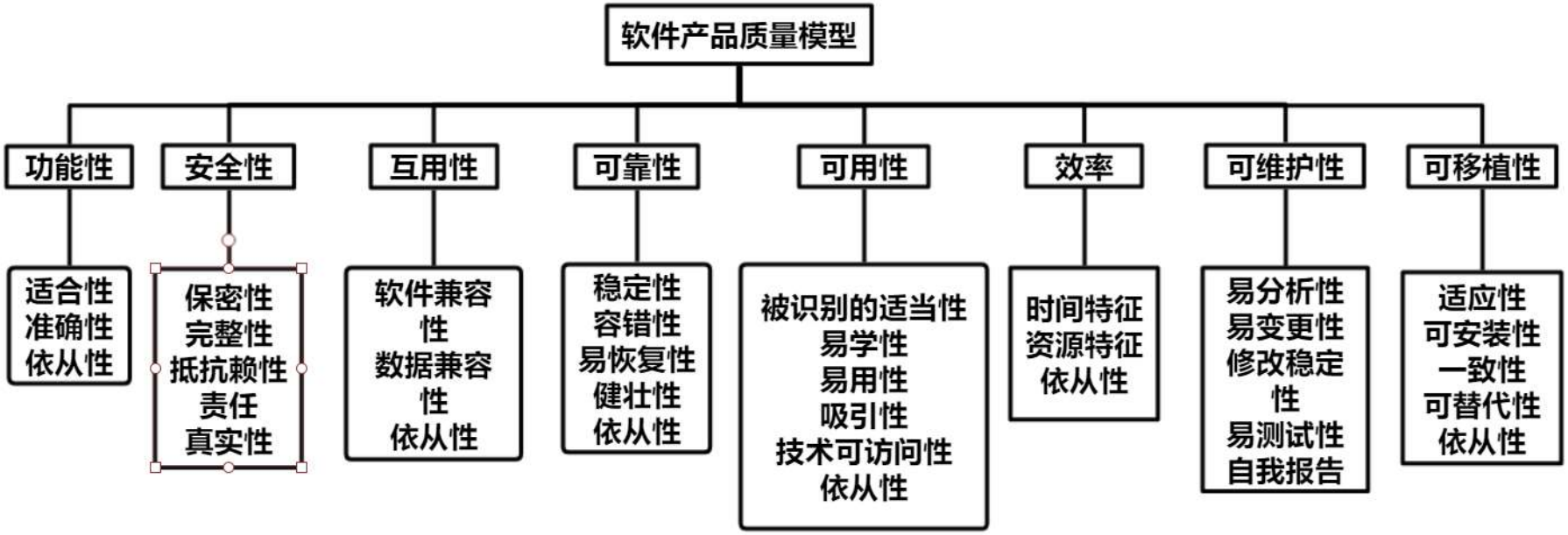
质量标准

ISO/IEC 9126、 ISO/IEC 25010 、 GB/T 16260.1-2006 《软件工程 产品质量 第1部分 质量模型》 、 GJB 5236-2004 军用软件质量度量，等等。

ISO/IEC 软件质量模型（ISO/IEC 25010）是一种用于评估和描述软件质量特性的国际标准。该标准定义了 8 种主要的软件质量特性：

- **功能适用性**：软件功能是否满足用户需求，包括正确性、完整性、适当性等。
- **可靠性**：软件能够在给定条件下正常运行的能力，包括成熟性、可靠性、容错性、可恢复性等。
- **易用性**：软件是否易于使用，包括可理解性、学习性、操作性、吸引力等。
- **效率**：软件在给定资源下实现所需功能的能力，包括时间效率、空间效率等。
- **可维护性**：软件的易修改性、稳定性和易测试性，包括代码可读性、可维护性、可测试性等。
- **可移植性**：软件在不同环境下的适用性，包括可适应性、可安装性、互换性、可替代性等。
- **安全性**：软件防范或抵御未经授权访问、窃取、破坏或修改数据的能力，包括机密性、完整性、可用性等。
- **兼容性**：软件与其他系统和组件的互操作性，包括协议、格式、接口等。

ISO/IEC 25010



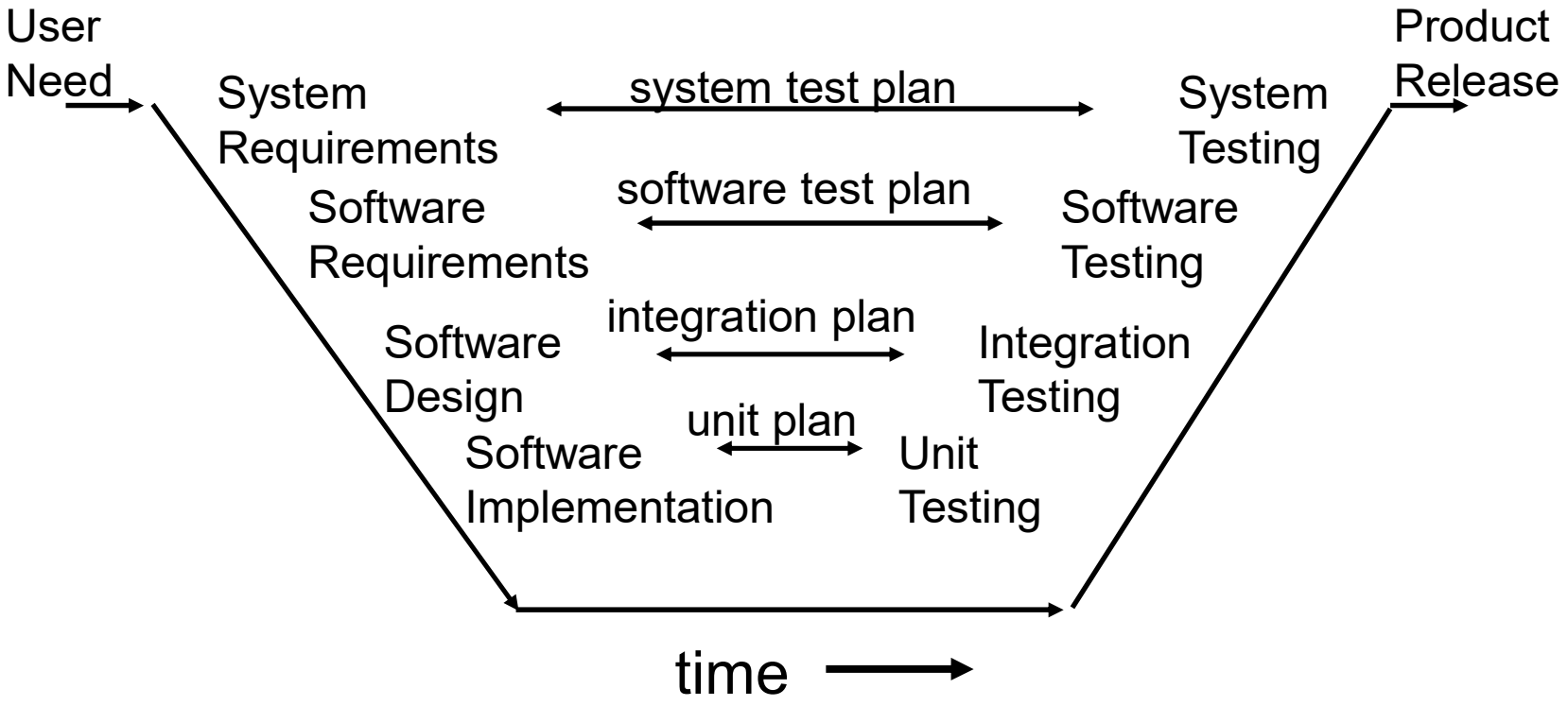
三 软件全生命周期质量保障问题

1 何谓全生命周期软件质量保障?
2 如何进行全生命周期软件质量保障?
3 软件质量和软件缺陷的关系

1 何谓全生命周期软件质量？

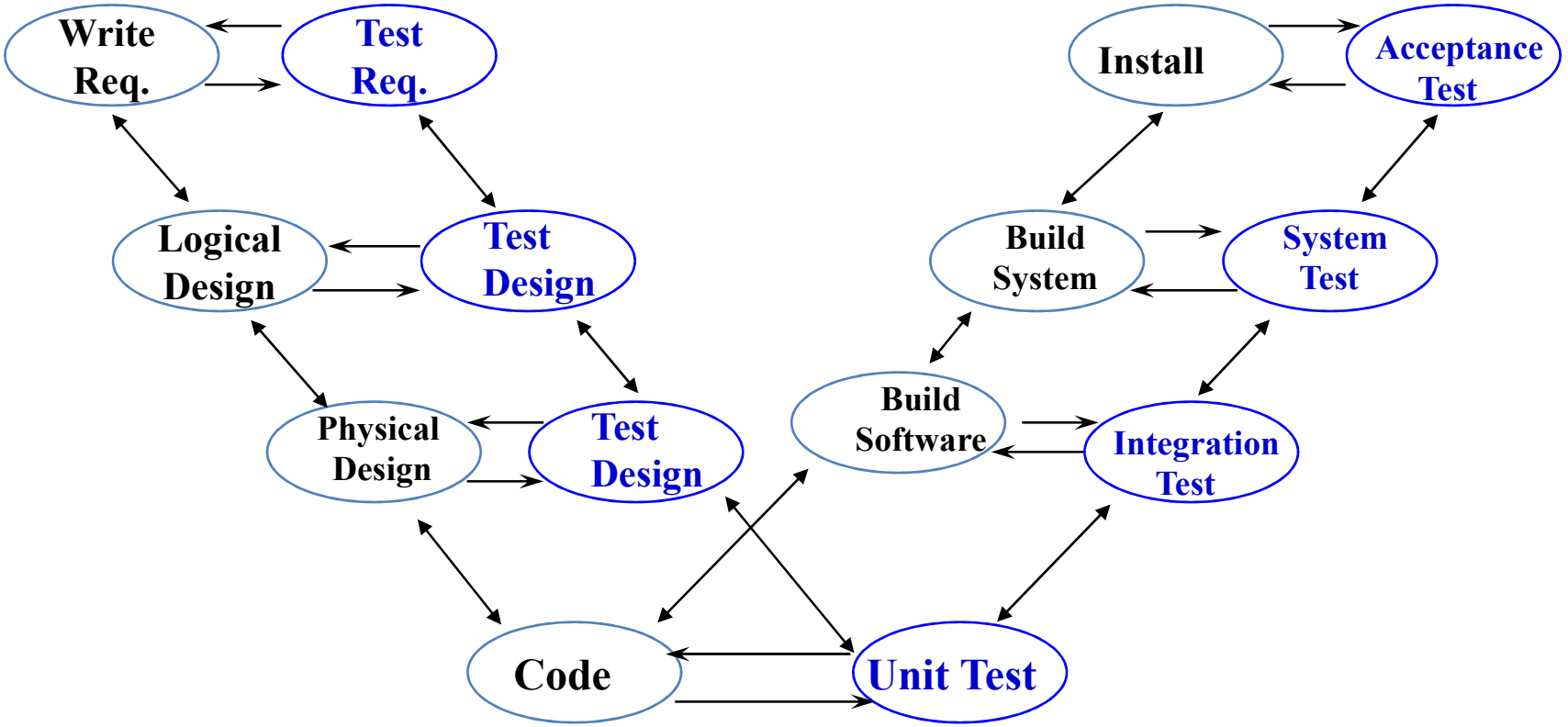
何为全生命周期？
何为质量保障？

- If we rely on testing alone, defects **created first** are **detected last**

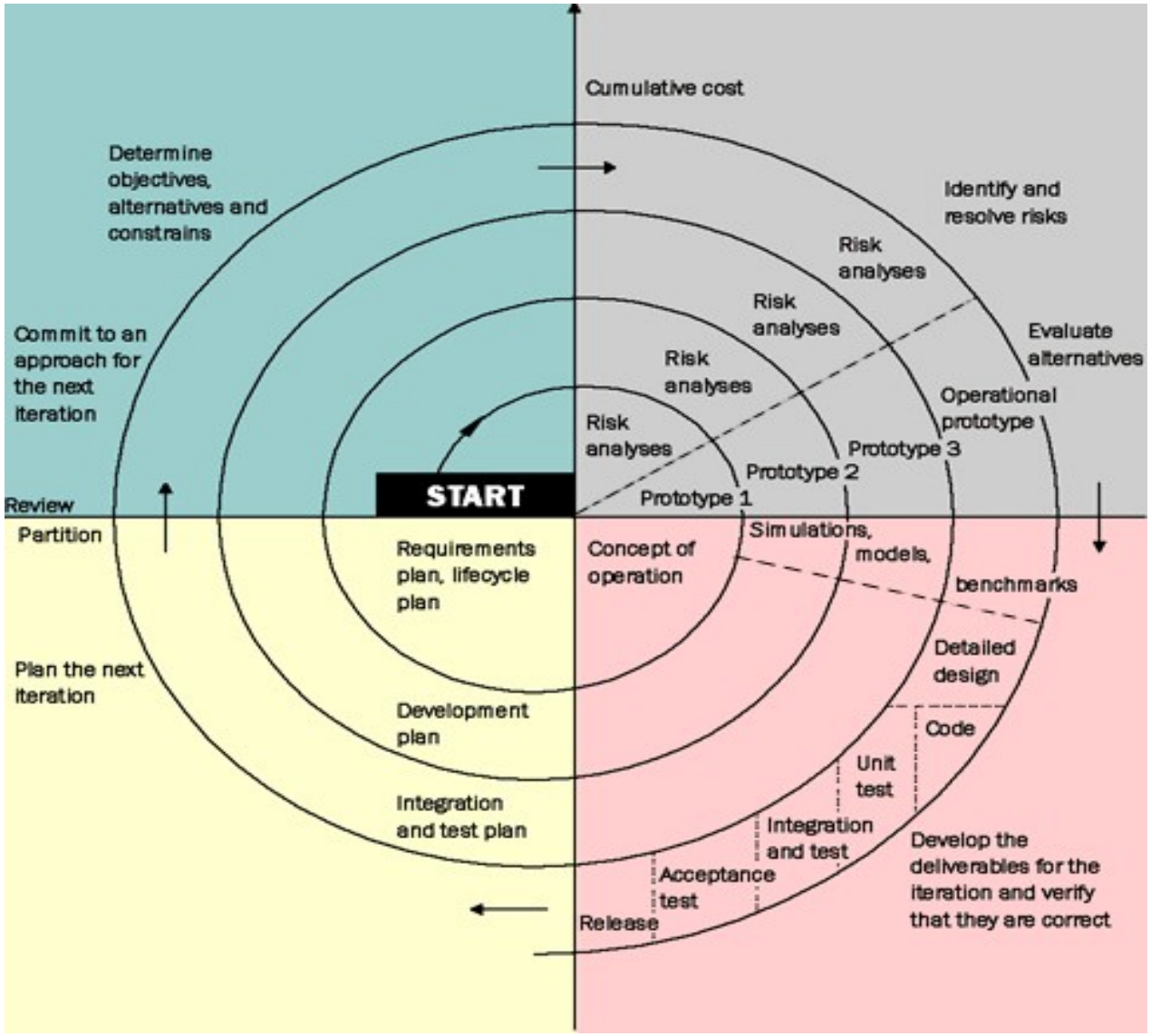


这是软件生命周期后期阶段的质量保障！

W - Whole Life Cycle

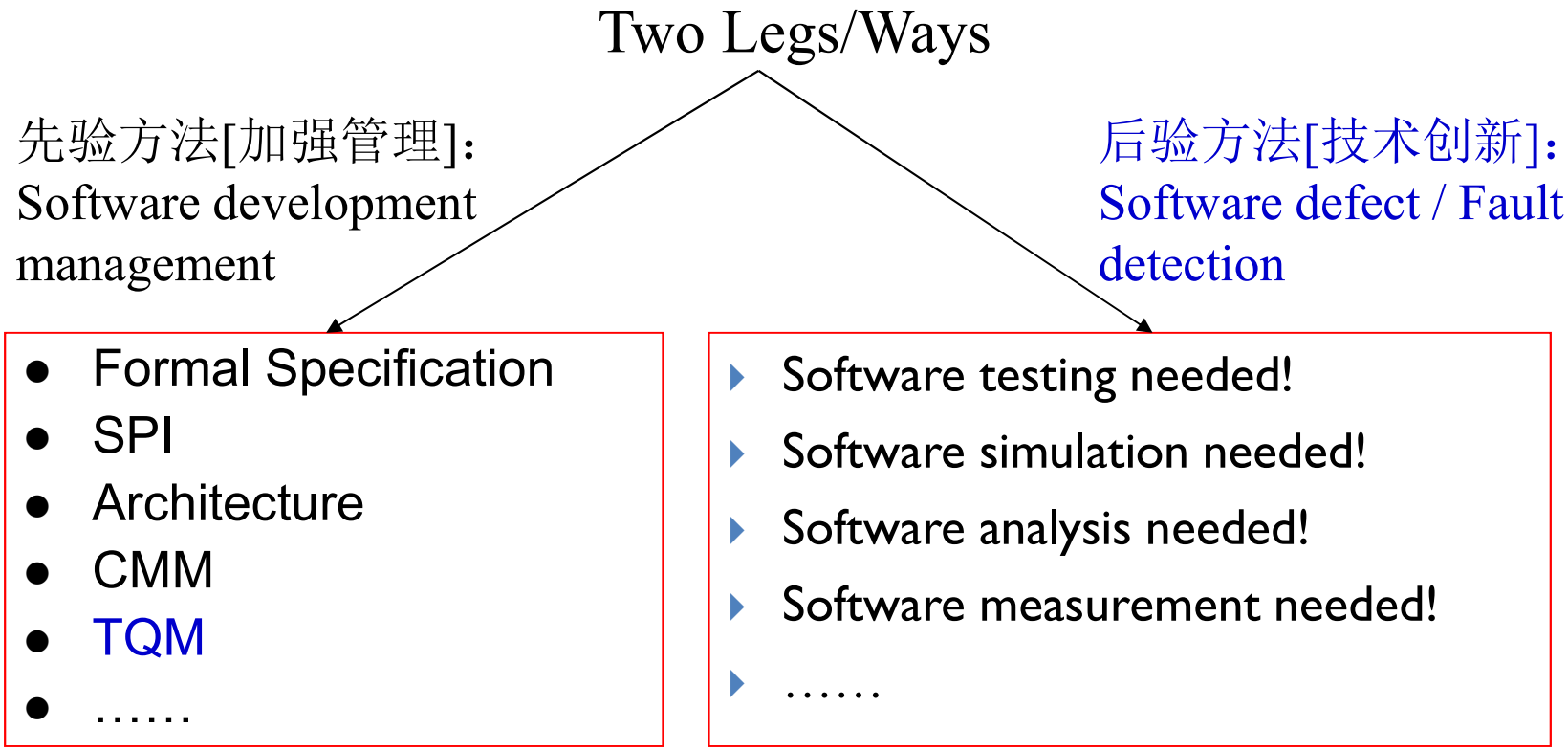


单个生命周期中每个阶段的质量保障！



多生命周期中每个生命周期的每个阶段的质量保障！

2 全生命周期软件质量如何保证？



关注问题：如何做好早期阶段的软件质量保证？

（2.1）早期阶段质量保障为什么重要？

- 降低风险
- 软件缺陷越早发现，修复代价越小

（2.2） 何为早期阶段？

- 分析阶段
  - 应用需求分析阶段（现实世界）：草图、非结构化文档
  - 软件需求分析阶段（计算机世界）：结构化文档、UC、SRS
- 设计阶段：
  - 架构设计
  - 模块设计
  - 数据库设计
  - 接口设计
  - 算法设计

（2.3） 早期软件质量保障方法

- 评审
- 分析
- 度量
- 评估
- 测试
- 验证
- 仿真
- 知识推理
- 人工智能

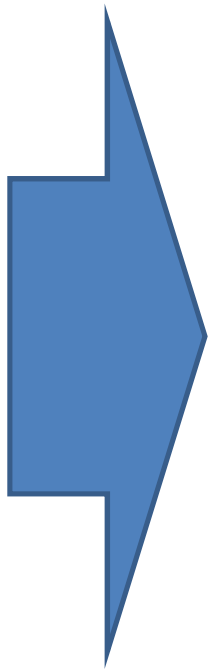
静态和动态

黑盒和白盒

定性和定量

形式化和非形式化

局部和整体

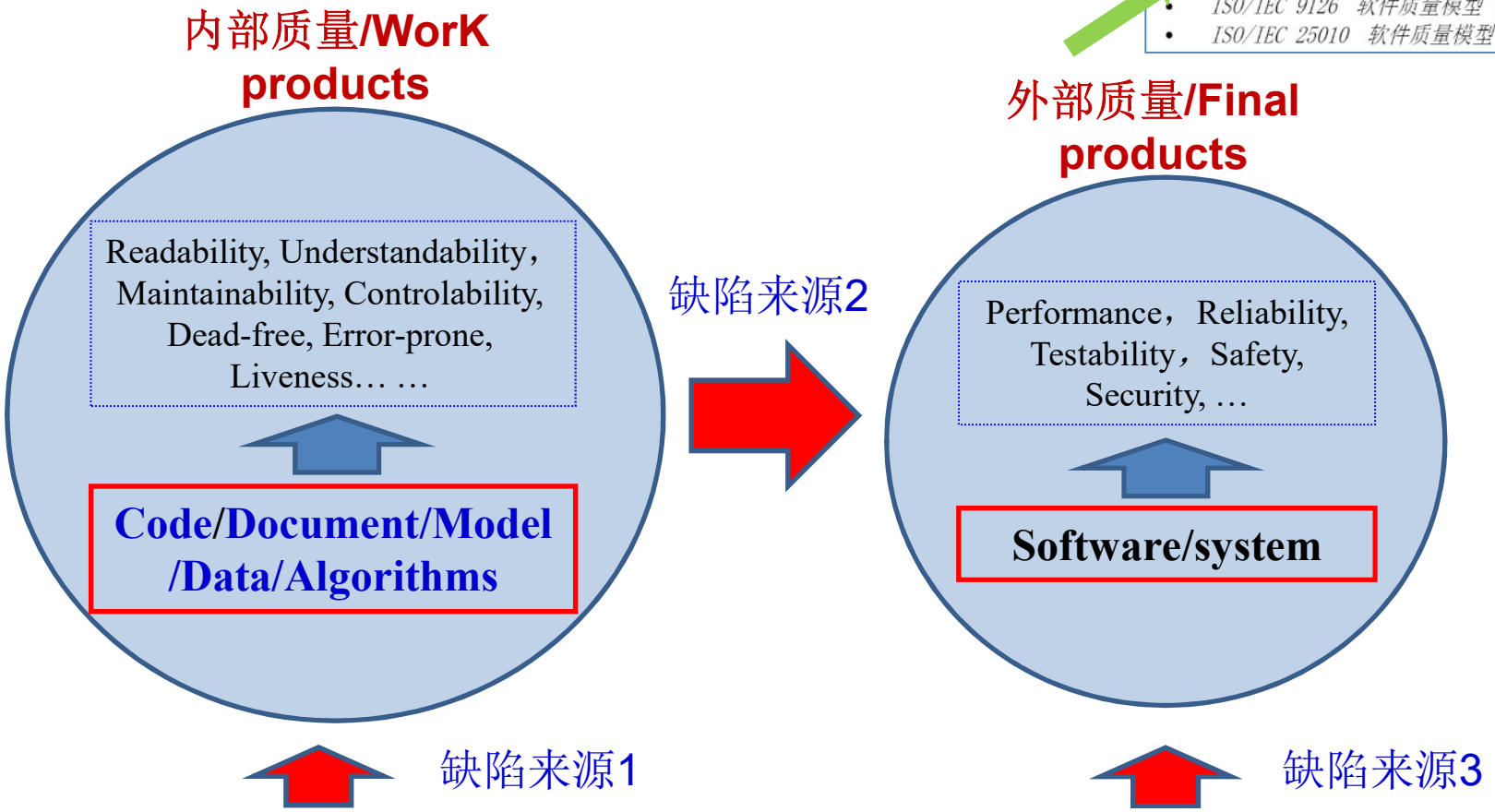


- 可靠性
- 安全性
- 正确性
- 易用性
- 可维护性
- 互操作性
- ……

测试：specification based testing

3 软件质量和软件缺陷的关系

- *Jim McCall 软件质量模型（1977 年）*
- *Barry W. Boehm 软件质量模型（1978 年）*
- *FURPS/FURPS+ 软件质量模型*
- *R. Geoff Dromey 软件质量模型*
- *ISO/IEC 9126 软件质量模型（1993 年）*
- *ISO/IEC 25010 软件质量模型（2011 年）*



开发人员、开发环境、开发语言、开发方法、开发过程…

用户、运行环境配置、安装过程、维护升级…

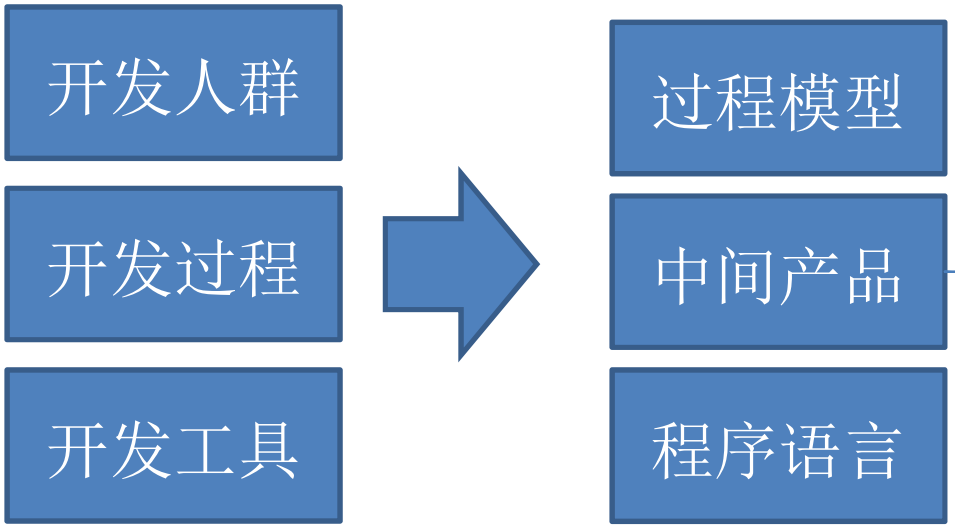
什么是软件缺陷？

- 软件缺陷（Defect），常常又被叫做Bug。所谓软件缺陷，即为计算机软件或程序中存在的某种破坏正常运行能力的问题、错误，或者隐藏的功能缺陷。
- 缺陷的存在会导致软件产品在某种程度上不能满足用户的需要。
- IEEE729-1983对缺陷有一个标准的定义：从产品内部看，缺陷是软件产品开发或维护过程中存在的错误、毛病等各种问题；从产品外部看，缺陷使得 系统所需要实现的某种功能的失效或违背，可能使软件发生故障（Faults）。

- 软件缺陷都表现成什么样子？
  - 错误：错误的算法、错误的模型、错误的代码、错误的公式、错误的单位，等等
  - 复杂度高：难以理解、不易修改、不易维护
  - Bad Smell：一些不好的设计、代码
  - 代码冗余：多余的、重复的代码
  - 不可行路径：从来不会被执行的程序路径
  - ……

Defect
Bug
Flaw
Error
Fault
Bad smell
Technical debt
Weakness
Vulnerability
……

- 软件缺陷都分布在那里？



- 应用需求文档
- 软件需求文档
- 设计文档
  - 架构设计
  - 模块设计
  - 算法设计
- 源代码
- 数据
- 模型
- 测试文档
  - 测试计划
  - 测试用例
  - 测试脚本
- 维护文档

Software Defect Detection

- 数据缺陷检测
  - 结构化数据类型缺陷
  - 半结构化数据类型缺陷
  - 非结构化数据类型缺陷

规范性、完整性、准确性、一致性、时效性、可访问性、多样性

- 文档缺陷检测
  - 用户需求文档
  - 软件需求文档
  - 架构设计文档
  - 模块设计文档
  - 接口设计文档
  - 数据库设计文档
  - 各种测试文档
  - 帮助文档
  - 用户使用说明书

规范性、完整性、准确性、一致性、无冗余、即时更新、模棱两可、遗漏、不确定性、有用性

- 模型缺陷检测
  - 需求模型
    - 用例模型
    - 领域模型
  - 设计模型
    - 交互模型
    - 功能模型
    - 接口模型
  - 实现模型
    - 算法模型
    - 数据模型

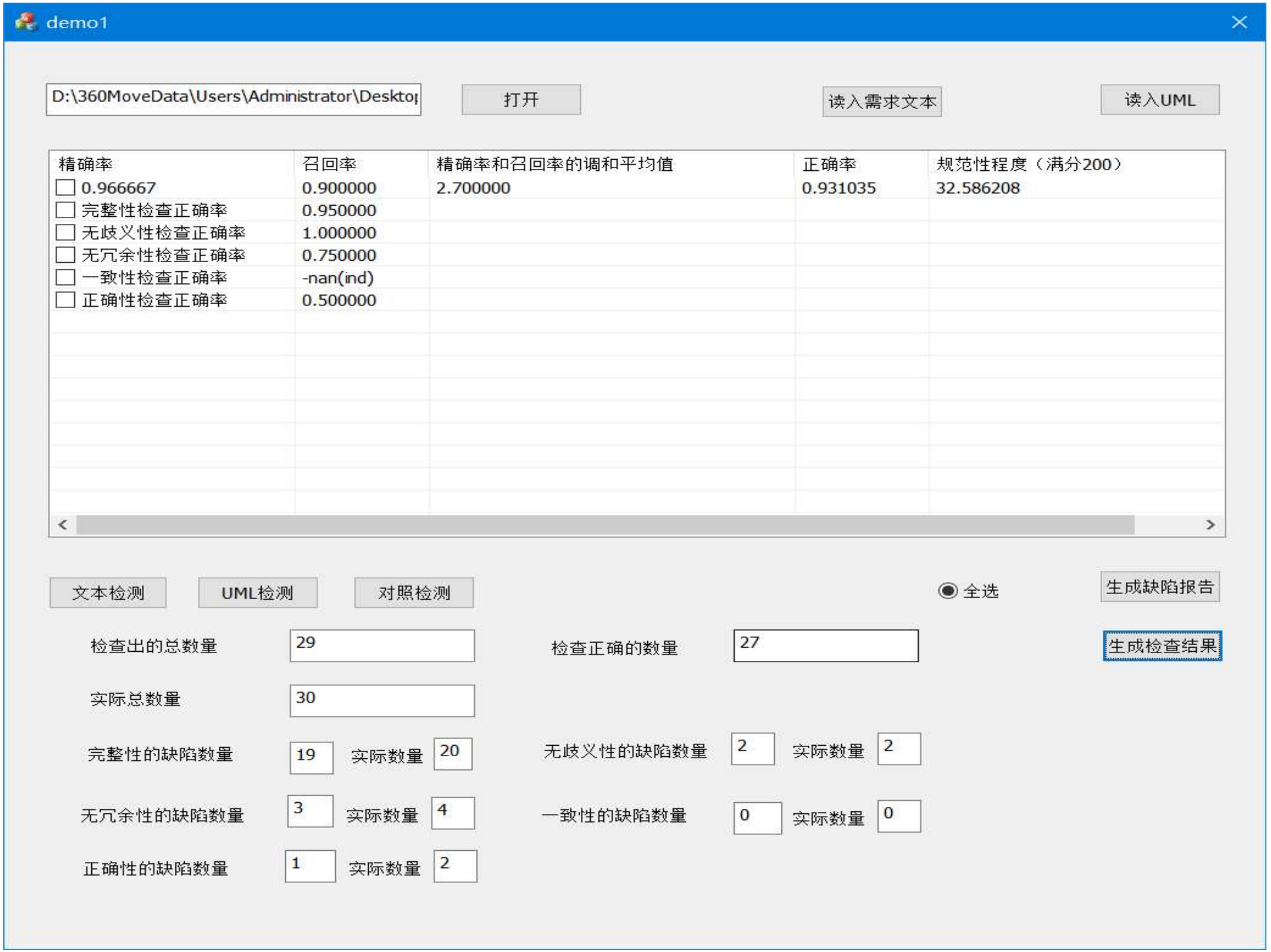
一致性、互操作性、兼容性、完整性、准确性、易理解性、可扩展性、可修改性、可维护性、复杂性、循环依赖

- 代码缺陷检测
  - 不同语言源代码
  - 中间码
  - 二进制码
- 算法缺陷检测

规范性、BUG、ERROR、Bad Smell、Technical Debt、代码漏洞、冗余代码，等等

示例： 需求文档检测

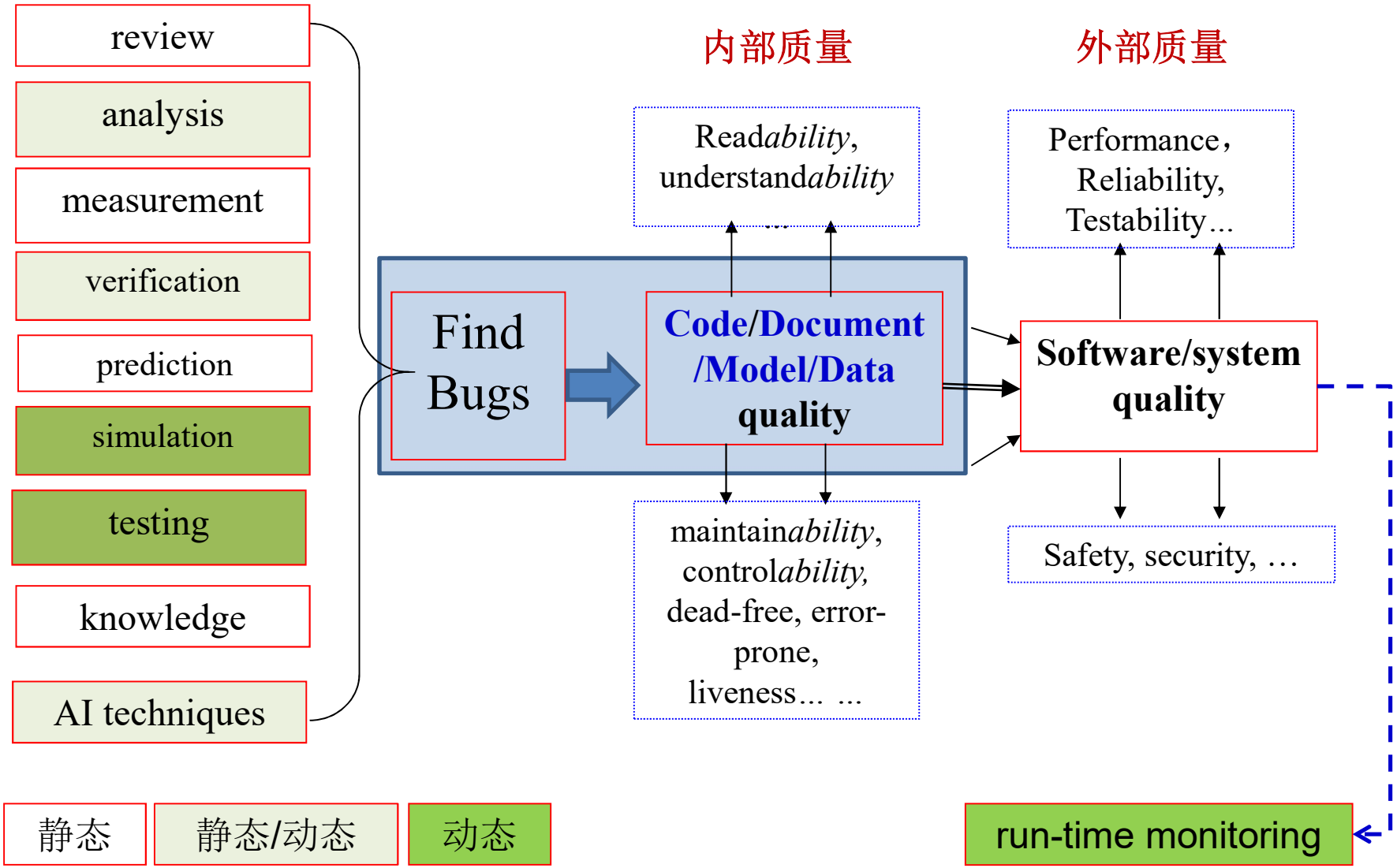

demo1
D:\360MoveData\Users\Administrator\Desktop
打开
读入需求文本
读入UML
缺陷内容
缺陷位置
缺陷名
规范性|权值
3.2.2.m.1输入数据实体
3.2.2.m.2过程算法或公式
文本描述内容遗漏
完整性|5
3.2.2.m.2过程算法或公式
3.2.2.m.3受影响的数据实体
文本描述内容遗漏
完整性|5
3.2.2.m.3受影响的数据实体
3.2.3数据构建规范
文本描述内容遗漏
完整性|5
3.2.3.p构建p
3.2.3.p.1记录类型
文本描述内容遗漏
完整性|5
3.2.3.p.1记录类型
3.2.3.p.2组成字段
文本描述内容遗漏
完整性|5
3.2.3.p.2组成字段
3.2.4数据词典
文本描述内容遗漏
完整性|5
3.2.4.q数据元素q
3.2.4.q.1名称
文本描述内容遗漏
完整性|5
3.2.4.q.1名称
3.2.4.q.2表示法
文本描述内容遗漏
完整性|5
3.2.4.q.2表示法
3.2.4.q.3单位／格式
文本描述内容遗漏
完整性|5
3.2.4.q.3单位／格式
3.2.4.q.4精确度／准确度
文本描述内容遗漏
完整性|5
3.2.4.q.4精确度／准确度
3.2.4.g.5范围
文本描述内容遗漏
完整性|5
3.2.4.g.5范围
3.3性能需求
文本描述内容遗漏
完整性|5
本部分一定可能应当提供整个SRS的概述
4附录12
文本模糊
歧义性|5
a)系统接口：这是一个系统的接口。/a)系统接口...
2.2产品描述/2.1.1系统接口
文本不一致
一致性|10
注：有时此条规定作为用户界面的一部分。/注：...
2.1.7操作/3.3性能需求
文本不一致
一致性|10
文本检测
UML检测
对照检测
全选
生成缺陷报告
检查出的总数量
29
检查正确的数量
27
生成检查结果
实际总数量
28
完整性的缺陷数量
19
实际数量
20
无歧义性的缺陷数量
2
实际数量
2
无冗余性的缺陷数量
3
实际数量
4
一致性的缺陷数量
0
实际数量
0
正确性的缺陷数量
1
实际数量
2

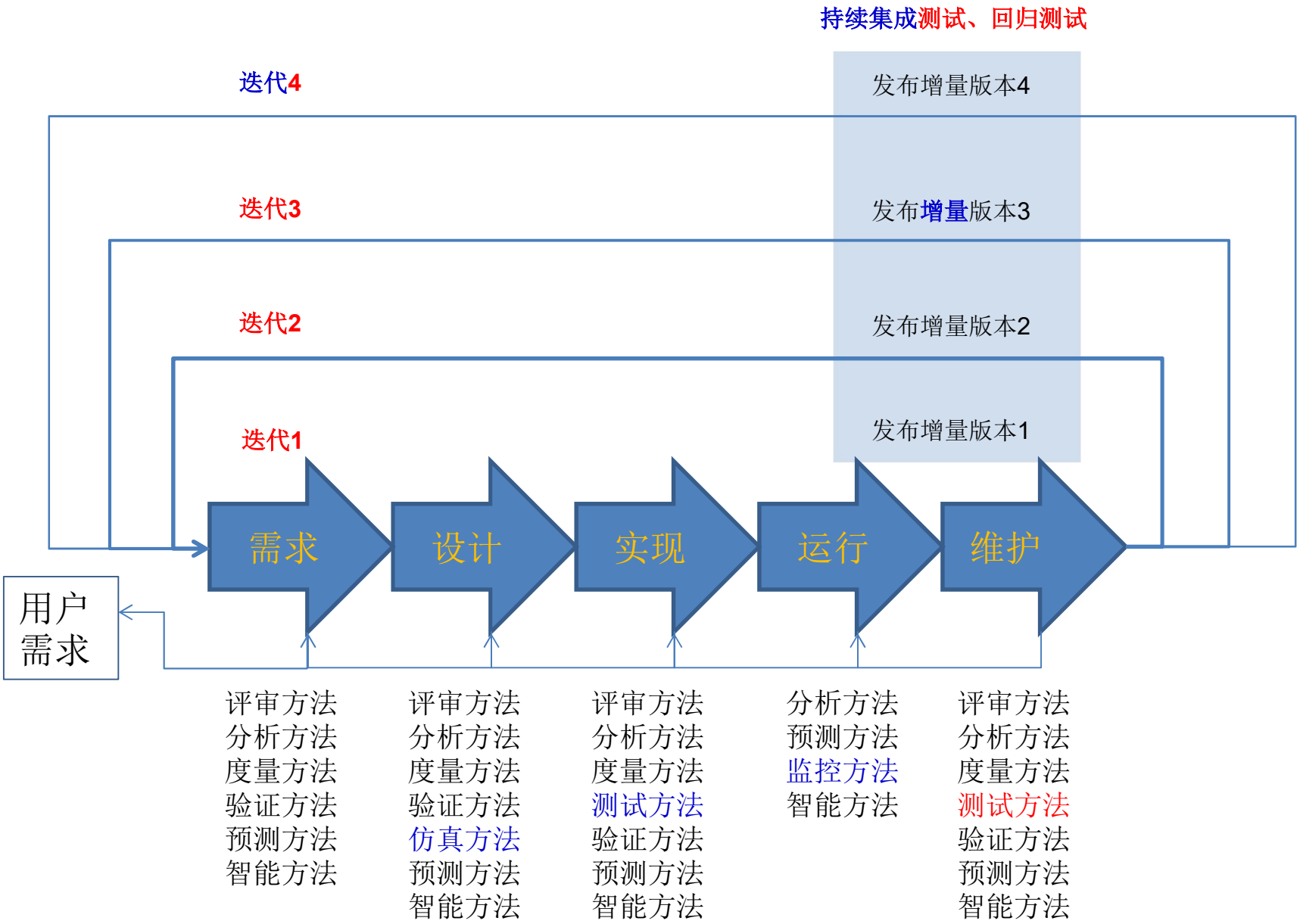

demo1
D:\360MoveData\Users\Administrator\Desktop
打开
读入需求文本
读入UML
精确率
召回率
精确率和召回率的调和平均值
正确率
规范性程度（满分200）
0.966667
0.900000
2.700000
0.931035
32.586208
完整性检查正确率
0.950000
无歧义性检查正确率
1.000000
无冗余性检查正确率
0.750000
一致性检查正确率
-nan(ind)
正确性检查正确率
0.500000
文本检测
UML检测
对照检测
全选
生成缺陷报告
检查出的总数量
29
检查正确的数量
27
生成检查结果
实际总数量
30
完整性的缺陷数量
19
实际数量
20
无歧义性的缺陷数量
2
实际数量
2
无冗余性的缺陷数量
3
实际数量
4
一致性的缺陷数量
0
实际数量
0
正确性的缺陷数量
1
实际数量
2

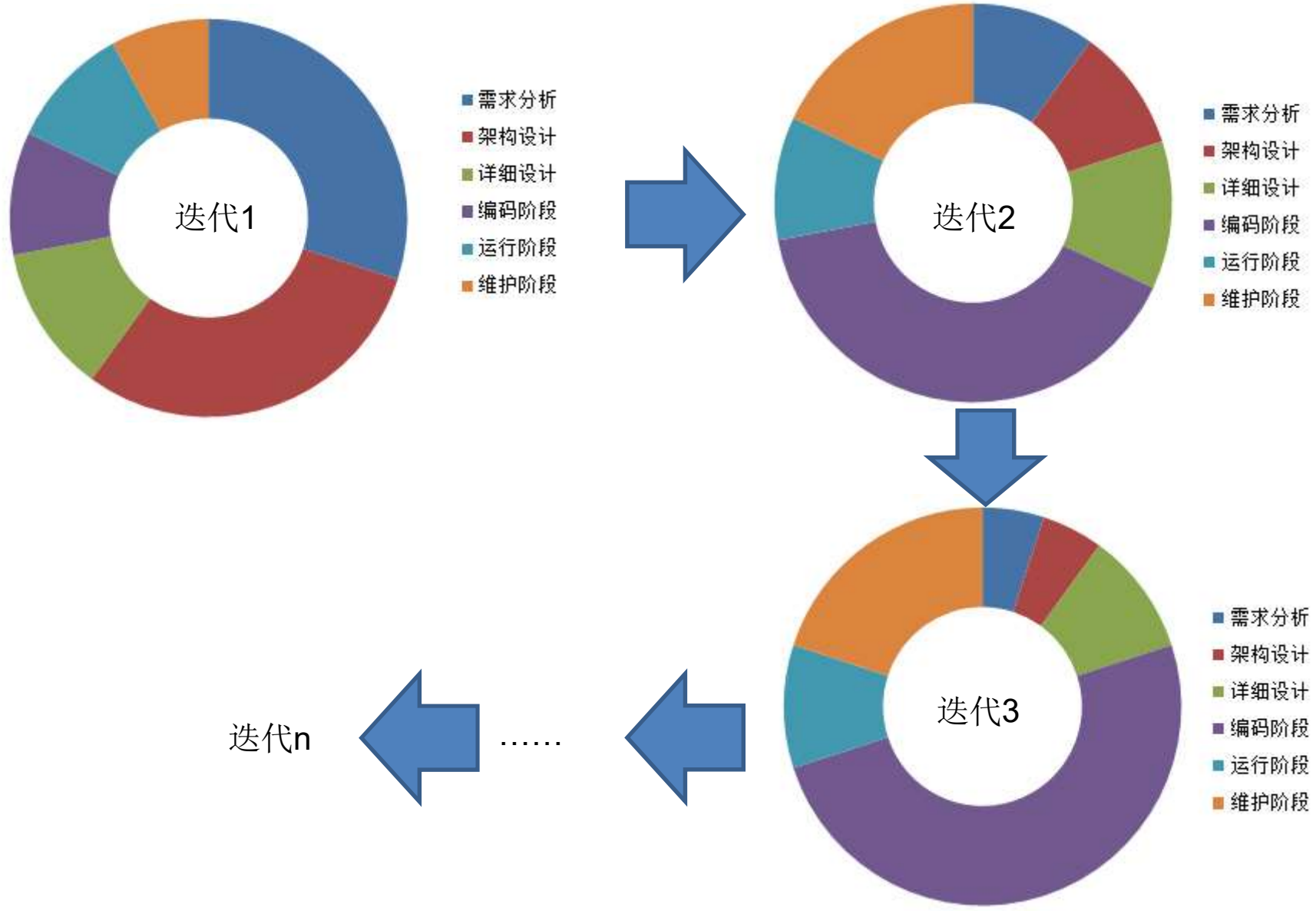
四 软件全方位缺陷检测的主流方法

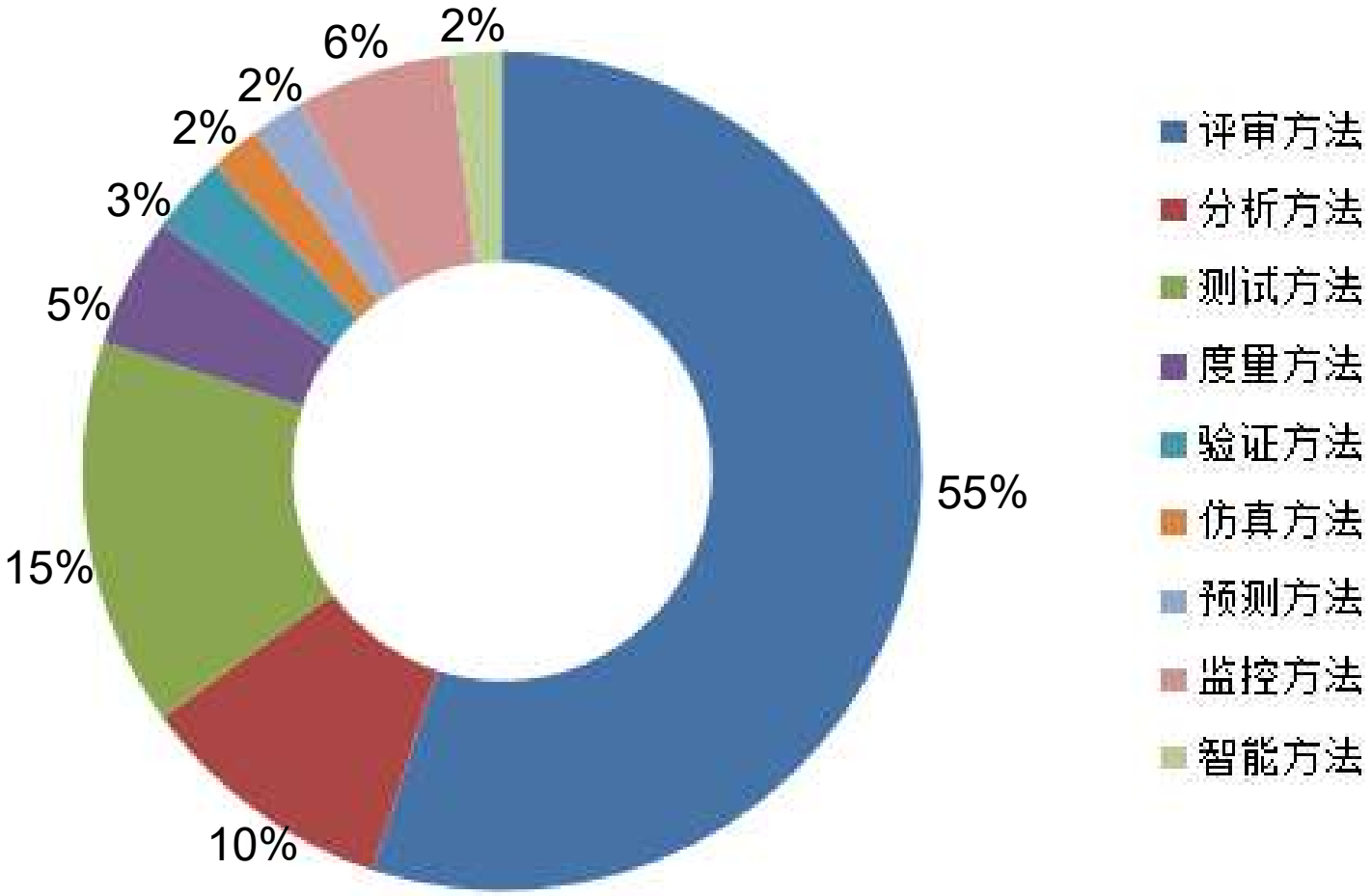
（一）全方位缺陷检测
（二）软件缺陷检测的9种主流方法

（一）全方位软件缺陷检测


review
analysis
measurement
verification
prediction
simulation
testing
knowledge
AI techniques
内部质量
外部质量
Readability, understandability
Performance，Reliability, Testability…
Find Bugs
Code/Document /Model/Data quality
Software/system quality
maintainability, controlability, dead-free, error-prone, liveness… …
Safety, security, …
静态
静态/动态
动态
run-time monitoring


持续集成测试、回归测试
迭代4
发布增量版本4
迭代3
发布增量版本3
迭代2
发布增量版本2
迭代1
发布增量版本1
需求
设计
实现
运行
维护
用户需求
评审方法
分析方法
度量方法
验证方法
预测方法
智能方法
评审方法
分析方法
度量方法
验证方法
仿真方法
预测方法
智能方法
评审方法
分析方法
度量方法
测试方法
验证方法
预测方法
智能方法
分析方法
预测方法
监控方法
智能方法
评审方法
分析方法
度量方法
测试方法
验证方法
预测方法
智能方法


迭代1
需求分析
架构设计
详细设计
编码阶段
运行阶段
维护阶段
迭代2
需求分析
架构设计
详细设计
编码阶段
运行阶段
维护阶段
迭代3
需求分析
架构设计
详细设计
编码阶段
运行阶段
维护阶段
……
迭代n


55%
10%
15%
5%
3%
2%
2%
6%
2%
评审方法
分析方法
测试方法
度量方法
验证方法
仿真方法
预测方法
监控方法
智能方法

（二）软件缺陷检测的9种主流方法

1. **评审方法**：利用走查、检查单、审计、代码阅读等方式进行人工或自动评审，发现描述规范性、完整性、一致性、冗余等方面的缺陷。
2. **分析方法**：从控制流分析、数据流分析、代码坏味道检测、修改影响分析、路径剖析的角度进行代码层面的缺陷检测。
3. **度量方法**：度量设计和代码的好坏，发现设计和代码的缺陷。
4. **验证方法**：从模型检验的角度检测和定位系统的安全性、一致性等时态属性存在的缺陷。
5. **仿真方法**：通过仿真找出系统设计的性能缺陷。
6. **测试方法**：从软件功能测试和非功能测试进行软件缺陷检测。
7. **监测方法**：通过软件运行过程中各种数据的监测，可以发现软件中存在的缺陷。
8. **基于知识方法**：利用领域知识检查设计和代码中存在的缺陷。
9. **智能化方法**：利用机器学习的方法发现缺陷。

1 评审方法

什么是软件评审？

在软件开发中，软件评审（Software Review） 是一种系统化的质量保障活动，通过团队协作对软件相关文档、设计、代码或流程进行检查，目的是尽早发现缺陷、优化实现方案，并确保交付成果符合需求和质量标准。它是软件工程中预防缺陷扩散和降低开发风险的核心实践。

软件评审的核心目标

1. 发现缺陷：在需求、设计、代码等阶段暴露错误，避免后期修复的高成本。
2. 验证一致性：确保技术方案与需求对齐，避免偏离目标。
3. 知识共享：促进团队成员对系统设计的共同理解。
4. 合规性检查：符合行业标准（如安全、性能规范）和团队开发规范。

软件评审的常见方法

1. 按形式化程度分类：正式评审(Formal Review)、非正式评审(Informal Review)和技术审查(Inspection)
2. 按评审对象分类：需求评审、设计评审、代码评审、测试用例评审、管理评审、过程评审

软件评审的三个主要阶段

准备阶段：

明确评审目标（如评审某个模块的API设计）；
提前分发评审材料（设计文档、代码链接、测试计划等）；
确定评审参与角色（主持人、记录员、评审专家）。

会议阶段：

主持人引导讨论，确保聚焦技术问题（避免发散）；
记录员汇总问题（如设计缺陷、代码坏味道、潜在风险）；
示例问题，设计：服务间通信未考虑超时机制，可能导致级联故障。代码：循环中频繁创建对象，存在内存泄漏风险。

跟踪与闭环：

生成评审报告（问题列表、责任人、修复期限）；
验证问题是否解决（如通过二次评审或自动化测试）。

例如，

- 规范性检查
  - 文档规范性检查：完整性、一致性、冗余描述、错误描述、模糊描述
  - 模型规范性检查：完整性、一致性
  - 代码规范性检查：完整性、一致性、正确性
  - 数据规范性检查：完整性、一致性
  - 算法规范性检查：输入/输出、确定性、有限步骤
  - 编码规则检查：编码规则冲突
- 技术评审[technical review]
  - Walk through：技术合理性、可行性
  - Inspection：特定方面的合理性、可行性、正确性
  - Auditing：标准符合度、政治文化、安全可靠、可信
  - Code reading：人工/自动阅读代码

2 分析方法

什么是程序分析？

程序分析（Program Analysis） 是计算机科学中通过系统化方法研究程序行为、结构、性能或安全性的技术，旨在理解、验证或优化软件。它通过自动或半自动化的工具，帮助开发者发现代码缺陷、提升效率、保障安全，并辅助复杂系统的维护。程序分析是编译器设计、软件工程和安全领域的重要基础。

程序分析是软件工程的“显微镜”，通过自动化技术深入代码内部，在开发早期暴露问题，显著降低维护成本（研究显示修复需求阶段缺陷的成本是实现阶段的100倍）。随着AI技术的融入（如深度学习辅助漏洞挖掘），程序分析正从“规则驱动”转向“智能驱动”，成为构建高可靠、高性能软件的核心支柱。

程序分析的核心目标

1. 发现缺陷：识别潜在的逻辑错误、内存泄漏、安全漏洞等。
2. 优化性能：分析代码执行效率，定位瓶颈（如冗余计算、低效算法）。
3. 验证正确性：确保程序行为符合预期（如并发程序的线程安全性）。
4. 辅助重构：分析代码依赖关系，支持模块化或架构调整。
5. 安全审计：检测恶意代码或合规性问题（如未加密的敏感数据传输）。

程序分析的分类

1. 按分析时机：

1）静态分析（Static Analysis）：不运行程序，直接分析源代码或二进制文件；覆盖所有代码路径，提前发现潜在问题；可能存在误报（False Positives）； SonarQube（代码质量）Clang Static Analyzer。

2）动态分析（Dynamic Analysis）：运行程序，观察其实际行为（如输入输出、内存使用）；捕捉运行时行为（如内存泄漏、竞态条件）;仅覆盖实际执行的代码路径； Valgrind（内存检测）、Profiler（性能分析）。

2.按分析深度：

1）语法分析：检查代码是否符合语言规范（如缺少分号、括号不匹配）。

2）语义分析：验证逻辑正确性（如类型错误、未初始化变量）。

3）控制流分析：追踪程序执行路径（如死代码、无限循环）。

4）数据流分析：跟踪变量值的传播（如空指针解引用、未使用变量）。

示例：静态分析发现空指针异常

```
// Java原始代码
public void processData(String input) {
    int length = input.length(); // 若input为null，此处抛出NullPointerException
    System.out.println("Length: " + length);
}


// 静态分析报告：
// Warning: Parameter 'input' may be null at line 2.
```

常见的程序分析类型

- 静态分析方法（static analysis）
  - 控制流分析（CFA）
  - 数据流分析（DFA）
  - 关联关系分析
  - 调用依赖关系分析（CRA）
  - 层次依赖关系分析（HRA）
  - 数据依赖关系分析（dependence relationship）
  - 控制依赖关系分析（dependence relationship）
  - 传递依赖关系分析（dependence relationship）
  - 循环依赖关系分析（dependence relationship）
  - 静态程序切片
  - 修改影响分析（change impact analysis）
  - ……
- 动态分析方法（dynamic analysis）
  - 修改传播分析（change propagation analysis）
  - 路径剖析（path profiling）
  - 执行轨迹跟踪
  - 动态切片
  - ……
- 混合分析方法（hybrid analysis）
  - 符号执行
  - 半静态程序切片
  - 条件程序切片
  - ……

控制流分析

1.什么是控制流分析？

控制流分析（Control Flow Analysis，CFA）是程序分析中的一种技术，主要用于研究程序代码的执行顺序和逻辑路径。它通过分析程序中的控制结构（如条件分支、循环、函数调用等），推断程序在运行时的可能行为，从而支持优化、错误检测或安全验证等任务。

2. 什么是控制流图？

控制流图（Control Flow Graph, CFG）： 将程序分解为基本块（Basic Block），每个基本块是一段顺序执行的代码（无跳转）； 用有向边表示基本块之间的跳转关系（如if 分支、循环、函数调用等）。

例如：python程序片段

```
if x > 0: print("正数")
else:     print("非正数")
```

对应的 CFG 会有两个分支，分别表示 if 和 else 的路径。

3. 典型控制流结构

顺序执行：代码按顺序逐行执行。

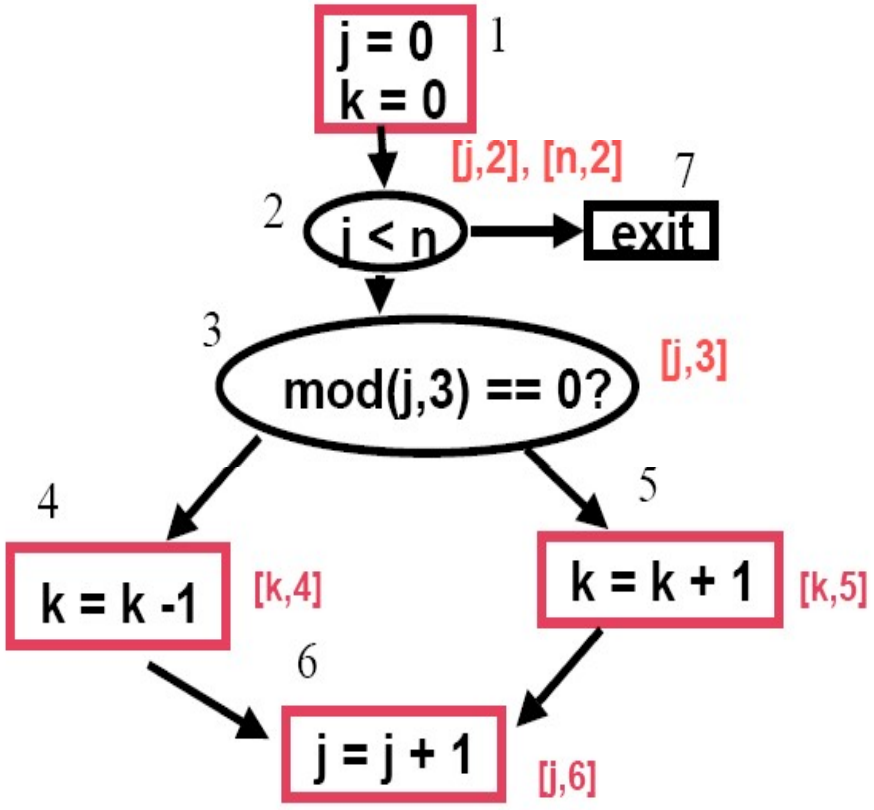
条件分支：如 if-else、switch语句等。

循环：如 for、while语句等。

函数调用：可能涉及跨函数的控制流跳转（需过程间分析）。

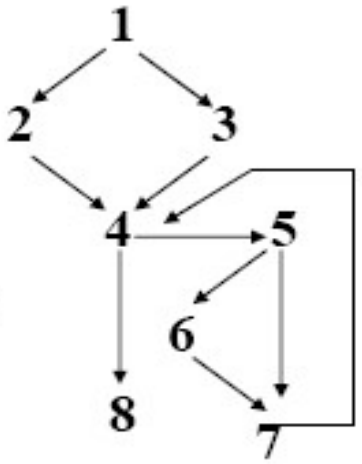
Control Flow Graph

```
1. j = 0
2. k = 0
3. if (j<n) goto 7
4. if (mod(j,3)==0)
            then k=k-1
            else k=k+1
5. j = j+1
6. goto 3
7.
```



Applications of Control Flow

- Testing: branch, path, basis path
  - Branch: must test 1→2, 1→ 3, 4→5, 4→8, 5→6, 5→7
  - Path: infinite number because of loop
  - Basis path: set of paths such that each path executes at least one more edge (cyclomatic complexity gives max necessary); example: 1,2,4,8; 1,3,4,5,6,7,4,8



- **Measurement**:
  - 入度/出度fan-in/fan-out
  - 耦合度coupling
  - 圈复杂度McCabe's Cyclomatic Complexity: CC=number of basis paths
  - ……..

示例：不可达代码检测[javascript代码]

```
function example(x) {
    return x * 2;
    console.log("这行代码永远不会执行"); // 控制流分析可标记为“不可达”
}
```

控制流分析会发现 return语句后的代码无法执行，提示开发者删除。

数据流分析

什么是数据流分析？

数据流分析（Data Flow Analysis，DFA）是程序分析中的一种技术，旨在追踪程序中数据的定义、使用和传播过程，分析变量或表达式在程序执行时的可能状态（如值、生命周期、依赖关系等）。它通过静态或动态方法，推导数据在程序中的流动路径，从而支持代码优化、错误检测和安全验证。

核心作用

（1）优化代码：删除无用赋值（如未使用的变量）、删除复用重复计算（如公共子表达式消除）。

（2）检测错误：发现未初始化变量、空指针引用、内存泄漏等。

（3）安全分析：追踪敏感数据（如密码）的传播路径，防止泄露。

（4）并行化支持： 分析数据依赖关系，识别可并行执行的代码区域。

几个核心概念

1. 数据流图（Data Flow Graph, DFG）

将程序表示为数据流动的节点和边，节点对应程序操作（如赋值、运算），边表示数据依赖关系。

例如：python代码

```
a = 1        //定义变量a
b = a + 2    //使用a的值
```

此处，`b` 的值依赖于 `a` 的定义。

2. 数据流属性

-到达定义（Reaching Definitions）：分析某个变量的定义（如赋值语句）能“到达”哪些使用点。
-活跃变量（Live Variables）：判断变量在某个程序点是否可能被后续代码使用。
-可用表达式（Available Expressions）：判断表达式在某个程序点是否已被计算且未被修改。

3. 传递函数（Transfer Function）

-描述程序操作（如赋值、条件分支）如何影响数据流状态。
-例如：x = y + z 会生成 x 的新定义，并依赖 y 和 z 的当前值。

数据流分析

- For structured programs, data-flow analysis can be performed on an AST; in general, intra-procedural (global) data-flow analysis performed on the CFG
- Exact solutions to most problems are un-decidable

--- e.g.,

---May depend on input

---May depend on outcome of a conditional statement

---May depend on termination of loop

Data-flow analysis is approximate

- Approximate analysis can overestimate the solution:
  - Solution contains actual information plus some spurious information but does not omit any actual information
  - This type of information is safe or conservative
- Approximate analysis can underestimate the solution:
  - Solution may not contains all information in the actual solution
  - This type of information in unsafe
- For optimization, need conservative, safe analysis
- For software engineering tasks, may be able to use unsafe analysis information
- Biggest challenge for data-flow analysis: how to provide safe but precise (i.e., minimize the spurious information) information in an efficient way

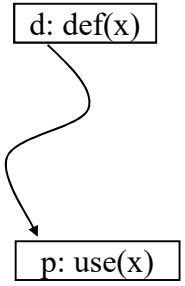
Two kinds of DFA

- Reaching Definitions
  - for each basic block (program statements) s find which of all definitions in the program reach the boundaries of s.

先说明什么是"可达的定义"：在程序点p的一个定义d，到程序点q是可达的，当且仅当这个定义在 $p \rightarrow q$ 这条路径上不会被kill掉。

注意这里说的一个变量的定义(definition)即是指为这个变量设置值的statement，在程序语言中初始化和赋值都算是definition。

也就是说变量v在p处有definition，然后在 $p \rightarrow q$ 不能被新的definition盖掉，这样它（指p处v的definition，而不是v本身）在这条路径上就是**可达定义(reaching definition)**。



- Liveness analysis
  - a variable is live at a **particular point** in the program if its value at that point will be used in the future (dead, otherwise) ,so, to compute liveness at a given point, need to look into the future

核心概念

1. **活跃变量定义**
   变量在程序点 P 是活跃的，当且仅当存在一条从 P 出发的路径，在该路径中变量在重新定义之前被使用。
2. **分析方向**
   采用 **后向分析**（从程序出口向入口方向），因为需要判断变量是否在未来的路径中被使用。
3. **数据流方程**
   - **LiveOut(B)**：基本块 B 出口处的活跃变量集合，等于所有后继块入口活跃变量的并集：

$$\text{LiveOut}(B) = \bigcup_{S \in \text{succ}(B)} \text{LiveIn}(S)$$

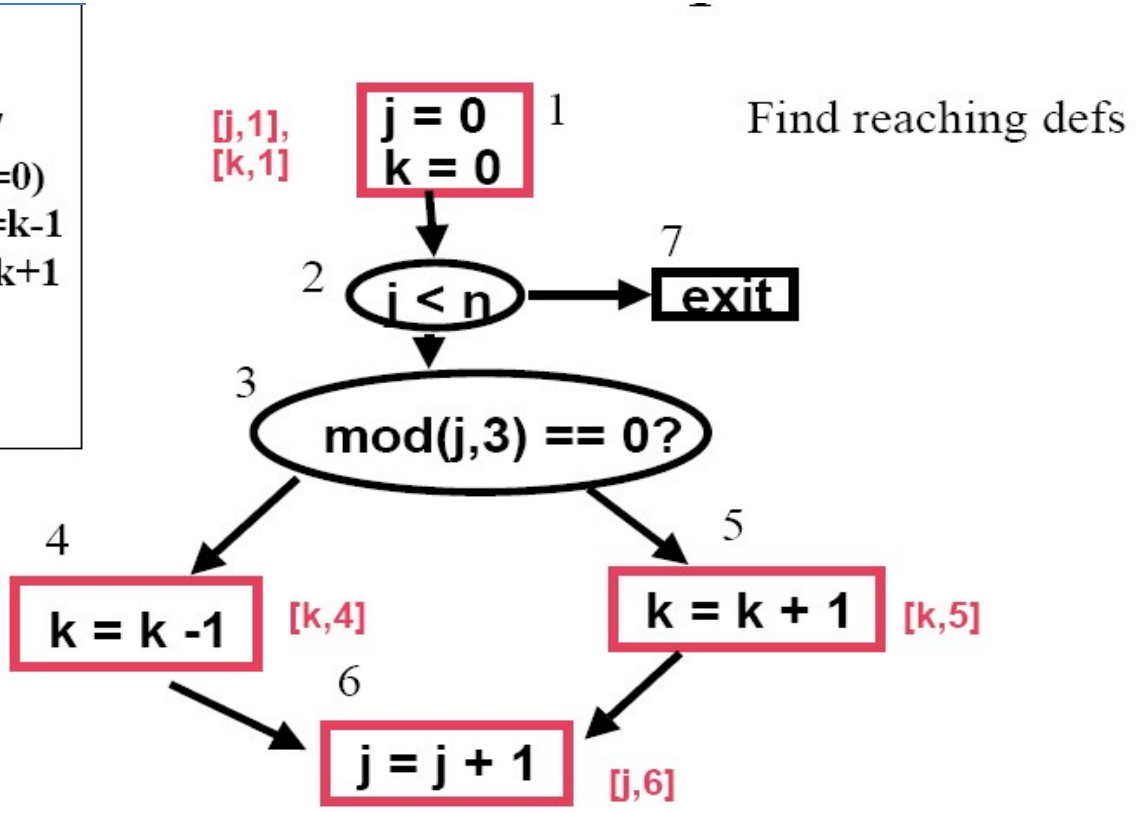
   - **LiveIn(B)**：基本块 B 入口处的活跃变量集合，由以下公式计算：

$$\text{LiveIn}(B) = \text{Use}(B) \cup (\text{LiveOut}(B) \setminus \text{Def}(B$$

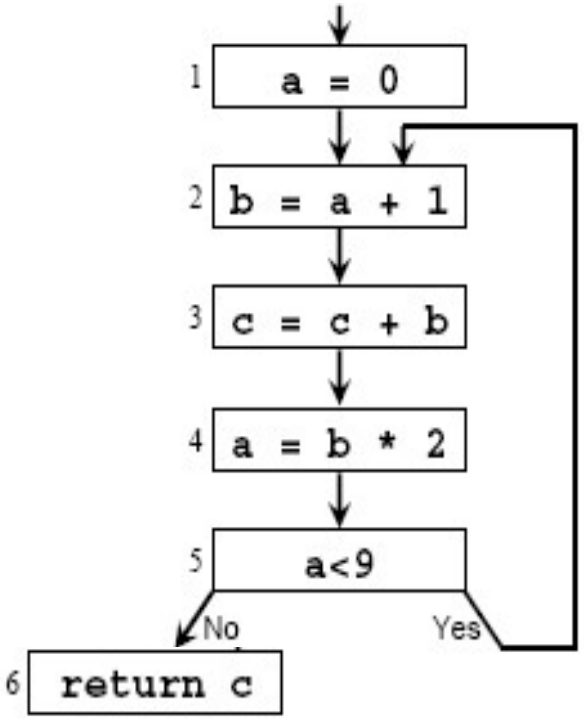
   - **Use(B)**：块 B 中在定义前被使用的变量。
   - **Def(B)**：块 B 中定义的变量。

Example for Reaching Definitions

```
1. j = 0
2. k = 0
3. if (j<n) goto 7
4. if (mod(j,3)==0)
            then k=k-1
            else k=k+1
5. j = j+1
6. goto 3
7.
```



Example for Liveness



What is the live range of variables a, b and c?

Answer:

Live range of a: 1-2

Live range of b: 2-3-4

Live range of c:

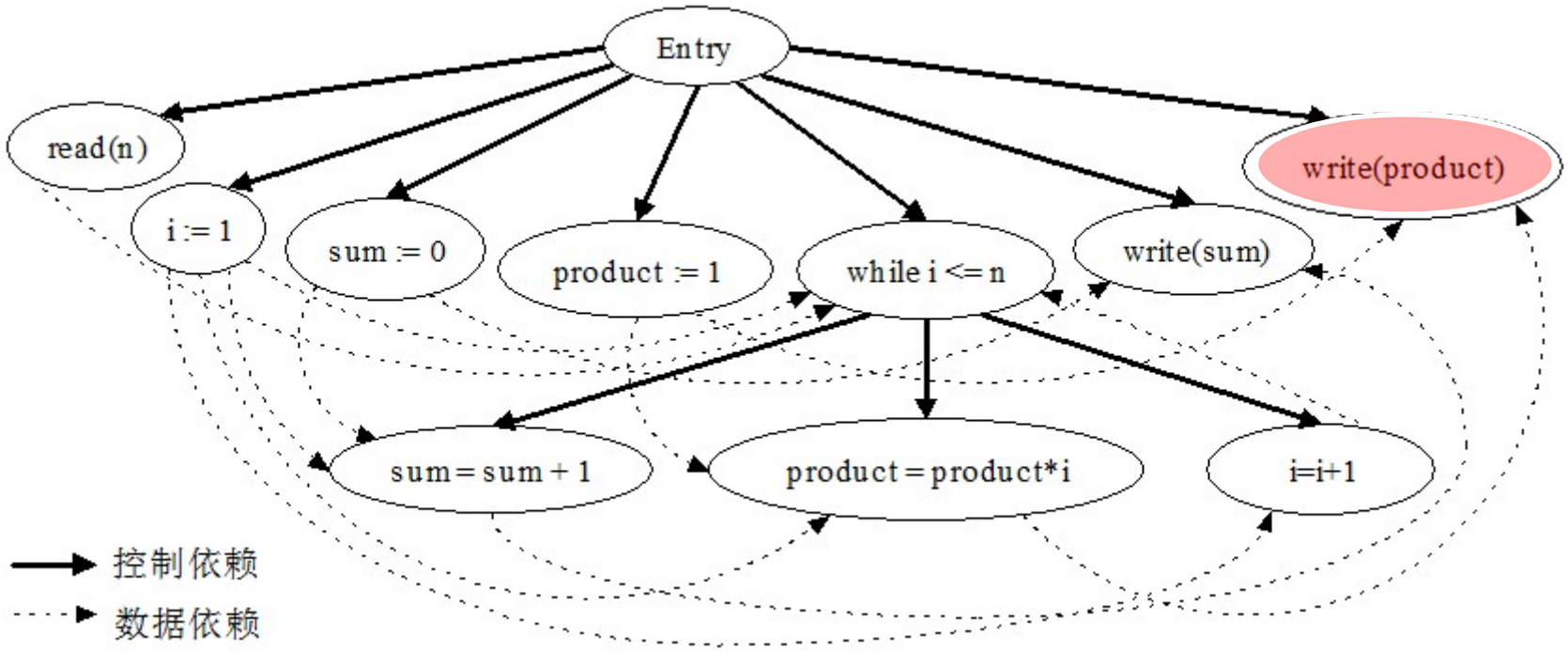
程序切片

- 给定某个程序的兴趣点和兴趣变量(或变量集合),即二元组(S,V),观察程序中哪些语句对S处的变量V有影响;或者观察S处的变量V对程序中哪些语句有影响,等等.

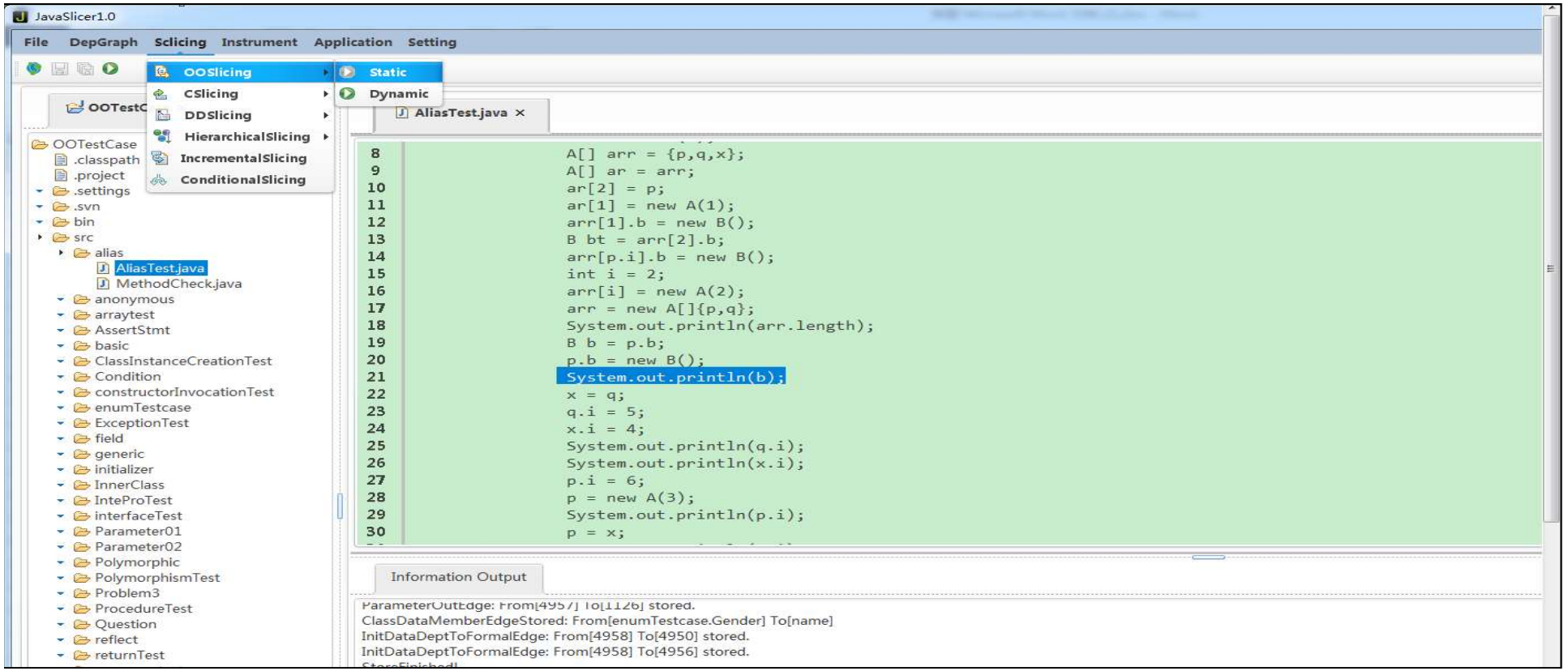
例 过程内切片例子源程序。

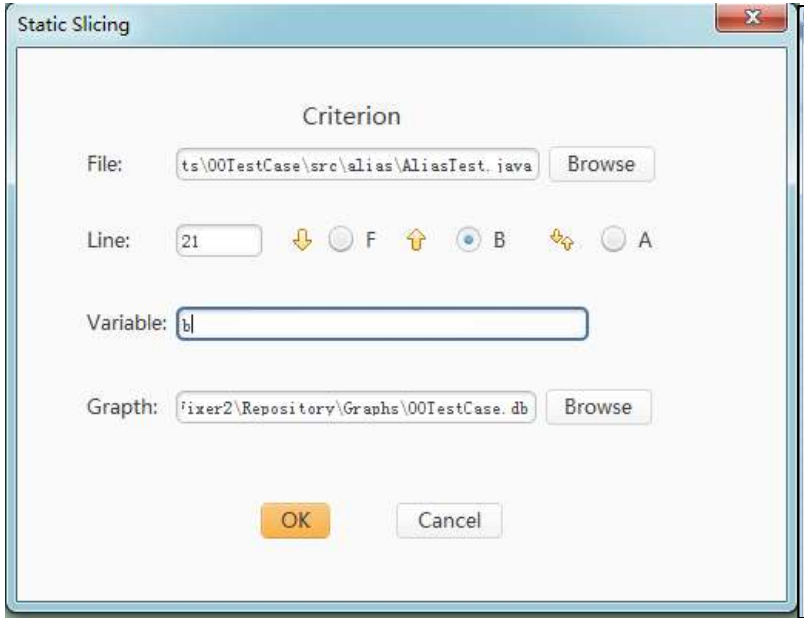
```
1  read(n);
2  i:=1;
3  sum:=0;
4  product:=1;
5  while i<=n do
6  begin
7    sum:=sum+1;
8    product:=product*i;
9    i:=i+1;
10 end;
11 write(sum);
12 write(product)
```

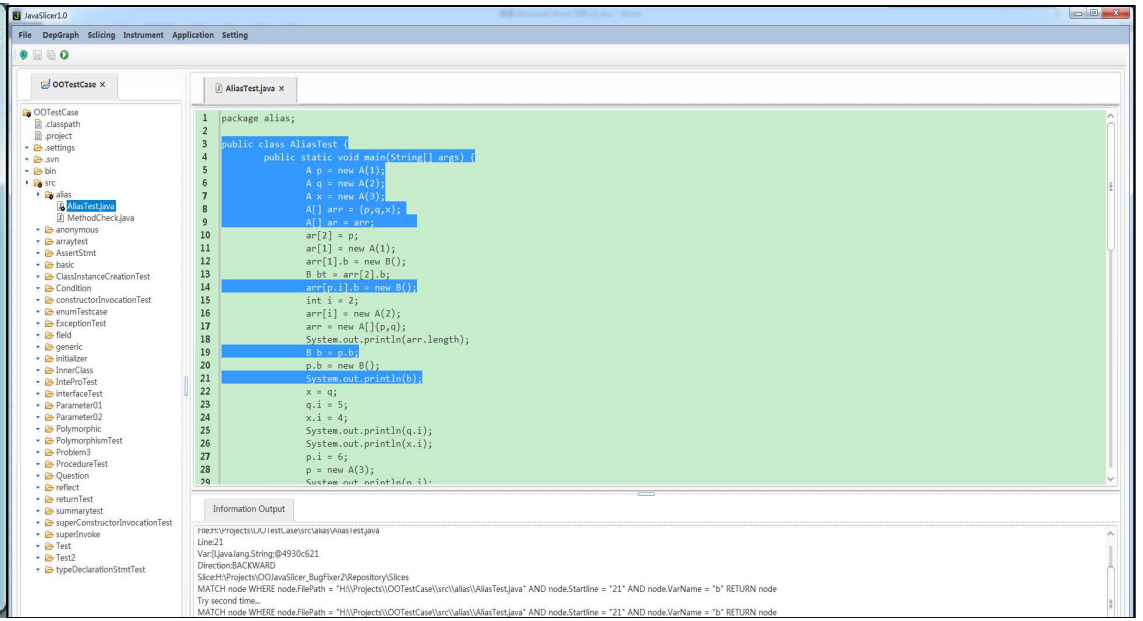
- ✓ 静态切片、半静态切片、和动态切片、条件切片、无定型切片
- ✓ 正向切片和反向切片
- ✓ 过程内切片和过程间切片
- ✓ 并发程序切片和分布式程序切片
- ✓ 切片、砍片、削片、碎片
- ✓ 程序切片、规约切片、模型切片
- ✓ …….



具体计算过程内程序切片的步骤是：1）先构造例子程序的程序依赖图（如上图2所示），其中含有write(product)的节点代表切片准则所在的位置。2）图可达性算法的思想就是：找出所有从依赖图的入口节点Entry到切片准则节点write(product)的所有路径，把这些路径所有经过的节点标记出来，这些标记节点构成的集合就是该程序关于切片准则<12, product>的一个（过程内）依赖图切片，这些节点对应的源程序的语句和控制谓词构成的集合就是该程序关于切片准则<12, product>的程序切片。最终得到的切片是由例子源程序中黑体部分语句构成的集合。


JavaSlicer1.0
File
DepGraph
Sclicing
Instrument
Application
Setting
OOSlicing
CSlicing
DDSlicing
HierarchicalSlicing
IncrementalSlicing
ConditionalSlicing
Static
Dynamic
AliasTest.java
OOTestCase
.classpath
.project
.settings
.svn
bin
src
alias
AliasTest.java
MethodCheck.java
anonymous
arraytest
AssertStmt
basic
ClassInstanceCreationTest
Condition
constructorInvocationTest
enumTestcase
ExceptionTest
field
generic
initializer
InnerClass
InteProTest
interfaceTest
Parameter01
Parameter02
Polymorphic
PolymorphismTest
Problem3
ProcedureTest
Question
reflect
returnTest
A[] arr = {p,q,x};
A[] ar = arr;
ar[2] = p;
ar[1] = new A(1);
arr[1].b = new B();
B bt = arr[2].b;
arr[p.i].b = new B();
int i = 2;
arr[i] = new A(2);
arr = new A[]{p,q};
System.out.println(arr.length);
B b = p.b;
p.b = new B();
System.out.println(b);
x = q;
q.i = 5;
x.i = 4;
System.out.println(q.i);
System.out.println(x.i);
p.i = 6;
p = new A(3);
System.out.println(p.i);
p = x;
Information Output
ParameterOutEdge: From[4957] To[1126] stored.
ClassDataMemberEdgeStored: From[enumTestcase.Gender] To[name]
InitDataDeptToFormalEdge: From[4958] To[4950] stored.
InitDataDeptToFormalEdge: From[4958] To[4956] stored.


Static Slicing
Criterion
File:
ts\OOTestCase\src\alias\AliasTest.java
Browse
Line:
21
F
B
A
Variable:
b
Grapth:
ixer2\Repository\Graphs\OOTestCase.db
Browse
OK
Cancel


JavaSlicer1.0
File
DepGraph
Sclicing
Instrument
Application
Setting
OOTestCase
AliasTest.java
Information Output

2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)

ReMoS: Reducing Defect Inheritance in Transfer Learning via Relevant Model Slicing

Ziqi Zhang*
Key Laboratory of High-Confidence Software Technologies (MOE), School of Computer Science, Peking University
Beijing, China
ziqi_zhang@pku.edu.cn

Yuanchun Li*†
Institute for AI Industry Research (AIR), Tsinghua University
Beijing, China
liyuanchun@air.tsinghua.edu.cn

Jindong Wang
Microsoft Research
Beijing, China
jindong.wang@microsoft.com

Bingyan Liu
Key Laboratory of High-Confidence Software Technologies (MOE), School of Computer Science, Peking University
Beijing, China
lby_cs@pku.edu.cn

Ding Li
Key Laboratory of High-Confidence Software Technologies (MOE), School of Computer Science, Peking University
Beijing, China
ding_li@pku.edu.cn

Yao Guo†
Key Laboratory of High-Confidence Software Technologies (MOE), School of Computer Science, Peking University
Beijing, China
yaoguo@pku.edu.cn

Xiangqun Chen
Key Laboratory of High-Confidence Software Technologies (MOE), School of Computer Science, Peking University
Beijing, China
cherry@sei.pku.edu.cn

Yunxin Liu
Institute for AI Industry Research (AIR), Tsinghua University
Beijing, China
liuyunxin@air.tsinghua.edu.cn

2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)

Static Stack-Preserving Intra-Procedural Slicing of WebAssembly Binaries

Quentin Stiévenart
Vrije Universiteit Brussel
Brussels, Belgium
quentin.stievenart@vub.be

David W. Binkley
Loyola University Maryland
Baltimore, MD, USA
binkley@cs.loyola.edu

Coen De Roover
Vrije Universiteit Brussel
Brussels, Belgium
coen.de.roover@vub.be

ABSTRACT

The recently introduced WebAssembly standard aims to be a portable compilation target, enabling the cross-platform distribution of programs written in a variety of languages. We propose an approach to *slice* WebAssembly programs in order to enable applications in reverse engineering, code comprehension, and security among others. Given a program and a location in that program, program slicing produces a minimal version of the program that preserves the behavior at the given location. Specifically, our approach is a static, intra-procedural, backward slicing approach that takes into account WebAssembly-specific dependences to identify the instructions of the slice. To do so it must correctly overcome the considerable challenges of performing dependence analysis at the binary level. Furthermore, for the slice to be executable, the approach needs to ensure that the stack behavior of its output complies with WebAssembly's validation requirements. We implemented and evaluated our approach on a suite of 8 386 real-world WebAssembly binaries, finding that the average size of the 495 204 868 slices computed is 53% of the original code, an improvement over the 60% attained by related work slicing ARM binaries. To gain a more qualitative understanding of the slices produced by our approach, we compared them to 1 956 source-level slices of benchmark C programs. This inspection helps to illustrate the slicer's strengths and to uncover potential future improvements.

WebAssembly [25] "*is a binary instruction format for a stack-based virtual machine*" [65] designed as a compilation target for high-level languages. The specification of its core has been a W3C standard since December 2019 [49]. WebAssembly was designed for the purpose of embedding binaries in web applications in a portable manner, thereby enabling intensive computations on the web. A 2021 empirical study by Hilbig et al. [30] found use cases on the web as diverse as game engines, natural language processing, and media players. Thanks to its ability to incorporate runtime functions exported by the host environment, WebAssembly has also found usage beyond web applications, broadening the value of analyses for WebAssembly. Examples include desktop applications [63], smart contracts [19], IoT back ends [27], and embedded software [52].

Program slicing [12, 66] is a program decomposition technique that, based on a specific program point called the *slicing criterion*, identifies a subprogram of the code relevant to the slicing criterion. Program slicing has numerous applications, in debugging [32, 37, 67], program comprehension [11, 16, 31, 36, 59], software maintenance [23, 26], re-engineering [14], refactoring [20], testing [4, 28, 29], reverse engineering [2, 3], tierless or multi-tier programming [45, 46], and vulnerability detection [50].

As such, there are numerous invaluable applications of slicing for WebAssembly binaries. Slicing, for example, can provide a building block for reverse engineering and for tools such as binary translators, profilers, and debuggers [14]. In terms of security, slicing can

Effectively Detecting Software Vulnerabilities via Leveraging Features on Program Slices

Xiaodong Zhang; Haoyu Guo; Zhiwei Zhang; Guiyuan Tang; Jun Sun; Yulong Shen; Jianfeng Ma

IEEE Internet of Things Journal

Year: 2025 | Early Access Article | Publisher: IEEE

Improved Quantitative Analysis of Concrete Defect Volumes Using Enhanced Point Cloud Slicing Methodology

Peng Xu; Yonglong Li; Fuqiang Gou; Yi Xia; Jialong Li; Chunyao Hou; Gang Wan; Haoran Wang

IEEE Transactions on Instrumentation and Measurement

Year: 2025 | Volume: 74 | Journal Article | Publisher: IEEE

Sliver: A Scalable Slicing-Based Verification for Information Flow Security

Xue Rao; Cong Sun; Dongrui Zeng; Yongzhe Huang; Gang Tan

IEEE Transactions on Dependable and Secure Computing

Year: 2025 | Volume: 22, Issue: 1 | Journal Article | Publisher: IEEE

Intelligent Reflecting Surface and Network Slicing Assisted Vehicle Digital Twin Update

Li Li; Lun Tang; Yaqing Wang; Tong Liu; Qianbin Chen

IEEE Transactions on Intelligent Transportation Systems

Year: 2025 | Volume: 26, Issue: 3 | Journal Article | Publisher: IEEE

Two-timescale Optimization for E2E Network Slicing-aided Cloud-edge Collaborative Networks

Yunfeng Wang; Liqiang Zhao; Xiaoli Chu; Shenghui Song; Yansha Deng; Arumugam Nallanathan; Guorong Zhou

IEEE Transactions on Vehicular Technology

Year: 2025 | Early Access Article | Publisher: IEEE

Advancing 6G: Survey for Explainable AI on Communications and Network Slicing

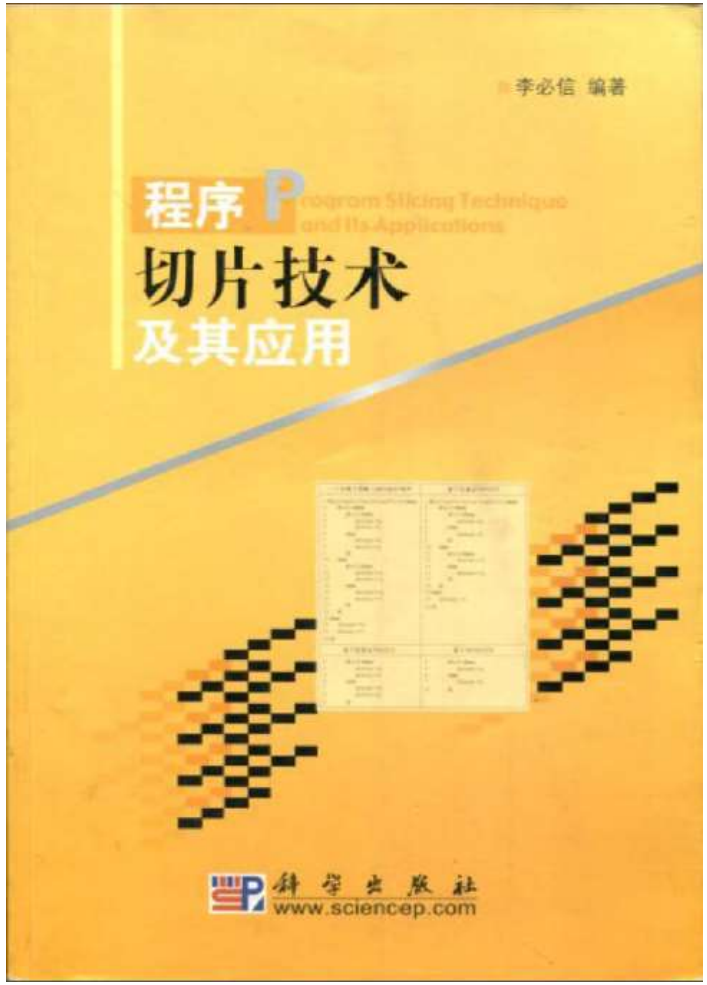
Haochen Sun; Yifan Liu; Ahmed Al-Tahmeesschi; Avishek Nag; Mohadeseh Soleimanpour; Berk Canberk; Hüseyin Arslan; Hamed Ahmadi

IEEE Open Journal of the Communications Society

Year: 2025 | Volume: 6 | Journal Article | Publisher: IEEE

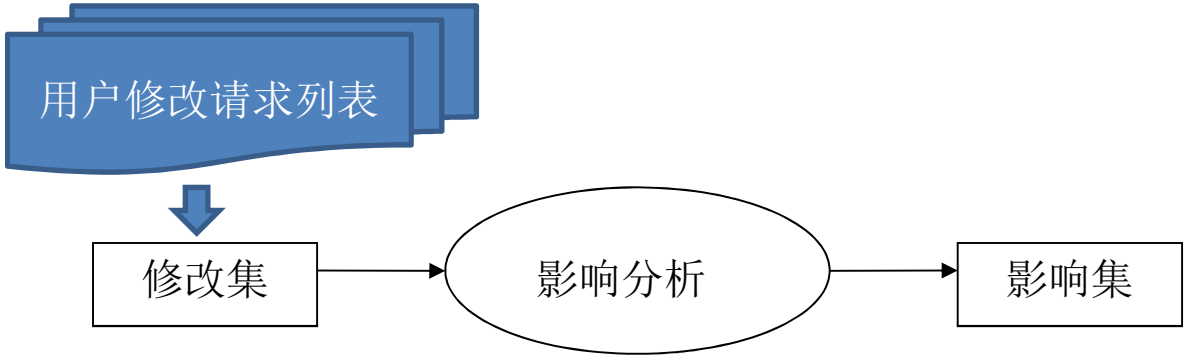
ISEU的工作

1. Wang Lulu, Li Bixin, Kong Xianglong. **Type slicing**: An accurate object oriented slicing based on sub-statement level dependence graph, Information and Software Technology, 127(2020)106369.
2. 文万志. 基于程序切片谱的软件错误定位技术研究，东南大学博士论文，2013.
3. Wanzhi Wen, Bixin Li, Xiaobing Sun, Jiakai Li: Program slicing spectrum-based software fault localization. SEKE 2011: 213-218
4. Wanzhi Wen.Software fault localization based on program slicing spectrum. ICSE 2012: 1511-1514.
5. 文万志; 李必信; 孙小兵; 刘翠翠. 一种基于层次切片谱的软件错误定位技术，软件学报，2013,24（5）：977-992.
6. 文万志; 李必信; 孙小兵; 齐珊珊. 基于条件执行切片谱的多错误定位, 计算机研究与发展，2013,50（5）：1030-1043.



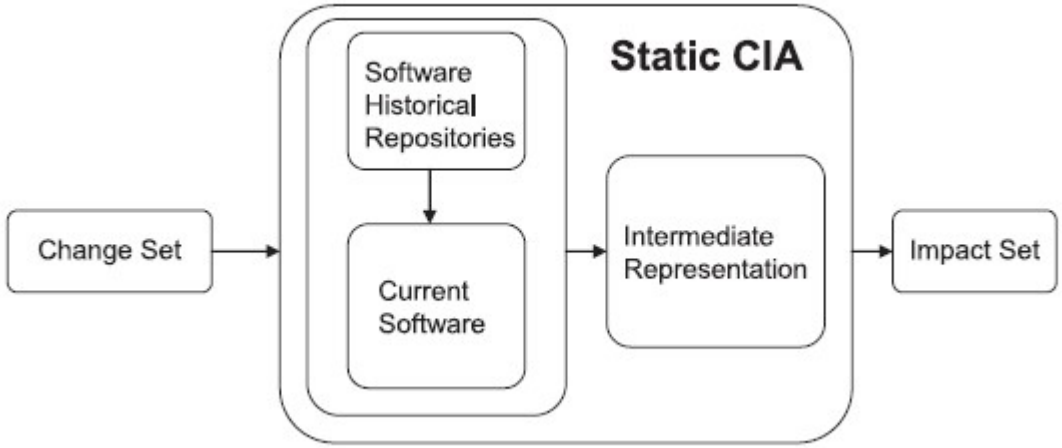
修改影响分析

- 当对软件进行修改时，肯定会对软件的其他部分造成一些潜在影响，从而带来软件的不一致性，如果实施该修改所需的成本比较高（或者影响范围比较广泛），甚至超过重新开发该软件所需的成本（几乎影响整个系统），那么就需要考虑其他代替修改方案或者重新开发软件，而如果接受了修改，我们**需要准确地预测修改带来的波动效应**，而准确地对波动效应进行预测既可以提高维护人员实施修改的信心，又可以帮助维护人员准确地找到需要进行二次修改的程序代码，从而节省了维护时间。



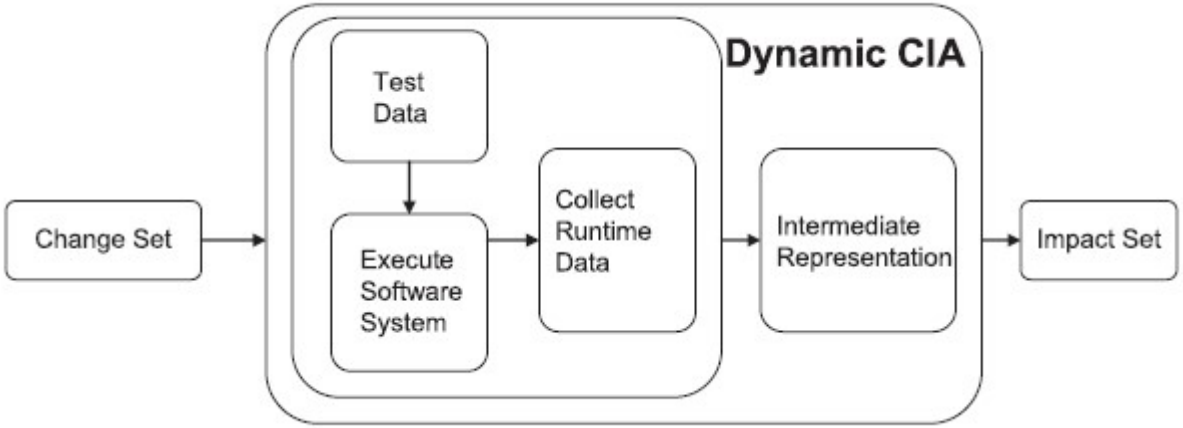
修改影响分析（Change Impact Analysis）过程

Static CIA techniques take all possible behaviour and inputs into account, and thus are conservative at the cost of precision.



静态修改影响分析过程

Dynamic analysis considers some specific inputs and relies on the analysis of the information collected during program execution (e.g. execution traces information, coverage information and execution relation information) to calculate the impact set【类似修改传播分析】



动态修改影响分析过程

Fast Configuration Change Impact Analysis for Network Overlay Data Center Networks

Lizhao You, *Member, IEEE*, Jiahua Zhang, Yili Jin, Hao Tang, and Xiao Li

***Abstract*—This paper presents the first network configuration verifier that provides fast all-pair reachability analysis of incremental configuration changes for network overlay data center networks (DCNs). Network overlay DCNs leverage distributed routing protocol on edge leaf switches to disseminate overlay routes and establish overlay tunnels. In addition, network overlay DCNs use access control lists, microsegmentation policy, policy-based routing and firewall policy to control east-west and north-south traffic. Although some incremental verification approaches have been proposed, they either do not support certain forwarding features of the network, or are not efficient. Our configuration verifier addresses these issues through the following components: 1) a port predicate based forwarding model that is general to support all features; 2) fine-grained association technique to index possibly affected reachable pairs by changed interfaces in the original network; and 3) required waypoint path computation that finds all reachable pairs related to changed interfaces in the new network. Based on these components, our verifier presents two incremental verification algorithms that are specially designed for different service update cases. Experiment results show that our incremental verification algorithms are accurate and fast. For all-pair reachability, our verifier performs change-impact analysis within 15s for networks with 200 leafs (4000 subnets and 16 million pairs), outperforming existing approaches by up to 10x.**

***Index Terms*—Network verification, configuration verification, incremental verification, data center networks.**

on edge leaf switches to advertise overlay routes and establish virtual extensible local area network (VXLAN) tunnels [3]. In addition, various policies such as access control lists (ACLs), microsegmentation (MCS) and policy-based routing are enforced by switches inside the network to control east-west and north-south traffic.

In this paper, we consider the incremental configuration verification problem in network overlay DCNs. When new services are to be deployed, incremental configurations of network switches are generated, and are to be verified so that they do not violate intended reachability requirements. In particular, we focus on the all-pair reachability change impact analysis: at the beginning, all endpoint interfaces that generate overlay traffic have a mutual reachability relationship; after incremental configurations are generated (but before deployment), the verifier returns changed all-pair reachability caused by incremental configurations so that users can check whether the impact follows their intent.

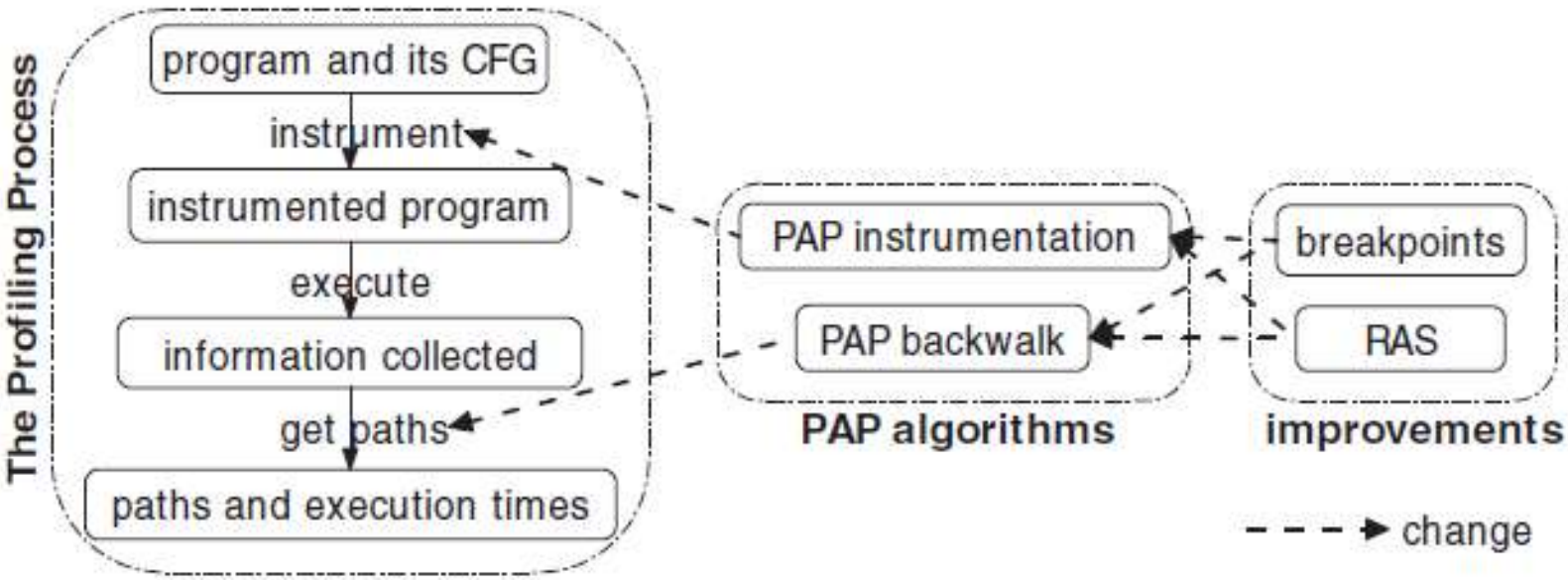
Given that overlay services are frequently deployed, the computation overhead of change impact analysis should be low. In addition, the pre-deployment analysis should be as fast as possible compared with post-deployment offline verification. Re-computing all pairs' reachability [4] is not scalable given the potential large amount of endpoint interfaces

ISEU的工作

- 孙小兵，基于概念格的软件修改分析研究，东南大学博士论文, 2013.
- Bixin Li, Xiaobing Sun, Hareton Leung. *A Survey of Code-based Change Impact Analysis Techniques.* Software Testing, Verification and Reliability, 2013,23:613–646 Wiley. [SCI/EI]
- Bixin Li, Xiaobing Sun, Jacky Keung. *FCA-CIA: An Approach of Using FCA to Support Cross-level Change Impact Analysis for Object Oriented Java Programs.* Information and Software Technology, Elsevier, 55 (2013) 1437–1449. [SCI/EI]
- Bixin Li, Qiandong Zhang, Xiaobing Sun, Hareton Leung.*Using Water Wave Propagation Phenomenon to Study Software Change Impact Analysis.* Advances in Engineering Software, Elsevier, 2013, 58(4):45-53. [SCI/EI]
- Xiaobing Sun, Bixin Li, Chuanqi Tao, Wanzhi Wen, Sai Zhang. *Analyzing Impact Rules of Different Change Types to Support Change Impact Analysis.* International Journal of Software Engineering and Knowledge Engineering, World Scientific, 2013, 23(3): 259-288. [SCI/EI]

路径剖析

- 动态程序分析是基于程序执行的分析技术，所以收集程序的执行信息是动态分析方法不可缺少的一部分。收集信息的方法包括
  - 程序追踪（program tracing）：程序追踪技术要求完整的记录程序执行中的控制流、数据流等信息，以便再现该次执行。
  - 路径剖析（path profiling）：路径剖析仅收集路径信息，要求较小的耗费且侧重于路径的执行频率。
  - 边的剖析（edge profiling）：边的剖析则是记录控制流中每条边的执行频率。
- 路径剖析是收集程序执行信息的重要手段。与程序追踪相比，其缺少对数据流信息的记录，但是耗费低廉；与边的剖析相比，其耗费略高，但是提供的信息远多于边的剖析。
- 路径剖析一般遵循如下步骤：
  - (1) 输入：待路径剖析程序及其CFG；
  - (2) 路径剖析算法：根据CFG得出需要在程序中插装的探针语句及位置；
  - (3) 插装：将算法得出的探针语句插装到程序相应的位置中；
  - (4)执行：执行插装后的程序若干次，并收集路径编码；
  - (5)输出：根据收集的信息统计所执行的路径及其相应的频率。



PAP（ *Profiling All Paths* ）方法的的基本过程如图所示：首先在单变量剖析方法的基本过程之上，改进了探针插装算法，并设计了回溯算法以通过探针值获取相应的路径；然后在此基础程上，提出了断点机制以处理探针值可能溢出的情况，并在CFG的可规约无环子图中结合使用EPP方法进行剖析，改进了效率。

RESEARCH-ARTICLE
September 2024

DDGF: Dynamic Directed Greybox Fuzzing with Path Profiling

Haoran Fang, Kaikai Zhang, Donghui Yu, Yuanyuan Zhang

ISSTA 2024: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis•Pages 832–843•https://doi.org/10.1145/3650212.3680324

Coverage-Guided Fuzzing (CGF) has become the most popular and effective method for vulnerability detection. It is usually designed as an automated "black-box" tool. Security auditors start it and then just wait for the results. However, after a period of ...

A Deep-Learning Method for Path Loss Prediction Using Geospatial Information and Path Profiles

Takahiro Hayashi; Koichi Ichige

IEEE Transactions on Antennas and Propagation
Year: 2023 | Volume: 71, Issue: 9 | Journal Article | Publisher: IEEE
Cited by: Papers (7)

2021 IEEE 27th International Conference on Parallel and Distributed Systems (ICPADS)

Jyane:Detecting Reentrancy vulnerabilities based on path profiling method

Yicheng Fang,Chunping Wang,Zhe Sun,Hongbing Cheng*
College of Computer Science and Technology College of Software
Zhejiang University of Technology
310023 Hangzhou, China
{2111912045,wangcp,2112012053,chenghb}@zjut.edu.cn

l Distributed Systems (ICPADS) | 978-1-6654-0878-3/21/$31.00 ©2021 IEEE | DOI: 10.1109/ICPADS53394.2021.00040

***Abstract*—Ethereum is essentially a transaction-driven state machine, and a smart contract is a piece of executable code on Ethereum. Compared with the scripting language on Bitcoin, the smart contract language solidity, which is Turing-complete and the expressive capabilities are very powerful. However, this attribute also brings many potential security threats, vulnerabilities, and various other issues. In this paper, we propose a novel smart contract security technology, named Jyane, to detect the Reentrancy vulnerability, which is one of the most threatening vulnerabilities to smart contracts. More importantly, Our tool–Jyane is the first path profiling solution for smart contracts. Firstly, we use EVM (Ethereum Virtual Machine) binary bytecode to construct control flow graphs (CFG), then use the improved Ball-Larus Path profiling algorithm (BLPP) to generate IDs for acyclic paths. Finally, after profiling the constructed paths, the suspicious paths can be detected successfully.We evaluate Jyane and other technology through comprehensive test and comparison; the results show that Jyane can profile the actual execution path of smart contracts to detect vulnerabilities with a low false-positive rate accurately. From the results of the evaluation, Jyane marked 27 of 1,226 Ethereum smart contracts selected in 2016 and 2017 as vulnerable contracts, included the vulnerability of the DAO contract which once led to a $60 million loss. Furthermore, compared with some other existing detection tools, Jyane shows broader detection range for Reentrancy vulnerabilities with lower time overhead.**

***Index Terms*—Blockchain, Ethereum smart contract, Vulnerability, Path profiling**

TABLE I
TWO TYPES OF ACCOUNT MODELS ON SMART CONTRACTS

Externally owned account	Contract account
<balance> <nonce>	<nonce> <code> <storageRoot> <codeHash>

Ethereum virtual Machine EVM can easily implement complex business logic and applications in real application scenarios.

The term "smart contract" can be traced back to 1995 and was coined by a legal scholar, Nick Szabo. In the application of blockchain, a smart contract is a piece of executable code that has been authenticated by a node and generates a consensus. Users will be attacked by hackers when they entrust a smart contract to transfer Ether. These malicious attacks often have more serious consequences than hacker attacks on regular networks, because transactions on the blockchain platform cannot be tampered with, and the transactions generated by malicious attacks will always exist on the blockchain. Some vulnerabilities caused unprecedented losses. In 2016, hackers launched an attack on The Dao[5][18] that caused 60 million dollars in losses.

Ethereum accounts can be divided into two types: Ex-

Universal Path Tracing for Large-Scale Sensor Networks

Wei Dong, *Member, IEEE, ACM*, Yi Gao, *Member, IEEE, ACM*, Chenhong Cao, Xiaoyu Zhang, and Wenbin Wu

***Abstract*— Most sensor networks employ dynamic routing protocols so that the routing topology can be dynamically optimized with environmental changes. The routing behaviors can be quite complex with increasing network scale and environmental dynamics. Knowledge on the routing path of each packet is certainly a great help in understanding the complex routing behaviors, allowing effective performance diagnosis and efficient network management. We propose PAT, a *universal* sensornet path tracing approach. PAT includes an intelligent path encoding scheme that allows efficient decoding at the PC side. To make PAT more scalable, we propose techniques to *accurately* estimate the degree information by exploiting timing information, allowing more *compact* path encoding. Moreover, we employ subpath concatenation to infer excessively long paths with a high recovery probability. We propose an analytical model to quantify the benefits of PAT with varying network scale, network density, routing dynamics and packet delivery performance. We evaluate PAT's performance using testbed experiments, trace-driven study, and extensive simulations. Results show that PAT significantly outperforms existing approaches.**

***Index Terms*— Path reconstruction, multihop wireless networks, network measurement.**

which each node regularly estimates the expected number of transmissions (ETX) [6] to the sink and dynamically selects the next-hop forwarder with a minimum ETX along the path. Due to the dynamic routing scheme, a node could forward different packets to different next hops, although these packets have the same destination, i.e., the sink node.

The routing behaviors can be quite complex with increasing network scale and environmental dynamics: packets from a distant sensor node may follow numerous routing paths to the sink. Knowledge on the routing path of each packet is very useful in the following ways.

- It is important for understanding the dynamic routing topology, revealing the complex routing behaviors. Such a knowledge is essential for topology control, routing improvement, and load balance, enabling effective management and optimized operations for deployed WSNs consisting of a large number of unattended wireless sensor nodes [7].
- It is essential for deriving important system metrics, e.g., per-link loss ratio and per-link delays. Most exist-

ISEU的工作

◆Lulu Wang, Jingyue Li, Bixin Li: Tracking runtime concurrent dependences in java threads using thread control profiling. Journal of Systems and Software, 148: 116-131 (2019)

◆Lulu Wang, Bixin Li, Hareton Leung: *A New Method to Encode Calling Contexts with Recursions*. SCIENCE CHINA Information Sciences 59(5): 052104:1-052104:15 (2016)

◆王璐璐 李必信.一种面向有环兴趣路径的过程内剖析方法,计算机学报, 2014, 37(12):2464-2481.

◆王璐璐 李必信. 过程间循环路径剖析方法,计算机学报, 2013, 36(11):2224-2235.

◆Bixin Li, Lulu Wang, Hareton Leung. *Profiling Selected Paths with Loops.* Science China: Information Sciences, July 2014, 57(7):072105:1–072105:15.

◆Bixin Li, Lulu Wang, Hareton Leung. *Profiling All Paths: A New Profiling Technique for Both Cyclic and Acyclic Paths.* Journal of Systems and Software, Elsevier, July 2012, 85(7):1558-1576.

◆王璐璐 李必信 周晓宇.全路径剖析方法,软件学报，2012,23（6）：1413-1428.

◆王璐璐.有效路径剖析技术研究. 东南大学博士论文,2012.

3 度量方法

什么是软件度量？

软件度量是指对软件开发项目、过程及其产品进行数据定义、收集以及分析的持续性定量化过程。其目的是理解、预测、评估、控制和改善软件开发过程及产品。没有软件度量，就无法从软件开发的暗箱中跳将出来，通过软件度量可以改进软件开发过程，促进项目成功，开发高质量的软件产品。

软件度量的目的

通过软件度量，可以获得对过程、产品、资源的理解，确定预测的基线和模型，合理分配资源、制定计划，控制执行过程，并最终提高软件产品质量和开发效率。

软件度量的分类

软件度量可以分为以下几个基本度量项：

规模：软件工作产品的大小（如文档页数、代码量）。

工作量：完成各软件工作产品和活动所需的人时或人天。

进度：各软件工作产品和活动的开始和结束时间。

质量：在各软件工作产品和活动中产生的缺陷数。

- 软件产品度量/软件质量度量
  - 软件质量度量模型：ISO 9126、CMM、
- 软件过程度量/过程质量度量
  - 软件过程度量主要包括三大方面的内容：
    - 1）成熟度度量(maturity metrics)，主要包括组织度量、资源度量、培训度量、文档标准化度量、数据管理与分析度量、过程质量度量等等；
    - 2）管理度量(management metrics)，主要包括项目管理度量(如里程碑管理度量、风险度量、作业流程度量、控制度量、管理数据库度量等)、质量管理度量(如质量审查度量、质量测试度量、质量保证度量等)、配置管理度量(如式样变更控制度量、版本管理控制度量等)；
    - 3）生命周期度量(life cycle metrics)，主要包括问题定义度量、需求分析度量、设计度量、制造度量、维护度量等。
  - 软件过程度量的一般流程主要包括：确认过程问题；收集过程数据；分析过程数据；解释过程数据；汇报过程分析；提出过程建议；实施过程行动；实施监督和控制。

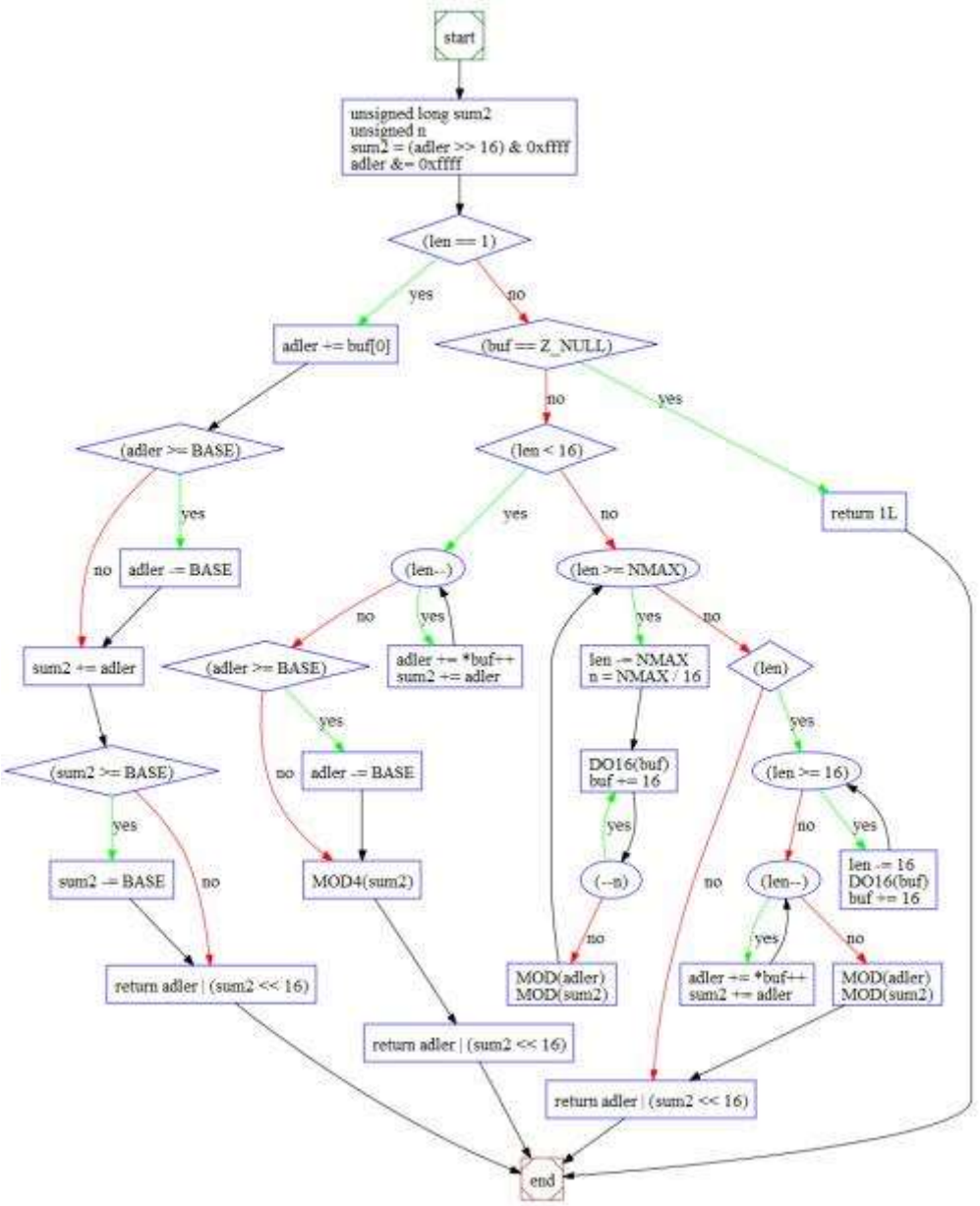
- 复杂度度量
  - 结构复杂度
    - Cyclomatic Complexity(CCN)： V(G)=e-n+2
    - Halstead复杂度
- 模块内聚度量TCC和LCC
- 扇入扇出度FFC
- 模块间耦合度CBO
- 模块间响应度RFC
- 软件架构静态成熟度SSAM
- 软件架构动态成熟度DSAM
- 软件架构综合成熟度ISAM
- ……

- **Cyclomatic Complexity: (McCabe's)**

  1.Computed in several ways:
  (1)Edges –nodes +2；
  (2)Number of regions in CFG；
  (3)Number of decision statements + 1 (if structured)

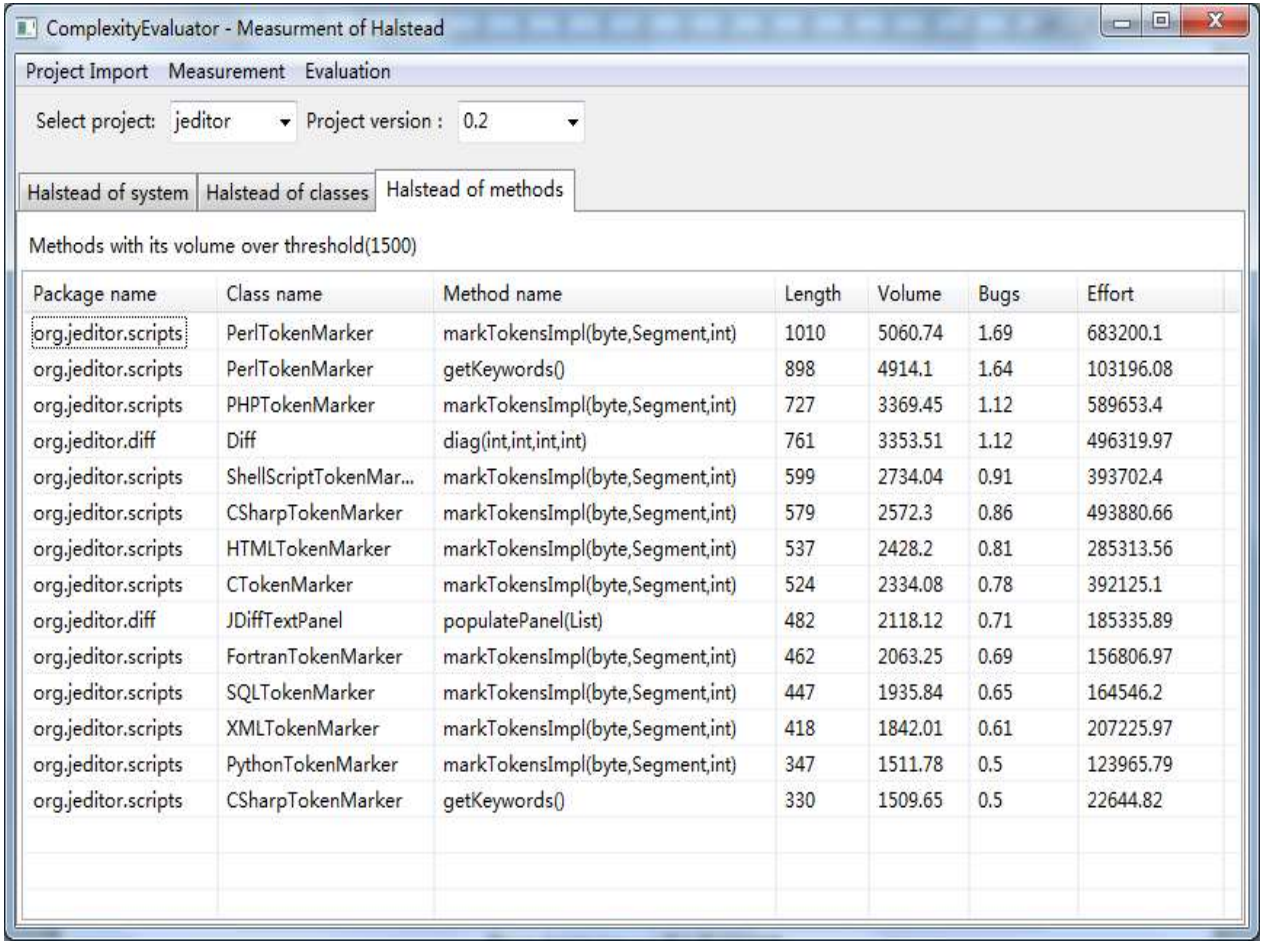
  2. Indication of number of test cases needed

  3. indication of difficulty of maintaining



名称	含义
n1	程序中出现不同的操作符的数量
n2	程序中出现的不同的操作数的数量
N1	程序中出现的操作符总数
N2	程序中出现的操作数总数

名称	公式
程序实际长度	N=N1+N2
词汇量	n=n1+n2
容量	V=N*log2(n)
程序难度	D=(n1/2)*(N2/n2)
程序级别	L=1/D
工作量	E=V*D
预测错误数	B=V/3000
实现时间	T=E/18



Package name	Class name	Method name	Length	Volume	Bugs	Effort
org.jeditor.scripts	PerlTokenMarker	markTokensImpl(byte,Segment,int)	1010	5060.74	1.69	683200.1
org.jeditor.scripts	PerlTokenMarker	getKeywords()	898	4914.1	1.64	103196.08
org.jeditor.scripts	PHPTokenMarker	markTokensImpl(byte,Segment,int)	727	3369.45	1.12	589653.4
org.jeditor.diff	Diff	diag(int,int,int,int)	761	3353.51	1.12	496319.97
org.jeditor.scripts	ShellScriptTokenMar...	markTokensImpl(byte,Segment,int)	599	2734.04	0.91	393702.4
org.jeditor.scripts	CSharpTokenMarker	markTokensImpl(byte,Segment,int)	579	2572.3	0.86	493880.66
org.jeditor.scripts	HTMLTokenMarker	markTokensImpl(byte,Segment,int)	537	2428.2	0.81	285313.56
org.jeditor.scripts	CTokenMarker	markTokensImpl(byte,Segment,int)	524	2334.08	0.78	392125.1
org.jeditor.diff	JDiffTextPanel	populatePanel(List)	482	2118.12	0.71	185335.89
org.jeditor.scripts	FortranTokenMarker	markTokensImpl(byte,Segment,int)	462	2063.25	0.69	156806.97
org.jeditor.scripts	SQLTokenMarker	markTokensImpl(byte,Segment,int)	447	1935.84	0.65	164546.2
org.jeditor.scripts	XMLTokenMarker	markTokensImpl(byte,Segment,int)	418	1842.01	0.61	207225.97
org.jeditor.scripts	PythonTokenMarker	markTokensImpl(byte,Segment,int)	347	1511.78	0.5	123965.79
org.jeditor.scripts	CSharpTokenMarker	getKeywords()	330	1509.65	0.5	22644.82

Architecture Evaluator - Evolution Evaluation

Project Import | Data Collection | Maturity Measurement | Evolution Evaluation

Multi-version comparison | Adjacent version comparison

Project name : Software Architecture Tool | Project version : 1.3

Direction : Backward | Forward | Compare

Measurements comparison | Physical difference | Cause analysis

Maturity comparison :

Static Attributes

VerID	CCN	FFC	CBO	RFC	TCC	LCC
1.3	4.0	13.0	0.44	1.0	1.0	1.0
1.4	8.0	61.0	0.53	2.0	1.0	1.0

Dynamic Attributes

VerID	Reliability	CorrectRate	Performance_Max	Performance_Avg
1.3	1.0	0.67	0.2986744	1.4642465
1.4	1.0	0.5	0.4537493	3.8460965

Performance Attributes

VerID	MaxExcTime	AvgExcTime	MaxResTime	AvgResTime	MaxMemUse	MemUseUnitTime	MaxCPUUse	CPUUseUnitTime
1.3	35476.0	35476.0	35138.0	3942.0	110168.0	3.0	62.0	0.0
1.4	95529.0	95529.0	60053.0	8684.0	111188.0	3.0	79.0	0.0

Architecture Evaluator - Evolution Evaluation

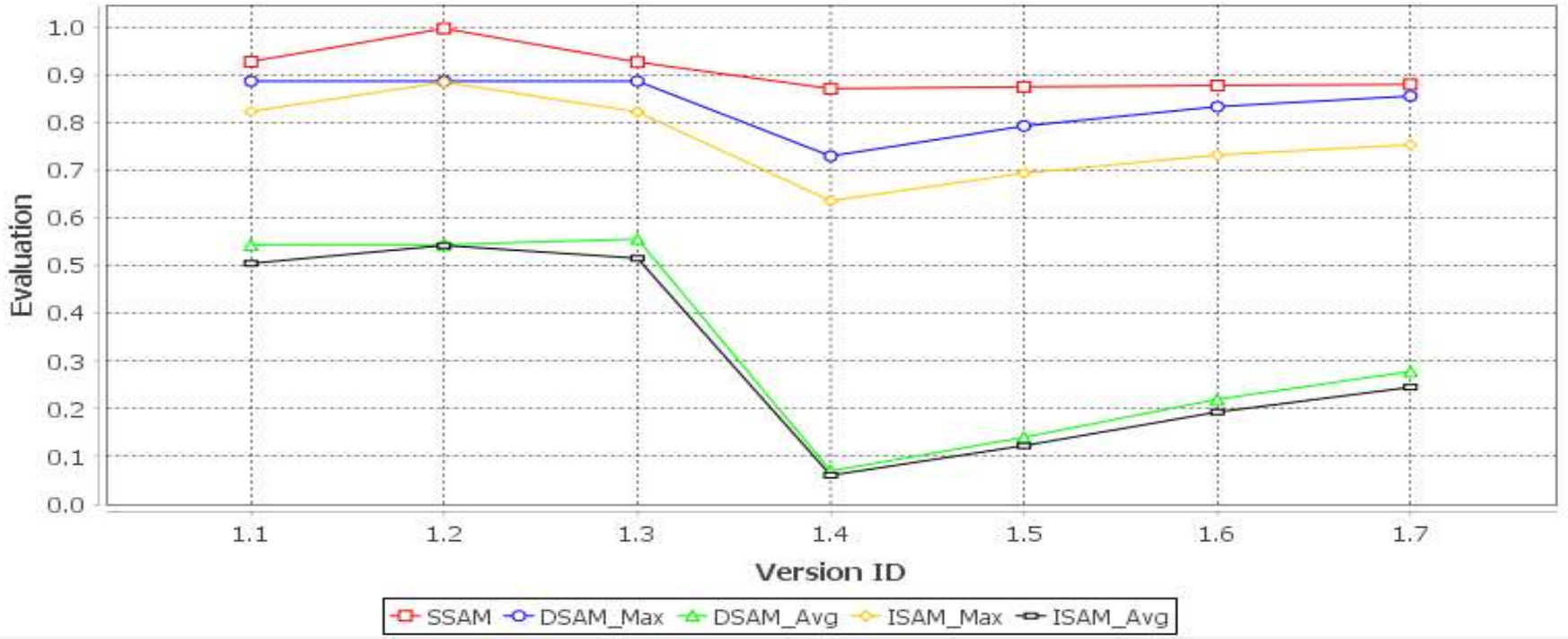
Project Import | Data Collection | Maturity Measurement | Evolution Evaluation

Multi-version comparison | Adjacent version comparison

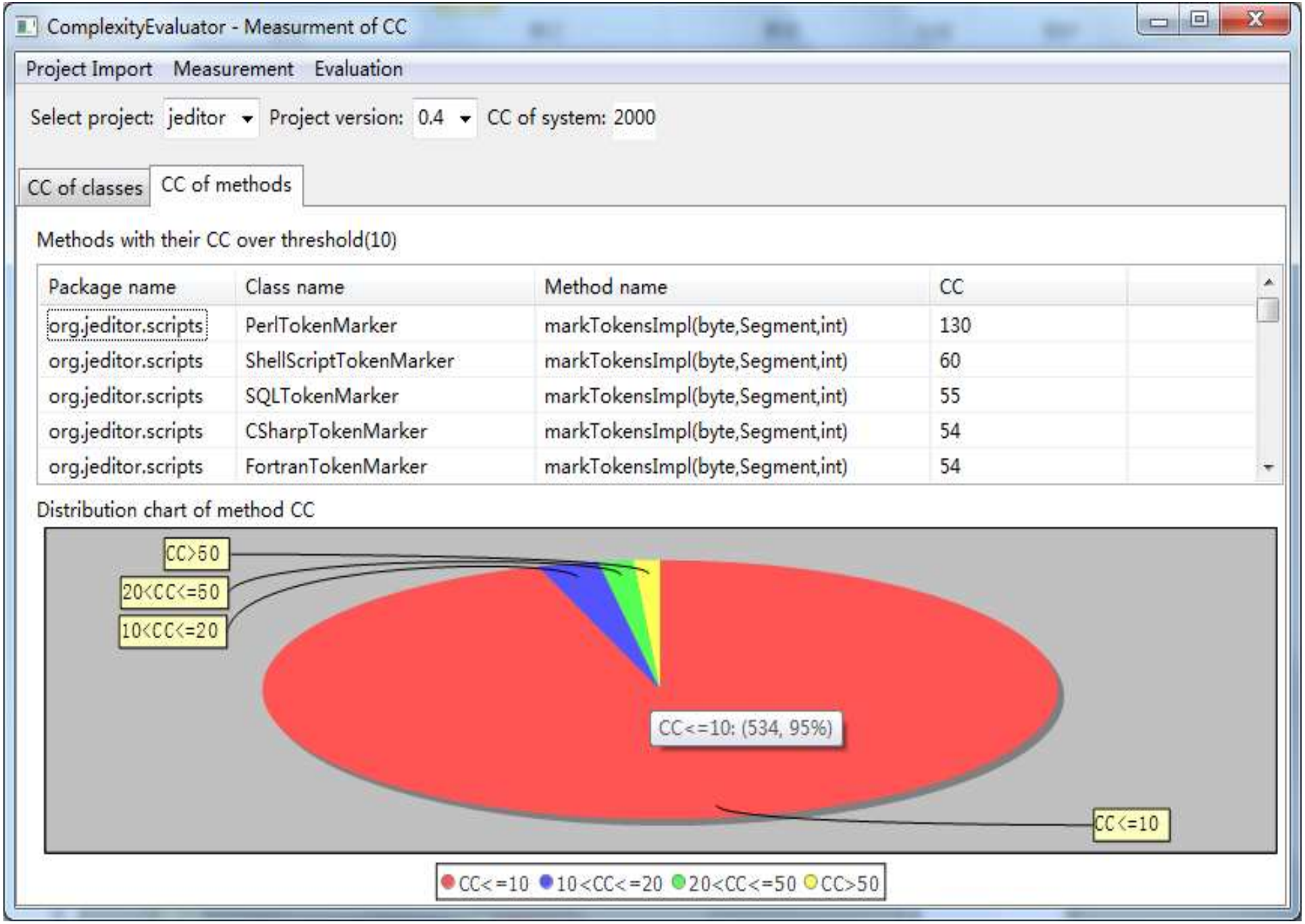
Select project : Software Architecture Tool

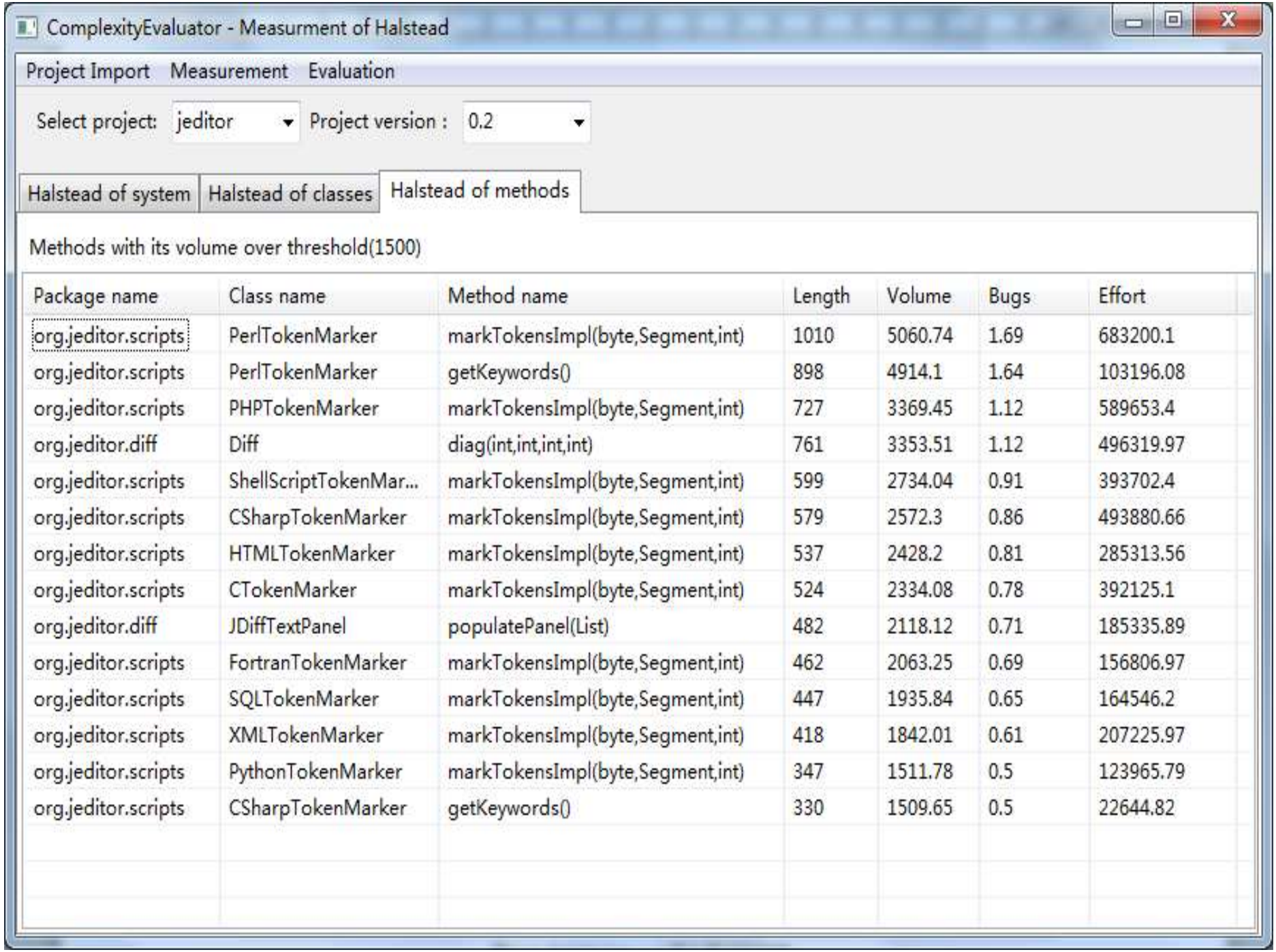
Query

Number of versions : 7



Version ID	SSAM	DSAM_Max	DSAM_Avg	ISAM_Max	ISAM_Avg
1.1	0.9283665	0.8872643	0.5439245	0.8237065	0.5049613
1.2	0.9976392	0.8872643	0.5439245	0.8851696	0.5426404
1.3	0.9272696	0.8872643	0.5563276	0.8227332	0.5158657
1.4	0.8716026	0.7301429	0.069536	0.6363945	0.0606078
1.5	0.8753381	0.7931122	0.1401735	0.6942413	0.1226992
1.6	0.8783263	0.8338272	0.2200002	0.7323724	0.193232
1.7	0.8805855	0.8560537	0.2784763	0.7538285	0.2452222


ComplexityEvaluator - Measurment of CC
Project Import
Measurement
Evaluation
Select project: jeditor
Project version: 0.4
CC of system: 2000
CC of classes
CC of methods
Methods with their CC over threshold(10)
Package name
Class name
Method name
CC
org.jeditor.scripts
PerlTokenMarker
markTokensImpl(byte,Segment,int)
130
org.jeditor.scripts
ShellScriptTokenMarker
markTokensImpl(byte,Segment,int)
60
org.jeditor.scripts
SQLTokenMarker
markTokensImpl(byte,Segment,int)
55
org.jeditor.scripts
CSharpTokenMarker
markTokensImpl(byte,Segment,int)
54
org.jeditor.scripts
FortranTokenMarker
markTokensImpl(byte,Segment,int)
54
Distribution chart of method CC
CC>50
20<CC<=50
10<CC<=20
CC<=10: (534, 95%)
CC<=10
CC<=10
10<CC<=20
20<CC<=50
CC>50


ComplexityEvaluator - Measurment of Halstead
Project Import
Measurement
Evaluation
Select project:
jeditor
Project version :
0.2
Halstead of system
Halstead of classes
Halstead of methods
Methods with its volume over threshold(1500)
Package name
Class name
Method name
Length
Volume
Bugs
Effort
org.jeditor.scripts
PerlTokenMarker
markTokensImpl(byte,Segment,int)
1010
5060.74
1.69
683200.1
org.jeditor.scripts
PerlTokenMarker
getKeywords()
898
4914.1
1.64
103196.08
org.jeditor.scripts
PHPTokenMarker
markTokensImpl(byte,Segment,int)
727
3369.45
1.12
589653.4
org.jeditor.diff
Diff
diag(int,int,int,int)
761
3353.51
1.12
496319.97
org.jeditor.scripts
ShellScriptTokenMar...
markTokensImpl(byte,Segment,int)
599
2734.04
0.91
393702.4
org.jeditor.scripts
CSharpTokenMarker
markTokensImpl(byte,Segment,int)
579
2572.3
0.86
493880.66
org.jeditor.scripts
HTMLTokenMarker
markTokensImpl(byte,Segment,int)
537
2428.2
0.81
285313.56
org.jeditor.scripts
CTokenMarker
markTokensImpl(byte,Segment,int)
524
2334.08
0.78
392125.1
org.jeditor.diff
JDiffTextPanel
populatePanel(List)
482
2118.12
0.71
185335.89
org.jeditor.scripts
FortranTokenMarker
markTokensImpl(byte,Segment,int)
462
2063.25
0.69
156806.97
org.jeditor.scripts
SQLTokenMarker
markTokensImpl(byte,Segment,int)
447
1935.84
0.65
164546.2
org.jeditor.scripts
XMLTokenMarker
markTokensImpl(byte,Segment,int)
418
1842.01
0.61
207225.97
org.jeditor.scripts
PythonTokenMarker
markTokensImpl(byte,Segment,int)
347
1511.78
0.5
123965.79
org.jeditor.scripts
CSharpTokenMarker
getKeywords()
330
1509.65
0.5
22644.82

架构可演进性度量 | 架构持续演进原则达成性度量

度量层次: ○ 架构层 ● 组件层

设计指标: ○ 规模 ○ 圈复杂度 ● 调用层次 ○ 内聚度 ○ 耦合度

质量指标: ○ 可扩展性 ○ 可维护性 ○ 可重用性 ○ 可替换性 ○ 易理解性 ○ 易测试性

视图: ○ 详情 ● 趋势

Information for 组件5

指标名	指标值	贡献率
规模	17121	
圈复杂度	2307	
调用层次	2	
内聚度	5.287	
耦合度	0.056	
可扩展性	0.7303	0.13044
可维护性	0.04092	0.00223
可重用性	0.5	0.17647
可替换性	0.2	0.0458
易理解性	0.33387	0.98056
易测试性	0.12222	0.10891

详情解释

	整体	组件13	组件5	件9
度量值	3	3	2	

Reflections on McCabe’ s Cyclomatic Complexity

Dennis Kafura

IEEE Transactions on Software Engineering

Year: 2025 | Early Access Article | Publisher: IEEE

Impact of “Evaluating Software Complexity Measures”

Elaine J. Weyuker

IEEE Transactions on Software Engineering

Year: 2025 | Early Access Article | Publisher: IEEE

Towards measurement-based Software Engineering

Victor Basili; David Weiss; Dieter Rombach

IEEE Transactions on Software Engineering

Year: 2025 | Early Access Article | Publisher: IEEE

ISO/IEC/IEEE International Standard Software engineering - Software Non-Functiona Sizing Measurement

ISO/IEC/IEEE 32430:2025(en)

Most Recent Versions

Year: 2025 | Standard | Publisher: IEEE

2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)

ing in Practice (ICSE-SEIP) | 979-8-3503-0037-6/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICSE-SEIP58684.2023.00025

Investigating a NASA Cyclomatic Complexity Policy on Maintenance Risk of a Critical System

Dan Port
Information Technology Management
University of Hawaii, Manoa
Honolulu, USA
ORCID 0000-0001-7020-7558

Bill Taber
Mission Design and Navigation
Jet Propulsion Laboratory, California Institute of Technology
Pasadena, USA
btaber@jpl.nasa.gov

LiQuo Huang
Dept. of Computer Science
Southern Methodist University
Dallas, USA
lghuang@smu.edu
ORCID 0000-0001-7790-0195

***Abstract*—Monte is a mission critical system used by NASA for navigation and design of deep space missions has been used in over 40 missions over the last 18 years. Its continuous, reliable operation is considered critical to the operation for over 18 ongoing missions. Recently Monte has been escalated to safety-critical software and subject to NASA Software Assurance and Software Safety Standard requirements. One of these requirements mandates a policy that the cyclomatic complexity (CC) for safety-critical components be under 16 or given a technically detailed explanation as to why it cannot or should not be lower. Conformance to this requirement would be costly and we had doubts about its benefit and efficacy in managing our maintenance risk (defect proneness and defect repair effort). The requirement was not substantiated either empirically or in principle, and guidance in the literature for use of CC as in indicator of maintenance risk is limited and often speculative or have contradictory empirical or non-definitive results.**

to as the "Google Maps to the solar system"). The system in its current state is a complex integration of hundreds of C++ and Python applications, with several versions used in over 40 missions over the last 18 years. This system has been under active, disciplined maintenance (CMMI level 3) by the Mission Design and Navigation (MDN) group for over 20 years. Defect reports and new feature requests continuously arrive at the rate of 160 unique defect reports per year and 150 feature requests per year. The codebase is currently 800,000 SLOC and growing at a rate of 2750 per release and an average of 11.75 releases per year.

Its continuous, reliable operation is critical to the operation for over 18 ongoing missions and is classified as *mission critical* (class B) software [2]. However, in consideration of its use on a potential human-mission to Mars, Monte is now being assessed as *human rated* (Class A) software. For risk management

ISEU的工作

- Huihui Liu, Xufang Gong, Li Liao, Bixin Li: Evaluate How Cyclomatic Complexity Changes in the Context of Software Evolution. COMPSAC (2) 2018: 756-761
- Tong Wang, Bixin Li: Analyzing Software Architecture Evolvability Based on Multiple Architectural Attributes Measurements. QRS 2019: 204-215
- Tong Wang, Yelian Zhang, Xufang Gong, Bixin Li: Recover and Optimize Software Architecture Based on Source Code and Directory Hierarchies (S). SEKE 2019: 469-599
- Tong Wang, Dongdong Wang, Ying Zhou, Bixin Li: Software Multiple-Level Change Detection Based on Two-Step MPAT Matching. SANER 2019: 4-14
- Tong Wang, Dongdong Wang, Bixin Li: A Multilevel Analysis Method for Architecture Erosion. SEKE 2019: 443-566
- Zecheng Li, Li Liao, Hareton Leung, Bixin Li, Chao Li: Evaluating the credibility of cloud services. Computers & Electrical Engineering 58: 161-175(2017)
- 赵希茜.面向对象Java切片及其在API度量中的应用.东南大学硕士论文,2017.
- 司静文.软件体系结构的度量和评估. 东南大学硕士论文,2015.
- 余析蒙.基于验证的软件架构演化分析与评估.东南大学硕士论文,2015.
- 何　磊.基于代码复杂度的软件演化评估与分析.东南大学硕士论文,2015.

4 验证方法

什么是形式化验证？

形式化验证（Formal Verification，简称FV）是一种使用数学工具分析设计可能行为空间的方法，而不是计算特定值的结果。形式化验证通过数学建模和严格的推理规则来验证设计的正确性，确保设计在所有可能的输入下都能满足预期的功能和性质。

形式化验证的基本原理和应用场景

形式化验证使用数学工具对设计进行建模，然后通过穷举系统运行过程中电路所能达到的所有状态，以断言的形式完成设计的功能验证和规则检查。这种方法适用于综合前后的等价性检查和RTL设计的功能验证，能够提供完全的覆盖率，并且能够发现仿真难以发现的极端情况。

形式化验证的优势和局限性

（1）形式化验证的主要优势包括：

完全覆盖：形式化验证能够分析设计行为的所有可能状态，提供完全的覆盖率。

精确性和可靠性：通过严格的数学推理，形式化验证能够确保设计的每一个状态都符合预期，减少了误判和漏判的可能性。

自动化：借助专业工具，形式化验证可以自动化部分审查工作，提高认证效率4。

（2）然而，形式化验证也存在一些局限性：

计算复杂度：对于超复杂的设计电路，形式化验证工具可能无法处理所有状态，导致长时间无法得出证明结果。

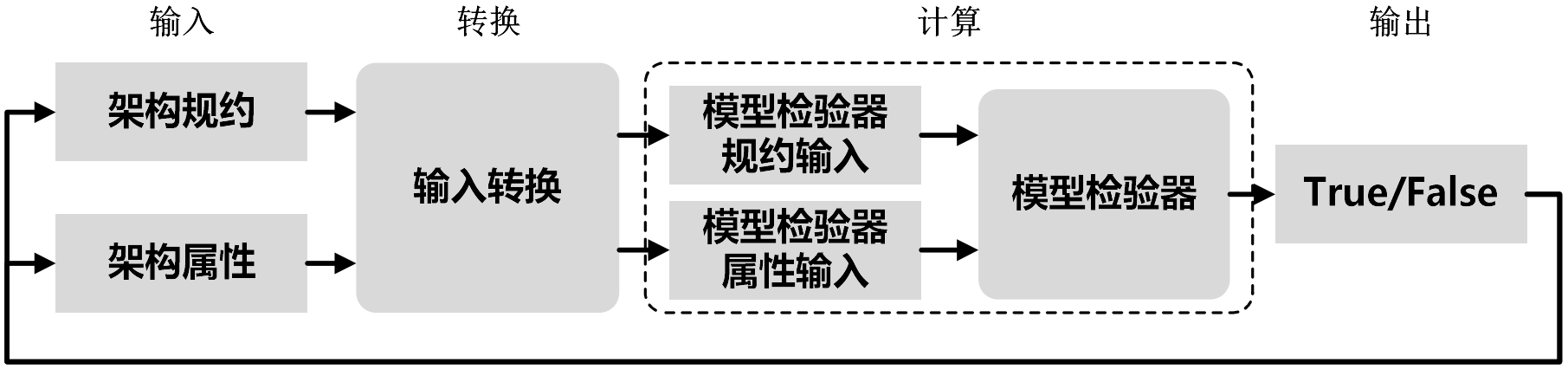
依赖强大的运算系统：形式化验证需要高性能的运算系统和EDA工具支持，否则可能面临状态爆炸问题。

- 定理证明：定理证明方法是一种将模型抽象为逻辑公式，然后使用自动的逻辑推理技术来验证电路是否正确的技术，定理证明方法十分严格，跟数理逻辑结合十分紧密，一般使用高阶逻辑（Higher-Order Logic，HOL）系统来进行证明。
- 模型检验(Model Checking)：主要是检查RTL代码是否满足规范中规定的一些特性
- 等效性检验：主要是验证在一个设计经过变换之后，穷尽地检验变化前后的功能的一致性

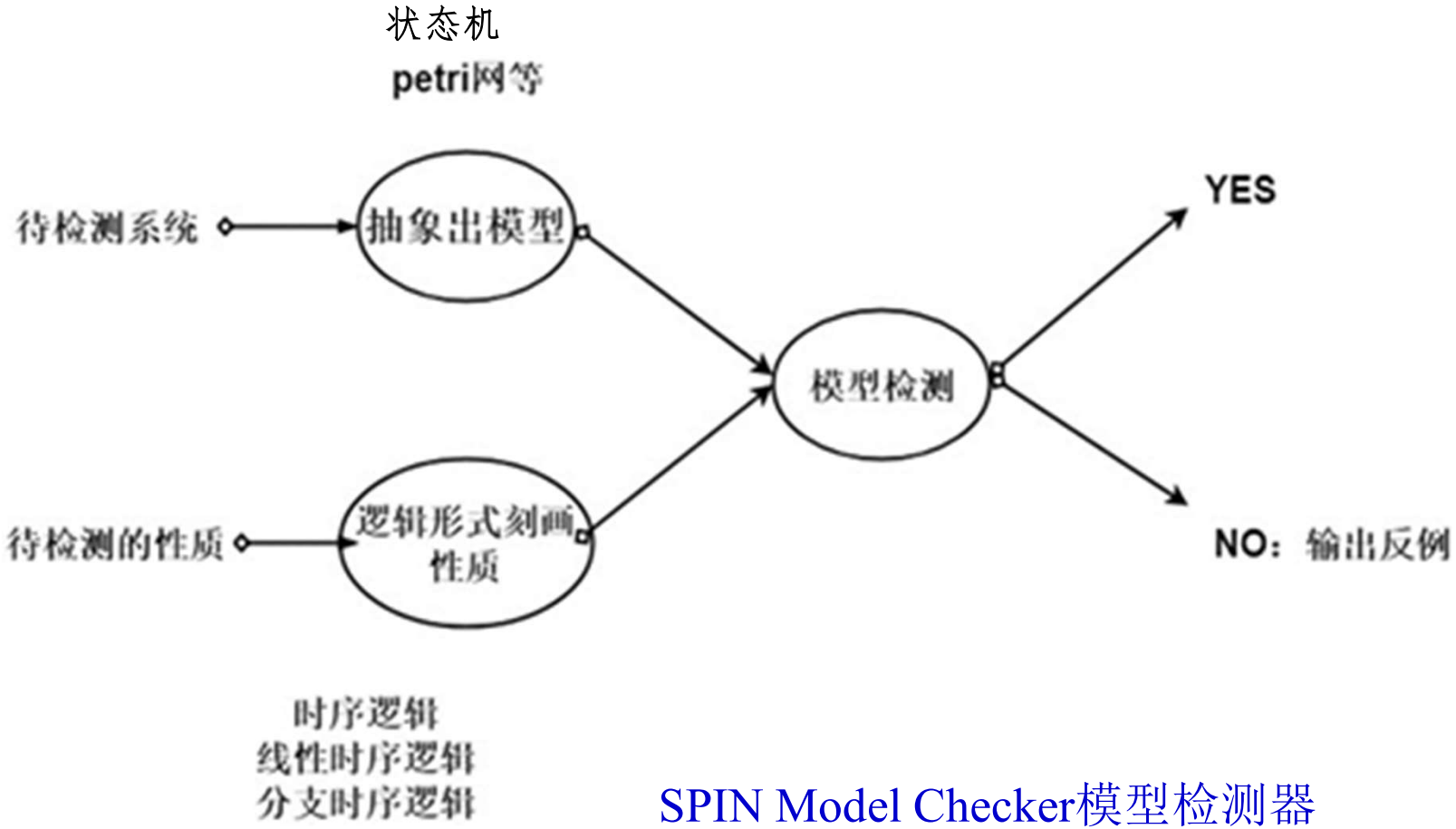
◆ 形式化验证

形式化验证包括模型检验、定理证明和逻辑推理等。

形式化方法的实质是以逻辑、自动机、代数和图论等数学理论为基础，用一套特定的符号和技术对软件系统进行描述和分析，以期提高软件可靠性。形式化方法的研究意义主要有以下几个方面：提供描述手段，以精确、无二义地描述系统赋予程序意义；提供分析手段，以证明系统正确性，或帮助开发人员找出系统出错原因提供开发方法和工具，以实现软件开发过程中全部或部分开发活动的自动或半自动化。



模型检验SA的一般过程

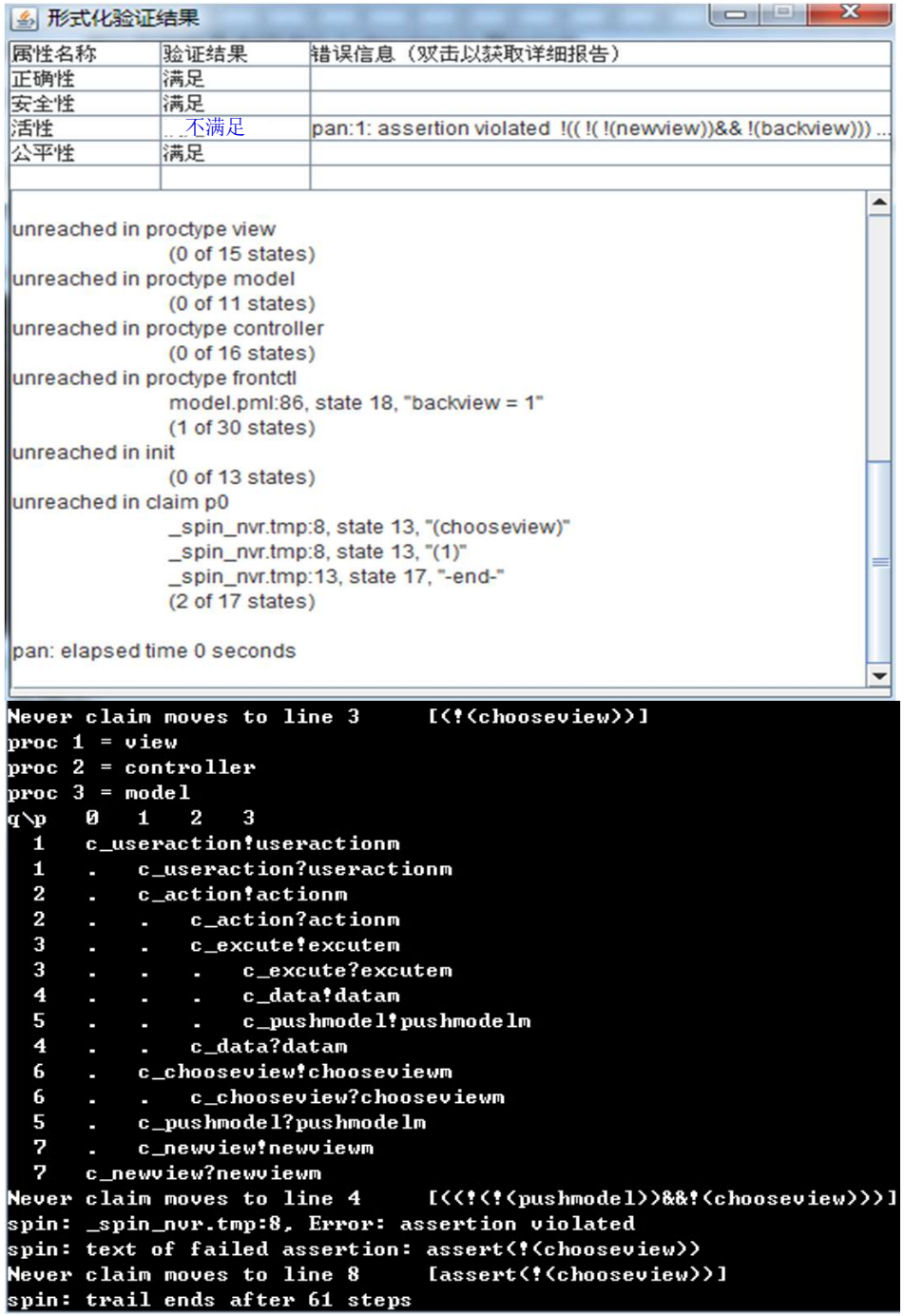


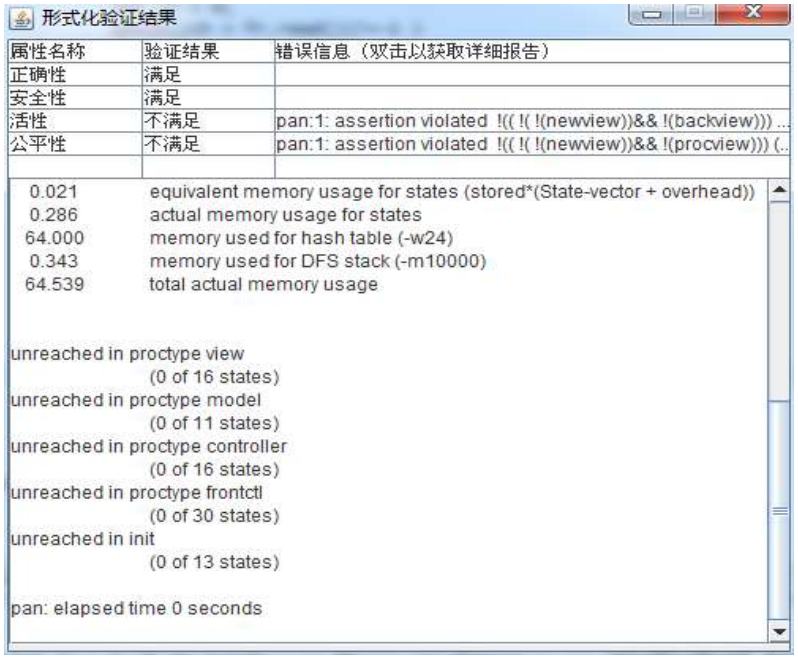
SPIN Model Checker模型检测器
PathFinder
UPPAAL

由MVC至SWMVC的演化序列

	演化操作	具体描述
1	OM	翻转消息chooseview
2	SMO	交换消息chooseview和pushmodel的时序
3	DM	移除消息pushmodel
4	DC	移除关于pushmodel的安全性属性
5	AO	增添新的交互对象frontctl
6	AC(F)	增加关于对象frontctl的公平性约束
7	CMM	改变消息的发送端和接收端，包括useraction，action，newview，chooseview
8	AM	增添新的消息modelandview，backview
9	AC(S&L)	增加关于backview的安全性和活性约束
10	DF	移除复合片段assert
11	AF(loop)	增加复合片段loop
12	FTC	将复合片段loop的类型改变至alt
13	FCC	增加选择条件modelandview

	OP	C	S	L	F
0	Initial	0	0	0	0
1	OM	0	1	0	0
2	SMO	0	1	0	0
3	DM	0	0	0	0
4	DC	0	0	0	0
5	AO	0	0	0	0
6	AC(F)	0	0	0	1
7	CMM	0	0	0	0
8	AM	0	0	0	0
9	AC(S&L)	0	0	1	0
10	DF	0	0	1	0
11	AF	1	0	?	0
12	FTC	0	0	1	0
13	FCC	0	0	0	0






```
proc 1 = view
proc 2 = controller
proc 3 = model
proc 4 = frontctl
q\p   0   1   2   3   4
  1   c_useraction!useractionm
  1   .   .   .   .   c_useraction?useractionm
  2   .   .   .   .   c_action!actionm
  2   .   .   c_action?actionm
  3   .   .   c_excute!excutem
  3   .   .   .   c_excute?excutem
  4   .   .   .   c_data!datam
  4   .   .   c_data?datam
  5   .   .   c_modelandview!modelandviewm
  5   .   .   .   .   c_modelandview?modelandviewm
  6   .   .   .   .   c_c!0
  6   .   c_c?0
  7   .   .   .   .   c_newview!newviewm
  7   c_newview?newviewm
spin: _spin_nvr.tmp:3, Error: assertion violated
spin: text of failed assertion: assert(!((!(!(newview))&&!(backview))))
Never claim moves to line 3     [assert(!((!(!(newview))&&!(backview))))]
spin: trail ends after 72 steps
```

	Operation	C	S	L*	F
0	none(Initial)	0	0	0	0
1	OM	0	1	0	0
2	SMO	0	1	0	0
3	DM	0	0	0	0
4	DC	0	0	0	0
5	AO	0	0	0	0
6	AC(F)	0	0	0	0
7	CMM	0	0	0	0
8	AM	0	0	0	0
9	AC(S&L)	0	0	0	0
10	DF	0	0	0	0
11	AF(loop)	1	0	?	0
12	FTC	0	0	0	0
13	FCC	0	0	1	1

```
proc 1 = view
proc 2 = controller
proc 3 = model
proc 4 = frontctl
q\p   0   1   2   3   4
  1   c_useraction!useractionm
  1   .   .   .   .   c_useraction?useractionm
  2   .   .   .   .   c_action!actionm
  2   .   .   c_action?actionm
  3   .   .   c_excute!excutem
  3   .   .   .   c_excute?excutem
  4   .   .   .   c_data!datam
  4   .   .   c_data?datam
  5   .   .   c_modelandview!modelandviewm
  5   .   .   .   .   c_modelandview?modelandviewm
  6   .   .   .   .   c_c!0
  6   .   c_c?0
  7   .   .   .   .   c_newview!newviewm
  7   c_newview?newviewm
spin: _spin_nvr.tmp:3, Error: assertion violated
spin: text of failed assertion: assert(!((!(!(newview))&&!(backview))))
Never claim moves to line 3     [assert(!((!(!(newview))&&!(backview))))]
spin: trail ends after 72 steps
```

Self-Learning Modeling in Possibilistic Model Checking

Wuniu Liu, Qing He, Zhihui Li, and Yongming Li, *Member, IEEE*

***Abstract*—Generalized possibility Kripke structure (GPKS) plays a key role in modeling fuzzy systems for possibilistic model checking. However, it is unrealistic, in practice, to produce manually a large number of fuzzy states of GPKSs and transition relations over states space. In this article, we develop an online supervised learning algorithm for GPKS to learn its fuzzy states and possibilistic transition matrix. We assume that true fuzzy states of GPKSs are unknown, only available knowledge is the external (sensor's) variables that have ambiguous connections with the states. We connect the sensor's variables to the atomic propositions used to describe the fuzzy states through a family of (Gaussian) fuzzy functions. The first GPKS's learning model is called the GPKS with Fuzzifier Sets (GPKS-FS) that consists of a standard GPKS and a group of fuzzy functions that connect model's states to different sensor variables. The learning algorithm based on stochastic gradient descent is derived to learn all parameters of the (Gaussian) fuzzy functions and all elements of atomic propositions evolution matrix used to explain mechanism of transition between system's states. Based on the evolution matrix, we propose a method of constructing the similarity-based possibilistic transition matrix. This produces the possibilistic transition matrix which is critical for model checking over GPKSs. The learning algorithm do not rely on any subjective information from humans and remove a significant bottleneck for modeling of possibilistic model checking. Computer simulation results showing learning performance.**

system is usually modeled by Boolean transition systems or Boolean state Kripke structures. Those models are useful for the representation and verification in the fields of hardware verification and software engineering, but not appropriate for the systems with uncertain information such as experts systems and medical diagnostic systems. In order to model checking such systems with uncertainty, many quantitative model-checking techniques have been proposed, such as the popular probabilistic model checking [1], multi-valued model checking [3], [4], [5], [6] and possibilistic model checking for quantitative verification of fuzzy systems [6], [7], [8], [9], [10], [11], [12], etc. Possibilistic model checking is developed based on the principles of model checking and possibility theory (possibility theory is an extension of fuzzy sets and logic, introduced by Zadeh in the 1978s [13]), which has been extensively studied by Li et al. in recent years.

The possibilistic model checking is also divided into three phases. The first phase is the modeling of fuzzy systems. The main model used in possibilistic model checking are *possibility Kripke structure* (PKS) [7] and its generalized version—*generalized* PKS (GPKS) [11], and the nondetermin-

ISEU的工作

- Pengfei Duan, Ying Zhou, Xufang Gong, Bixin Li: A Systematic Mapping Study on the Verification of Cyber-Physical Systems. IEEE Access 6: 59043-59064 (2018).
- Ying Zhou, Xufang Gong, Jiakai Li, Bixin Li: Verifying CPS for Self-Adaptability. ICIS 2018: 166-172.
- Ying Zhou, Xufang Gong, Bixin Li, Min Zhu: A Framework for CPS Modeling and Verification Based on dL. ICIS 2018: 173-179.
- Ying Zhou, Min Zhu, Xufang Gong, Bixin Li: Verifying CPS Using DDL. Intelligent Environments (Workshops) 2017: 15-24.
- Bixin Li, Shunhui Ji, Dong Qiu, Hareton Leung, Gongyuan Zhang: Verifying the Concurrent Properties in BPEL Based Web Service Composition Process. IEEE Trans. Network and Service Management 10(4): 410-424 (2013)
- Pengcheng Zhang, Henry Muccini, Bixin Li: A classification and comparison of model checking software architecture techniques. J. Syst. Softw. 83(5): 723-744 (2010).
- Pengcheng Zhang, Bixin Li, Lars Grunske: Timed Property Sequence Chart. J. Syst. Softw. 83(3): 371-390 (2010).
- Pengcheng Zhang, Henry Muccini, Yuelong Zhu, Bixin Li: Model and Verification of WS-CDL Based on UML Diagrams. Int. J. Softw. Eng. Knowl. Eng. 20(8): 1119-1149 (2010)
- 俞析蒙. 基于验证的软件架构演化分析与评估, 东南大学硕士论文, 2016.

基于验证的软件架构演化分析与评估

俞析蒙

东南大学

摘要： 演化反映了“在演化实体或其组成元素的属性方面不断改进的过程”,而软件演化就是指软件系统或内部组成元素不断地改变来满足新的功能需求或属性需求。在现代软件系统的生命周期内,演化是一项贯穿始终的活动,系统需求的改变、功能实现的增强、新型算法的发现、运行环境的改变等等均要求软件系统具有较强的演化能力,这就要求能够保障系统的正确性。而随着软件架构的发展,作为软件生命周期中的早期设计产品,架构从全局和整体的角度为系统提供结构、行为和属性等信息,使得架构设计的演化结果的正确与否十分关键。本文重点关注对于软件架构演化的分析,以及对架构及架构演化的正确性验证。由于架构设计的复杂化,使得对架构正确与否的人工判断工作变得复杂化和难以保证准确性,而模型检验的方法能够有效地验证架构演化的正确与否,来完成这项工作。基于模型检验的工具Spin能够有效地实现对架构设计的正确性验证,但是其所需要的Promela模型无法进行直观的建模工作,不具有通用性,使得在此基础上的演化工作难以进行。因此,这需要我们建立一个架构模型来帮助验证和演化分析工作。本文使用UML (Unified Modeling Language)作为架构...
更多

关键词： 软件架构演化; UML顺序图; 架构验证; 演化操作; 演化分析与评估;

专辑： 信息科技

专题： 计算机软件及计算机应用

分类号： TP311.52

导师： 李必信;

学科专业： 计算机软件与理论

硕士电子期刊出版信息： 年期：2016年第08期 网络出版时间：2016-07-16——2016-08-15

5 模拟[仿真]方法

什么是软件模拟？

软件模拟（Software Simulation）是一种技术或工具，用于模仿真实环境中软件的运行过程。通过建立数学模型，模拟软件的行为和运行情况，开发者可以在软件投入实际运行之前，预测其在各种条件下的表现，检测软件的稳定性、性能以及是否存在潜在问题。

软件模拟的工作原理

软件模拟通过建立数学模型来模拟真实环境中的软件运行过程。这种模拟可以是系统的、网络的，甚至是特定功能的模拟。通过模拟，开发者可以在不同的环境和条件下测试软件，确保软件的适应性和稳定性。

软件模拟的应用场景

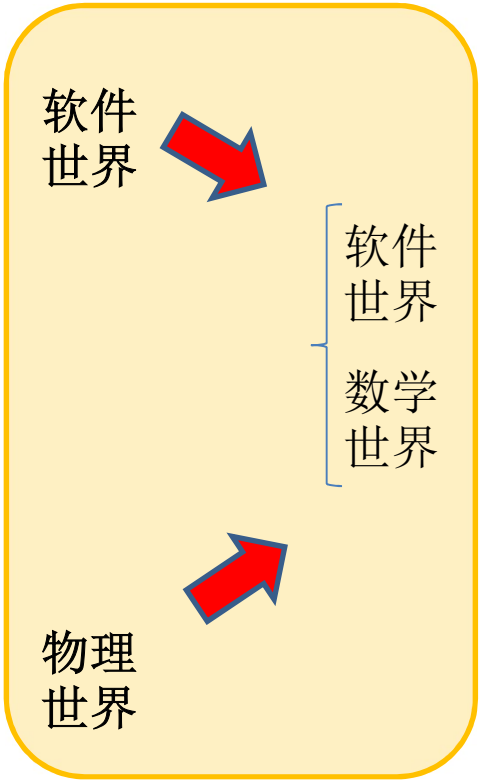
软件模拟广泛应用于多个领域，包括操作系统、数据库管理、网络通信等。特别是在复杂系统或大型软件的研发过程中，模拟软件可以帮助开发者创建一个可控的测试环境，以便更好地了解软件的运行情况并进行优化。此外，模拟软件也可以用来预测和测试软硬件之间的兼容性。

软件模拟的重要性

在现代软件开发中，软件模拟扮演着至关重要的角色。它不仅可以帮助开发者提前发现和解决潜在问题，还可以加速软件的研发周期，提高软件的质量和用户体验。通过模拟，开发者可以在不同的环境和条件下测试软件，确保软件的适应性和稳定性。

基于仿真模拟的软件缺陷检测方法通过在虚拟环境中模拟真实运行场景，动态执行软件并观察其行为，从而发现潜在缺陷。这种方法尤其适用于复杂系统（如嵌入式、分布式系统）或需要高环境依赖的缺陷检测（如硬件交互、网络延迟）。

仿真软件是通过数学模型和计算机算法，模拟和重现真实系统的行为和性能的软件工具。它可以对系统进行多方位、多层次的模拟和分析，以提供全面、准确的系统行为预测。例如， MATLAB Simulink.



- 架构模拟
  - 性能模拟
  - 可维护性模拟
  - 可靠性模拟
- 系统模拟
  - 性能模拟
  - 可维护性模拟
  - 可靠性模拟

A Simulation Model of Software Quality Assurance in the Software Lifecycle

Hiroto Nakahara; Akito Monden; Zeynep Yücel

2021 IEEE/ACIS 22nd International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)

Year: 2021 | Conference Paper | Publisher: IEEE

Evaluation of Software Change Propagation Using Simulation

Lin Li; Guanqun Qian; Li Zhang

2009 WRI World Congress on Software Engineering

Year: 2009 | Conference Paper | Publisher: IEEE

Software maintenance process analysis using discrete-event simulation

I. Podnar; B. Mikac

Proceedings Fifth European Conference on Software Maintenance and Reengineering

Year: 2001 | Conference Paper | Publisher: IEEE

Simulation-Based Logical Scenario Generation and Analysis Methodology for Evaluation of Autonomous Driving Systems

Jongwon Jeon; Jaeyeon Yoo; Taeyoung Oh; Jinwoo Yoo

IEEE Access

Year: 2025 | Early Access Article | Publisher: IEEE

From Modeling to Simulation: A Head Start for Digital Ecosystem Design

Matthias Koch

IEEE Software

Year: 2025 | Volume: 42, Issue: 2 | Magazine Article | Publisher: IEEE

Search-based DNN Testing and Retraining with GAN-enhanced Simulations

Mohammed Oualid Attaoui; Fabrizio Pastore; Lionel C. Briand

IEEE Transactions on Software Engineering

Year: 2025 | Early Access Article | Publisher: IEEE

The *Why*, *When*, *What*, and *How* About Predictive Continuous Integration: A Simulation-Based Investigation

Bohan Liu, He Zhang, Weigang Ma, Gongyuan Li, Shanshan Li, and Haifeng Shen, *Senior Member*

***Abstract*—Continuous Integration (CI) enables developers to detect defects early and thus reduce lead time. However, the high frequency and long duration of executing CI have a detrimental effect on this practice. Existing studies have focused on using CI outcome predictors to reduce frequency. Since there is no reported project using predictive CI, it is difficult to evaluate its economic impact. This research aims to investigate predictive CI from a process perspective, including why and when to adopt predictors, what predictors to be used, and how to practice predictive CI in real projects. We innovatively employ Software Process Simulation to simulate a predictive CI process with a Discrete-Event Simulation (DES) model and conduct simulation-based experiments. We develop the Rollback-based Identification of Defective Commits (RIDEC) method to account for the negative effects of false predictions in simulations. Experimental results show that: 1) using predictive CI generally improves the effectiveness of CI, reducing time costs by up to 36.8% and the average waiting time before executing CI by 90.5%; 2) the time-saving varies across projects, with higher commit frequency projects benefiting more; and 3) predictor performance does not strongly correlate with time savings, but the precision of both failed and passed predictions should be paid more attention. Simulation-based evaluation helps identify overlooked aspects in existing research. Predictive CI saves time and resources, but improved prediction performance has limited cost-saving benefits. The primary value of predictive CI lies in providing accurate and quick feedback to developers, aligning with the goal of CI.**

I. Introduction

CONTINUOUS Integration (CI) is a software development practice that automates the compilation, building, and testing of source code on dedicated servers, which makes it possible to accelerate development whilst securing quality. The benefits of CI, such as reducing merge conflicts, have been widely recognized [1], [2], [3], [4]. The annual GitLab Global Developer Survey[1] shows that only 10% respondents did not use CI tools. The Annual State of DevOps Survey[2] also asserts that "*CI is a must-do in the DevOps space, right after version control becomes ubiquitous*".

Some studies have identified the challenges practitioners are facing. A systematic literature review [5] suggests that the most critical testing problem in CI is the excessive consumption of time. Since developers need to wait for the CI process to complete before engaging in other development activities, a long duration of CI means not only the consumption of substantial computing resources, but also human resources [6], [7], [8]. Fowler indicated that every minute of reducing CI execution time is a minute saved for developers every time they commit code [9]. Among the 104,442 CI executions over 67 projects in TravisTorrent, 40% took more than 30 minutes [10], implying

Real-Time Adaptive Allocation of Emergency Department Resources and Performance Simulation Based on Stochastic Timed Petri Nets

Jun Wang and Jiacun Wang, *Senior Member, IEEE*

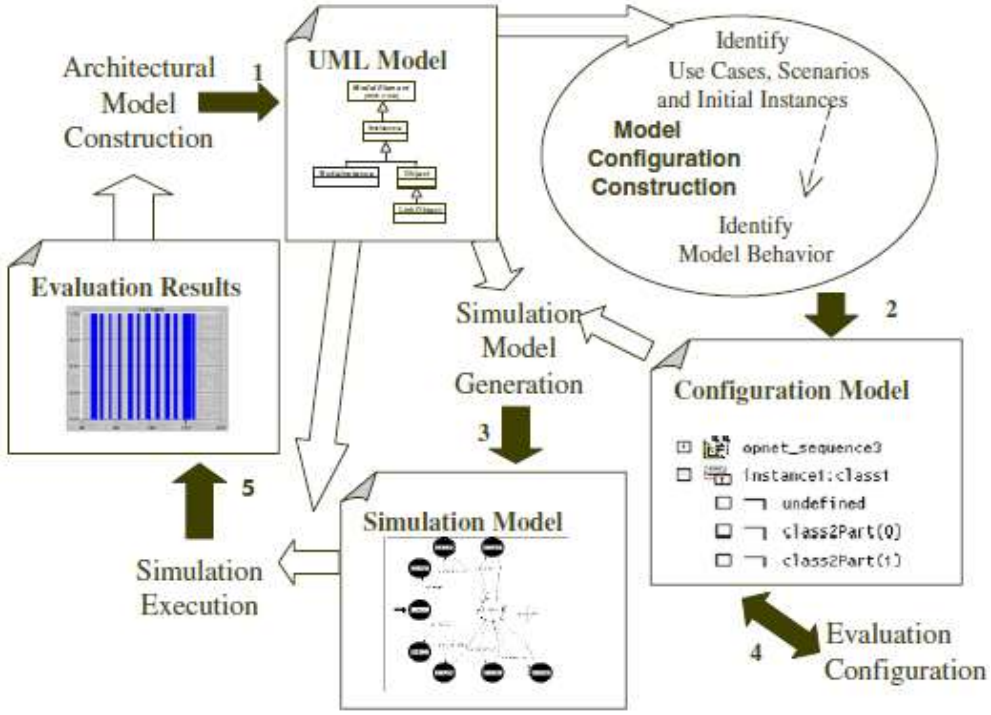
***Abstract*— Overcrowding in emergency departments (EDs) is a common problem encountered by healthcare systems worldwide. Its essence is the imbalance between the need for emergency care and available resources, such as doctors, nurses, medical supplies, and treatment facilities and spaces. Such an imbalance increases with the volume of visiting patients. To solve the problem of ED overcrowding, service providers need to ensure rational allocation of resources in the emergency process to the greatest extent. This article uses stochastic timed Petri nets (STPNs) as a modeling and simulation tool to optimize the resource allocation in the emergency care workflow. On the basis of STPN simulation architecture, we propose a novel "observation-response" block (ORB) to adaptively supplement the corresponding resources according to the local crowding situation in the emergence workflow, so as to reduce the waiting time of patients in urgent need of treatment. In this article, models of patient arrival, triage, and examination process are constructed. Then, considering the waiting time of patients as the optimization objective, the statistical simulation based on STPN models is performed to verify the effectiveness of the proposed ORB block in the emergency workflow resource optimization process. The presented work provides a feasible way for the optimal ED resource allocation.**

***Index Terms*— Adaptive resource allocation, emergency department (ED), observation-response block (ORB), overcrowding, stochastic timed Petri nets (STPNs), workflow simulation.**

at night. Thus, the core problem of ED overcrowding is the imbalance between the need for emergency care and available resources [5]. How to effectively give priority to save the lives of critically ill or injured patients without affecting the medical waiting time of noncritically patients requires the optimization of the emergency process and resource allocation (doctors, nurses, beds, medical equipment and drugs, etc.) and the efficient operation of the emergency system.

With the advances in science and technology, the current healthcare systems in China are gradually improving and have been able to deal with many problems, such as high-volume patient flow and efficient medical resource allocation in medical systems. Healthcare systems typically use QR codes for patent registration and check-in. However, uneven distribution of medical resources and the excessive number of attendances in large service providers make the effective consultation time of many patients too short and the waiting time for consultation too long, which leads to congestion in some outpatient wards in hospitals and a significant reduction in the efficiency of patient access [6]. According to a survey, the effective consultation time of patients during outpatient visits accounts for approximately 10%–15% of the outpa-

- 架构仿真的原理和步骤



架构仿真的五个阶段：

（1）**架构建模**：利用CASE工具建立架构模型，完成模型后利用XMI发生器输出模型信息，这些信息将在仿真场景构造和仿真执行中使用。

（2）**仿真配置**：对仿真模型进行配置，包含两个基本概念：仿真模型中包含的初始实例（initial instance）和应用程序元素在执行时用到的行为（behaviors）。其中，初始实例可在一个模型中利用两个时序图表示两个不同的执行场景，这两个时序图包含不同的对象实例，这样对初始实例的验证允许包含对象分布的初始仿真场景的构造。另一方面，就行为配置而言，状态机选择（state machine selection）在当有不同的候选时，选择模型元素的行为。

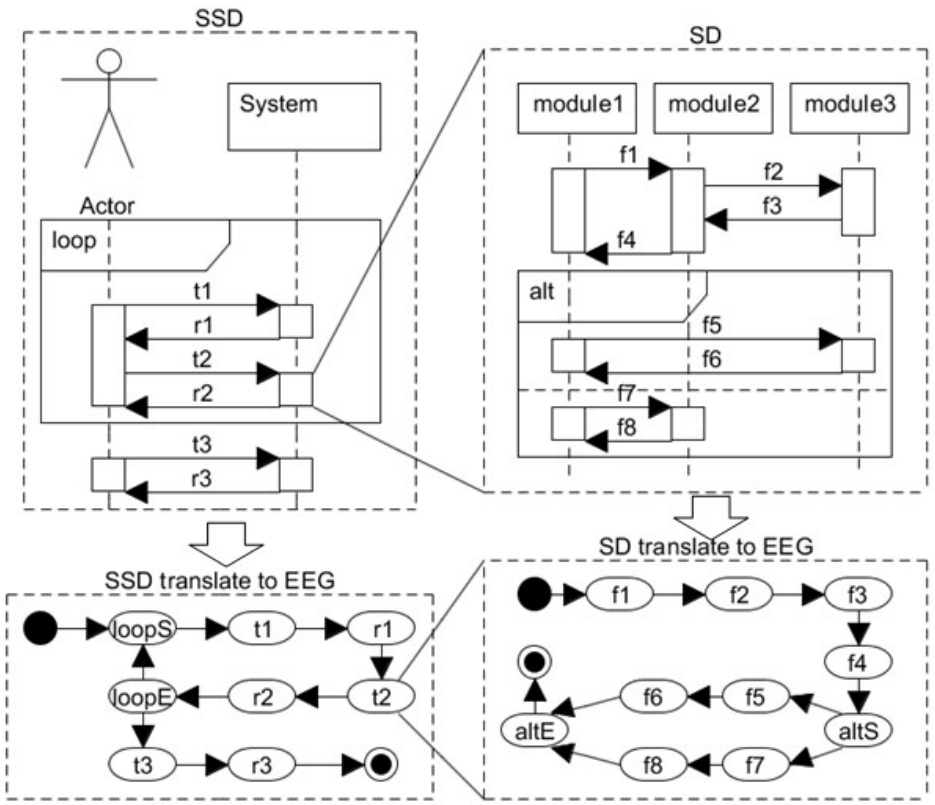
（3）**仿真产生**：配置完成仿真模型的产生。

（4）**配置的评价**：在配置时，在评估时使用过滤器选择想评价的统计值，并配置仿真时间等仿真参数。

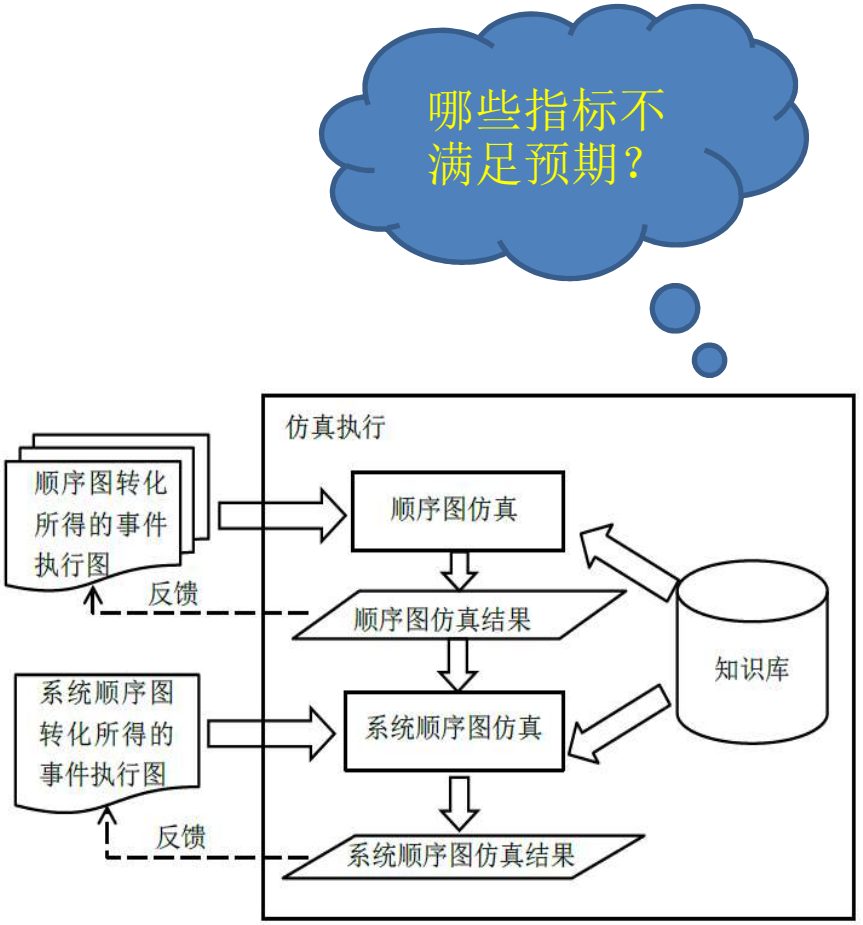
（5）**结果评估**：模型执行以后，对统计值分析。在分析时比较不同的值。图形化的表示等，这些分析结果可在UML架构重新配置时使用，或者可用作选择架构的标准。

1. de Miguel M, Lambolais T, Piekarec S, et al. Automatic generation of **simulation models** for the evaluation of performance and reliability of architectures specified in UML. Engineering Distributed Objects, Lecture Notes in Computer Science, 83-101. 2001.

ISEU成果：软件架构仿真



UML图转化为事件执行图



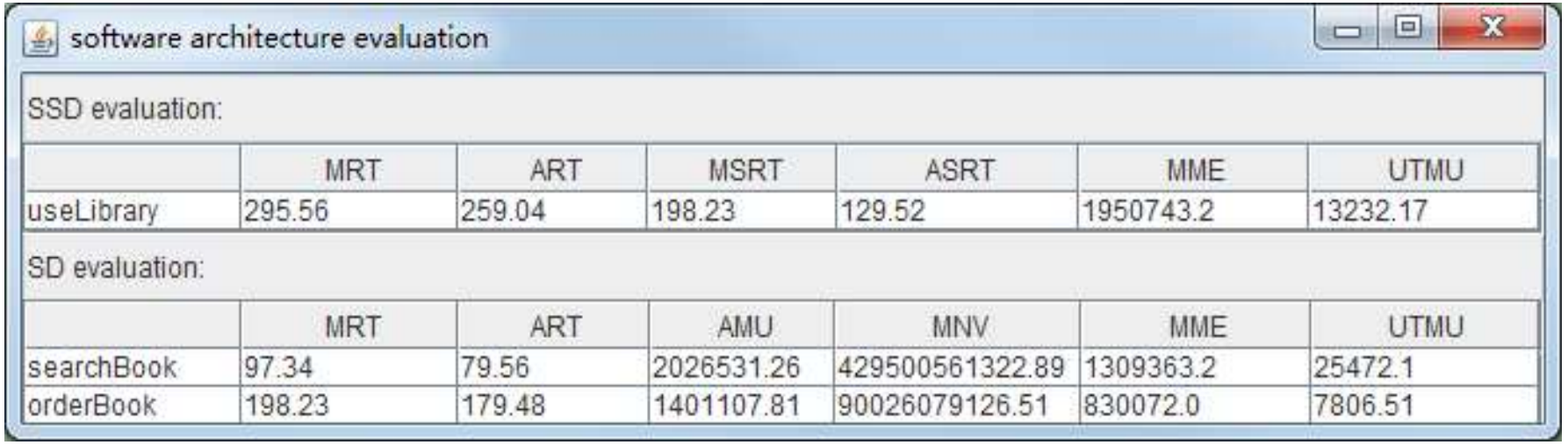
仿真执行流程图

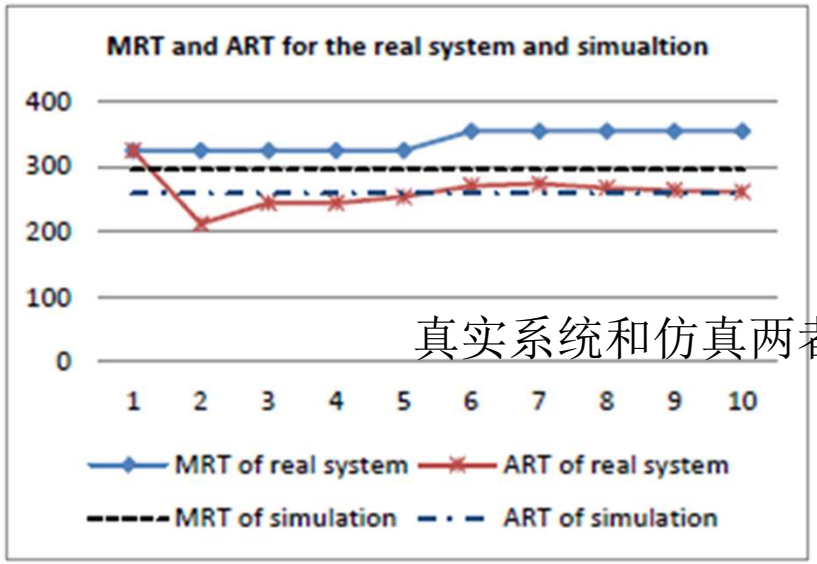
SD仿真指标

仿真指标	描述	标签
最大运行时间	所有事件序列中最大运行时间	MRT
平均运行时间	所有事件序列运行时间的平均值	ART
平均内存利用量	所有事件序列内存利用量的平均值	AMU
内存利用平衡率	模块间累计内存利用量的方差	MNV
最大内存占用量	所有事件序列执行过程中，内存最大占用量	MME
单位时间内存利用量	所有事件序列平均单位时间内存利用量	UTMU

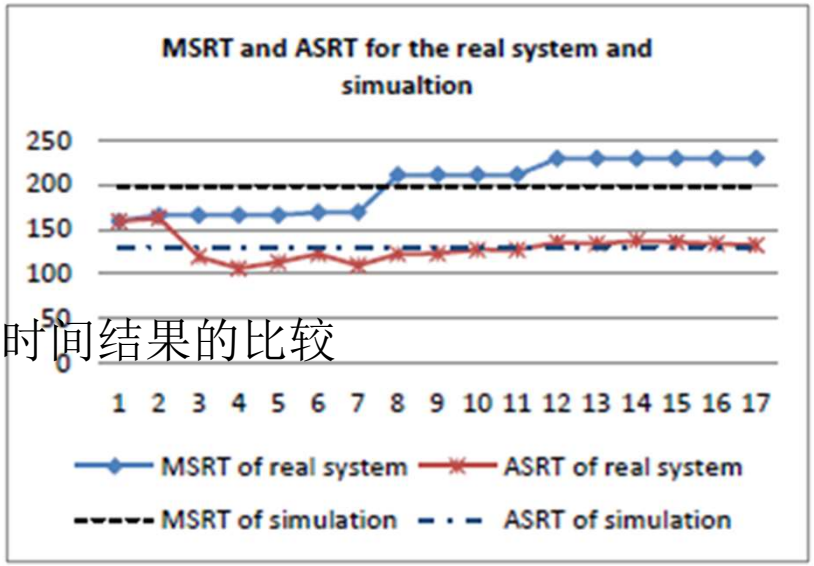
SSD仿真指标

仿真指标	描述	标签
最大运行时间	所有交互序列中，用户与系统交互的最长时间	MRT
平均运行时间	所有交互序列所需要的平均时间	ART
最大响应时间	所有用户操作中，系统对用户的最大响应时间	MSRT
平均响应时间	所有用户操作中，系统对用户的平均响应时间	ASRT
最大内存占用量	所有交互序列执行过程中，内存最大占用量	MME
单位时间内存利用量	所有交互序列平均单位时间内存利用量	UTMU



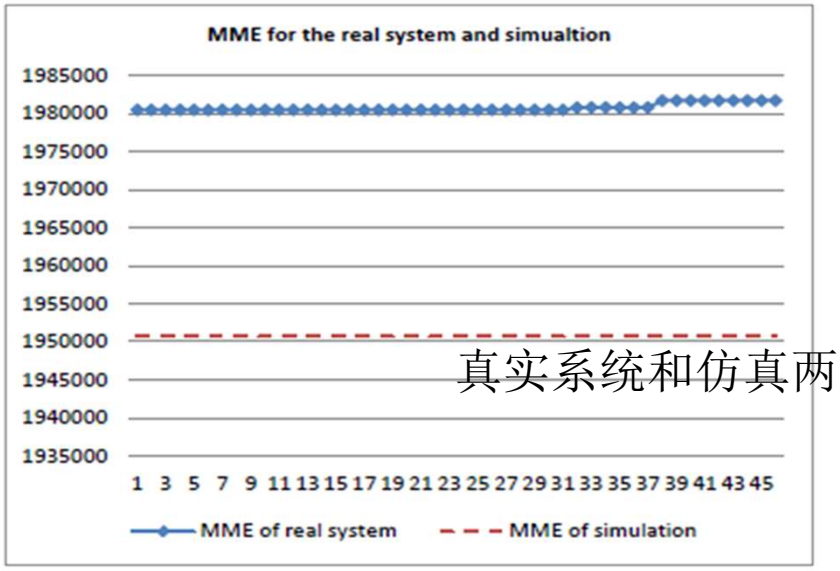


(a) 真实系统和仿真关于MRT和ART的误差

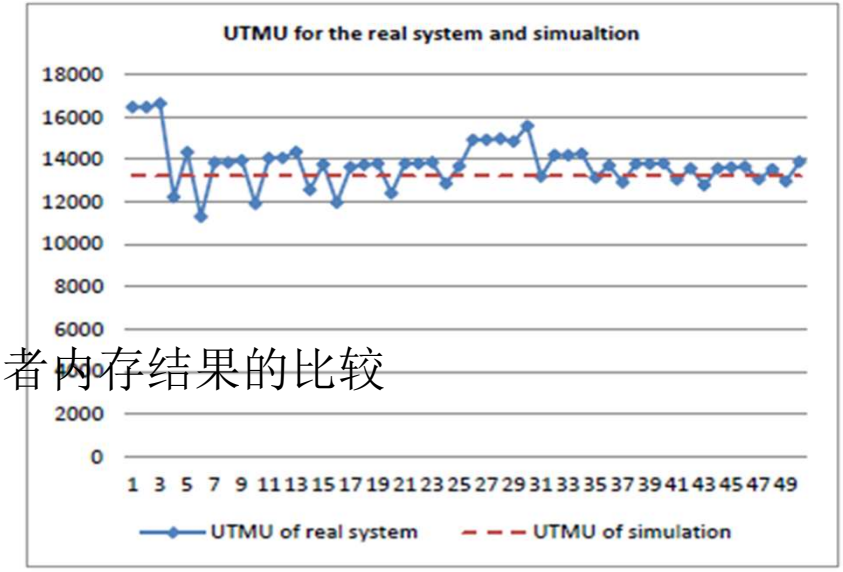


(b) 真实系统和仿真关于MSRT和SART的误差

真实系统和仿真两者时间结果的比较



(a) 真实系统和仿真关于MME的误差



(b) 真实系统和仿真关于UTMU的误差

真实系统和仿真两者内存结果的比较

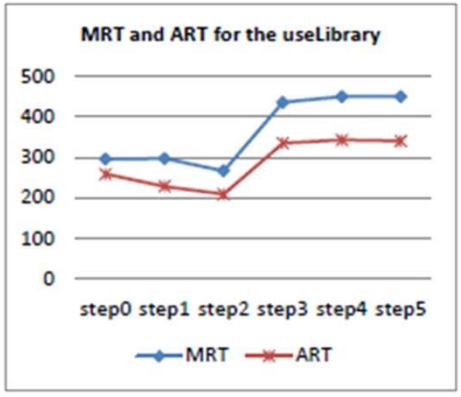
software architecture evolution evaluation

SSD evolution evaluation:

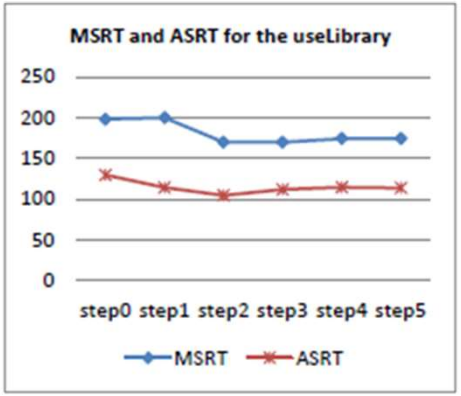
	MRT	ART	MSRT	ASRT	MME	UTMU
useLibrary	295.56	259.04	198.23	129.52	1950743.2	13232.17
useLibrary'	450.8	340.45	174.35	113.48	2081671.2	11178.56
useLibrary rv	52.52%	31.43%	-12.05%	-12.38%	6.71%	-15.52%

SD evolution evaluation:

	MRT	ART	AMU	MNV	MME	UTMU
searchBook	97.34	79.56	2026531.26	429500561322.89	1309363.2	25472.1
searchBook'	102.1	98.84	1579922.5	313000000000.0	1309363.2	15984.43
searchBook rv	4.89%	24.24%	-22.04%	-27.12%	0.0%	-37.25%
orderBook	198.23	179.48	1401107.81	90026079126.51	830072.0	7806.51
orderBook'	174.35	120.81	1112936.64	75400000000.0	950000.0	9212.54
orderBook rv	-12.05%	-32.69%	-20.57%	-16.25%	14.45%	18.01%

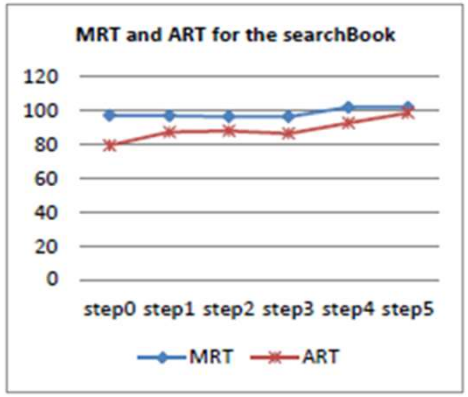


(a) 在线图书馆系统的MRT和ART变化情况

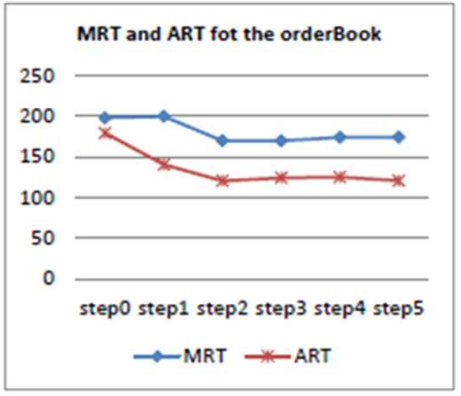


(b) 在线图书馆系统的MSRT和ASRT变化情况

SAE导致的系统整体的时间属性变化情况

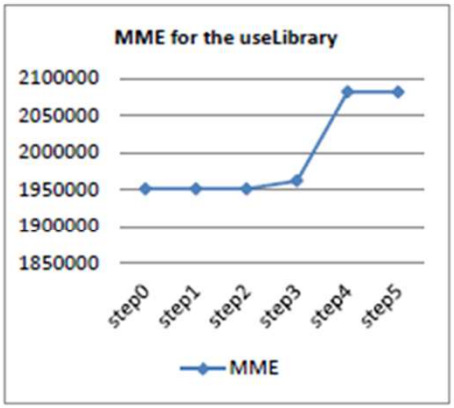


(a) 搜索书籍功能块的MRT和ART变化情况

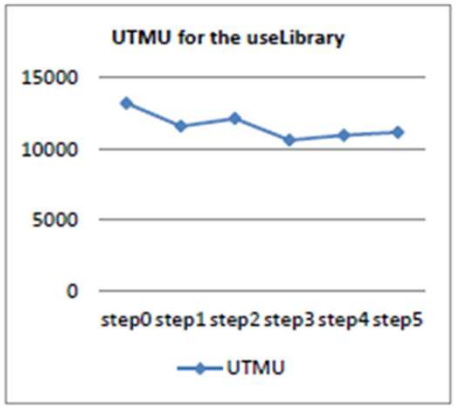


(b) 预定书籍功能块的MRT和ART变化情况

SAE导致的局部功能块的时间属性变化情况

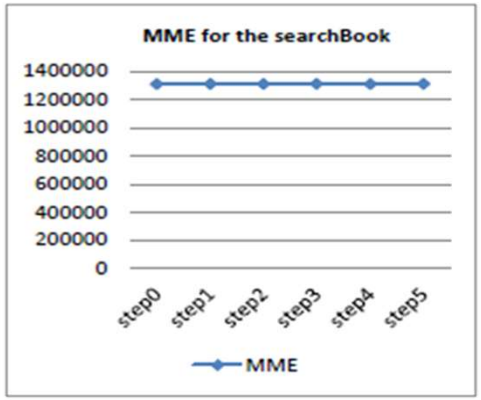


(a) 在线图书馆系统的MME变化情况

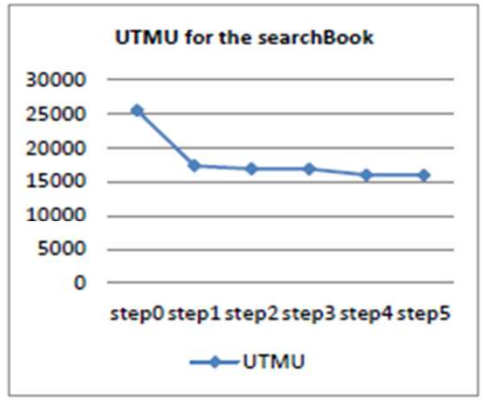


(b) 在线图书馆系统的UTMU变化情况

SAE导致的系统整体的时间属性变化情况

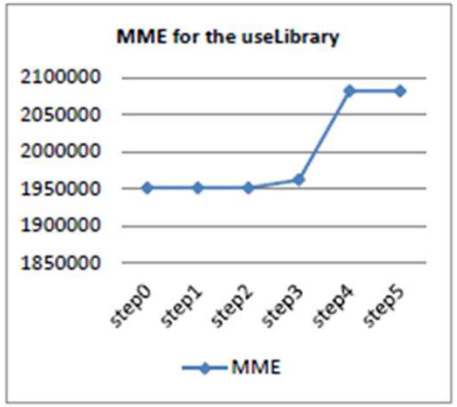


(a) 搜索书籍功能块的MME变化情况

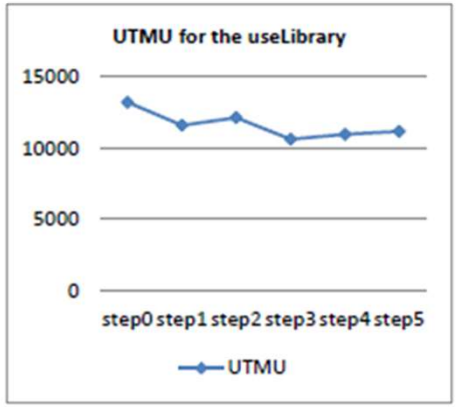


(b) 搜索书籍功能块的UTMU变化情况

SAE导致的局部功能块的时间属性变化情况

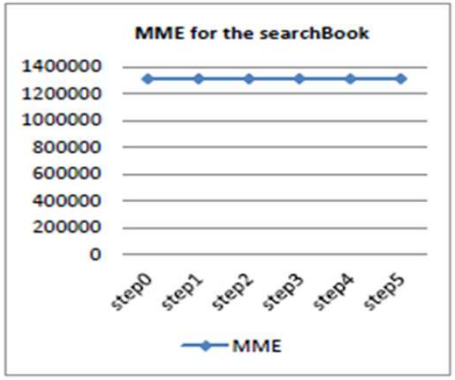


(a) 在线图书馆系统的MME变化情况

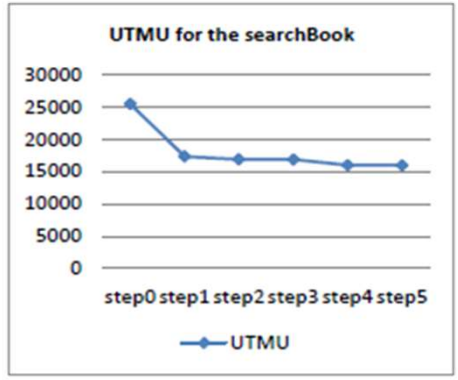


(b) 在线图书馆系统的UTMU变化情况

SAE导致的系统整体的MME和UTMU变化情况



(a) 搜索书籍功能块的MME变化情况



(b) 搜索书籍功能块的UTMU变化情况

SAE导致的搜索书籍功能块的MME和UTMU变化情况

6 测试方法

什么是软件测试？

软件测试（Software Testing）是一种用于评估软件产品质量的活动过程，旨在通过执行软件的各个功能、检查程序的行为等操作，发现软件中的缺陷（bugs）、错误（errors）或者不符合需求规格说明书的地方。其目的是确保软件产品能够满足用户需求、具有较高的质量和可靠性。

软件测试的意义

软件测试是软件工程的重要组成部分，是软件质量保证的重要前提。其目标是尽早发现并修复软件中的错误和缺陷，确保软件的功能和性能与需求说明相符合。软件测试贯穿整个软件开发生命周期，通过验证和确认活动过程，确保软件的质量。

软件测试的目标和原则

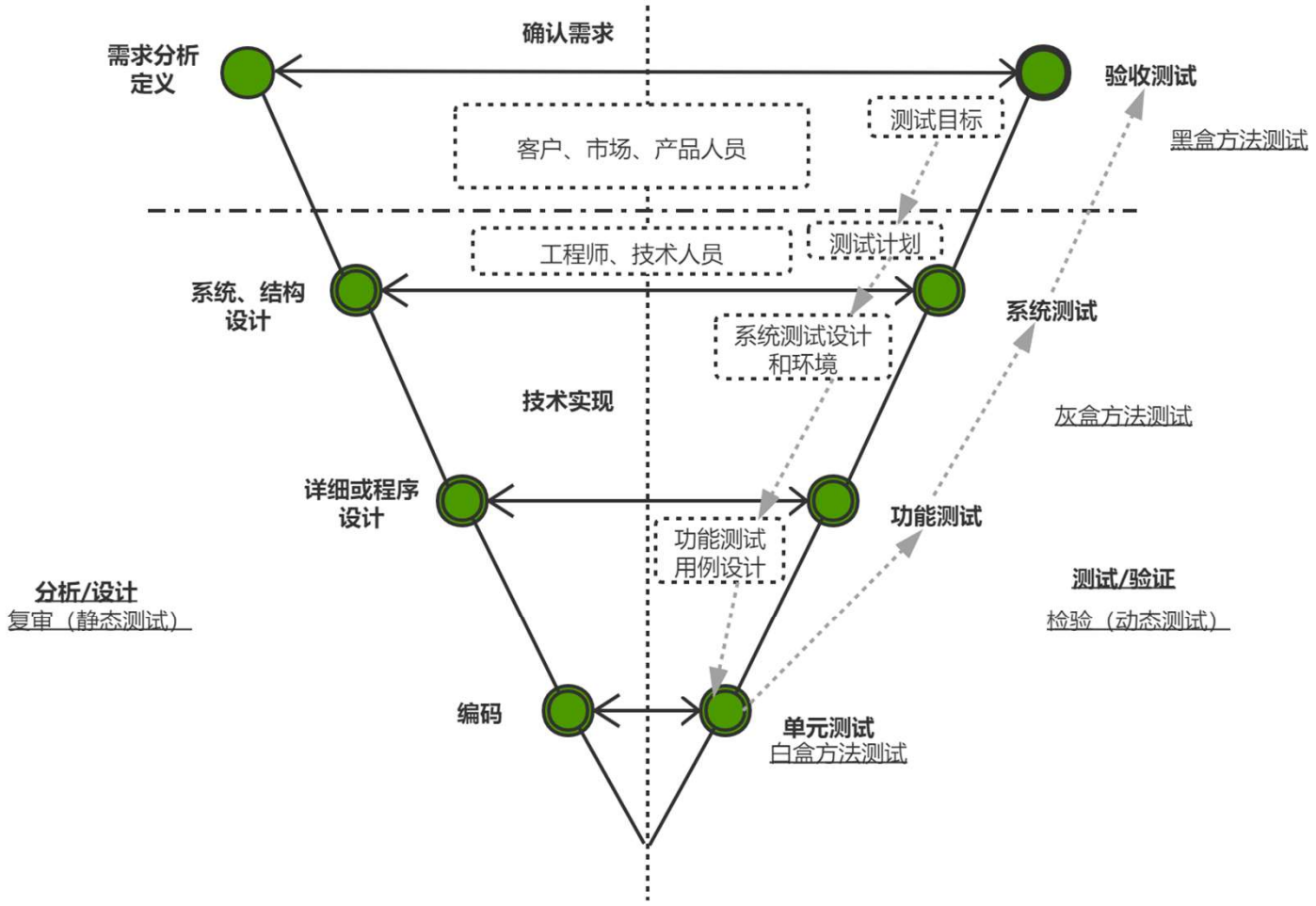
目标：以最少的时间和人力，尽可能多地发现程序中的错误和缺陷，并证明软件的功能和性能与需求说明相符合。

原则：

1）尽早和不断地进行测试，问题发现的越早，解决问题的代价就越小。

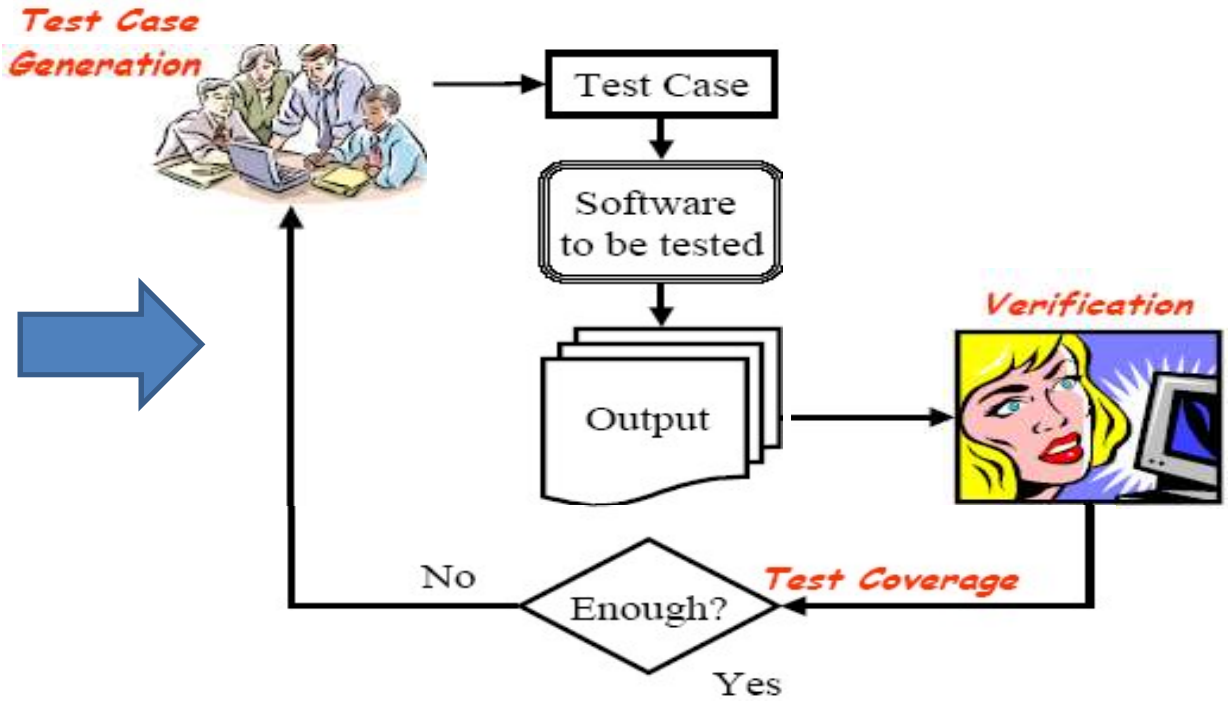
2）测试应追溯到用户需求，从“小规模”开始，逐步转向“大规模”，并对每个测试结果做全面检查。

- 软件测试（software testing），是通过在某些特定输入下运行软件，观察实际输出结果是否满足预期结果的一种技术手段。
- 软件测试的经典定义是：在规定的条件下对程序进行操作，以发现程序错误，衡量软件质量，并对其是否能满足设计要求进行评估的过程。
- 软件缺陷发现的主要手段之一。



一个典型的软件测试过程

一个典型的软件测试过程



三个核心关注点：

（1）测试用例生成；

（2）测试结果的验证（确认）：看是否满足预期结果（test oracle）

（3）测试覆盖的充分性评估：评估测试数据/测试用例的充分性

ISEU的工作

- 吉顺慧. 基于演化的Web组合服务测试和验证, 东南大学博士论文,2016.
- Shunhui Ji, Bixin Li, Pengcheng Zhang: Test Case Selection for All-Uses Criterion-Based Regression Testing of Composite Service. IEEE Access 7: 174438-174464 (2019)
- Shunhui Ji, Bixin Li, Pengcheng Zhang: Test Case Selection for Data Flow Based Regression Testing of BPEL Composite Services. SCC 2016: 547-554
- Dong Qiu, Bixin Li, Shunhui Ji, Hareton K. N. Leung: Regression Testing of Web Service: A Systematic Mapping Study. ACM Comput. Surv. 47(2): 21:1-21:46 (2014)
- Chuanqi Tao, Bixin Li, Jerry Gao: A Systematic State-Based Approach to Regression Testing of Component Software. J. Softw. 8(3): 560-571 (2013)
- Chuanqi Tao, Bixin Li, Jerry Gao: Testing Configurable Architectures For Component-Based Software Using an Incremental Approach. SEKE 2013: 356-361
- Bixin Li, Dong Qiu, Hareton Leung, Di Wang: Automatic test case selection for regression testing of composite service based on extensible BPEL flow graph. J. Syst. Softw. 85(6): 1300-1324 (2012)
- Xiaobing Sun, Bixin Li, Chuanqi Tao, Qiandong Zhang: Using FCA-based Change Impact Analysis for Regression Testing. SEKE 2012: 452-457
- Chuanqi Tao, Bixin Li, Jerry Gao: A Model-based Approach to Regression Testing of Component-based Software. SEKE 2011: 230-237
- Zhengshan Wang, Bixin Li, Lulu Wang, Qiao Li: A Brief Survey on Automatic Integration Test Order Generation. SEKE 2011: 254-257
- Bixin Li, Shunhui Ji, Dong Qiu, Ju Cai: Generating Test Cases of Composite Services Based on OWL-S and eh-CPN. Int. J. Softw. Eng. Knowl. Eng. 20(7): 921-941 (2010)
- Bixin Li, Dong Qiu, Shunhui Ji, Di Wang: Automatic test case selection and generation for regression testing of composite service based on extensible BPEL flow graph. ICSM 2010: 1-10

ReDefender: Detecting Reentrancy Vulnerabilities in Smart Contracts Automatically

Bixin Li, Zhenyu Pan, and Tianyuan Hu

***Abstract*—As one of the most complex types of vulnerabilities, reentrancy poses a significant threat to smart contract development. Indeed, millions of dollars have evaporated due to reentrancy vulnerabilities of smart contracts in past years. In this article, we propose a new approach to detect reentrancy vulnerabilities using fuzz testing and develop a novel tool named ReDefender. Our approach consists of three main steps: 1) *preprocess contract to be detected:* when a contract is uploaded, its source code will be preprocessed to extract candidate pool for fuzzing and dependency graph which guides the automatic deployment of contracts; 2) *fuzzing input generation:* fuzzing input is generated to constitute transactions which will be sent to an agent contract to stimulate attacks, where runtime information is collected and recorded in the execution log during each execution; and 3) *vulnerability verification:* the execution log is analyzed to determine whether a reentrancy process occurs and whether the reentrancy process is malicious. We conduct comparative experiments on 204 tagged smart contracts and 90 injected contracts. The results show higher accuracy and lower false negative rate of ReDefender than that of the other three famous tools. Moreover, we conduct an experiment on 4776 real-world contracts demonstrating the ability of ReDefender to find reentrancy vulnerabilities that really cause economic losses.**

***Index Terms*—Blockchain, ethereum, fuzzing, reentrancy, smart contract.**

are mainly written in Solidity language [7], which is Turing-complete unlike the scripting language adopted by Bitcoin. Similar to many other systems, failures caused by their software operations can have very severe consequences, including property damage, monetary loss, etc. [8], [9]. While expanding the application scope of blockchain, smart contracts also raise numerous security issues. Noted by several pieces of research [10], [11], due to the complexity of Solidity syntax and the characteristics of Ethereum virtual machine (EVM), it is an extremely difficult task to write smart contracts free from vulnerabilities.

Reentrancy is one of the most notorious vulnerabilities in smart contracts. As an infamous attack leading to the Ethereum main chain hard fork, the DAO attack in 2016 resulted in more than 60 million dollars in losses. Since then, many researchers [11]–[15] have analyzed the inner logic of reentrancy vulnerabilities and proposed various tools to detect and mitigate them. Unfortunately, although many researchers claim that they can detect reentrancy vulnerabilities accurately, according to a recent research [16], most of the state-of-the-art analysis tools are dissatisfactory with low effectiveness and high false-positive rate. The majority of existing tools use static analysis tools [16].

EvoFuzzer: An Evolutionary Fuzzer for Detecting Reentrancy Vulnerability in Smart Contracts

Bixin Li, Zhenyu Pan, and Tianyuan Hu

***Abstract*—Reentrancy vulnerability is one of the most serious security issues in smart contracts, resulting in millions of dollars in economic losses and posing a threat to the trust of the blockchain ecosystem. Therefore, researchers are paying more attention to this problem and have proposed various methods to detect and eliminate potential reentrancy vulnerabilities before contract deployment. Compared to symbolic execution and pattern-matching methods, fuzz testing method can achieve higher accuracy and are better suitable for detecting cross-contract vulnerabilities. However, existing fuzz testing tools often spend a long time exploring states with little pruning, and most of them adopt the reentrancy vulnerability oracle used by static analysis tools, which ignores whether the vulnerability can be exploited to compromise the access control, mutex, or time locks. To address these issues, we propose EvoFuzzer, an evolutionary fuzzer that focuses on the detection of reentrancy vulnerabilities. EvoFuzzer first leverages static analysis to exclude branches that have no impact on state transitions, then continuously optimizes test case generation using a genetic algorithm that considers both function sequence and parameter assignment, and Meanwhile, EvoFuzzer confirms whether reentrancy vulnerabilities can be exploited by simulating attacks. Our experiments have performed on 198 annotated contracts and 47 honeypot contracts, and experimental results show that EvoFuzzer can detect 91.7% of reentrancy vulnerabilities with no false positives, achieve the highest F1 score with 95.7%, which is 5.9% higher than the next best approach (Confuzzius), and we also find that it reduces more than 10% of branches when EvoFuzzer adopts a pruning strategy.**

***Index Terms*—Blockchain, smart contract, reentrancy, fuzz testing.**

During vulnerability detection tasks, reentrancy vulnerability detection is a critical issue in smart contract security. Reentrancy vulnerabilities are often intertwined with access control and race conditions on state variables, making their detection be coming a big challenge. The complexity of scenarios resulting from cross-contract interactions further exacerbates the problem. Effective detection of reentrancy vulnerabilities requires generating meaningful test cases to explore branches deeply, simulate cross-contract interactions, and collect sufficient runtime information for verification. If a tool performs well in the above aspects, it can be extended to identify various types of vulnerabilities by changing the validation specification [10], [11], [12]. Numerous researchers have focused on reentrancy vulnerabilities [13], [14], [15], [16], [17], however, there is still a large space for improvement [7], [18].

The existing methods which declare to protect the security of smart contracts on the Ethereum platform can be divided into three categories. The first category is *formal verification*, which uses model checking or theorem proving to comprehensively analyze smart contract attributes. However, few tools can support all Solidity [19] syntaxes, and defining vulnerability specifications is often reliant on expert understanding [20], [21], [22], [23]. The second category is *static analysis*, which includes pattern matching and symbolic execution. Pattern matching tools [24], [25], [26] lose information when converting smart contract source code to intermediate representations, while

7 监控方法

什么是软件运行监控？

软件运行监控是指通过使用专门的软件工具或系统，对软件应用程序的运行状况进行实时监测和控制，以确保其性能、可用性和安全性。这种监控通常包括对软件运行状态的实时跟踪、性能指标的记录与分析、错误检测和异常处理等功能。

软件运行监控的作用

其主要目的是确保软件的稳定运行、及时发现并解决问题，从而提高系统的可靠性和用户体验。具体作用包括：

性能监测：监控软件的运行速度、响应时间等性能指标，帮助开发者优化软件性能。

可用性监测：检测软件是否可用，及时处理服务中断等问题。

错误检测与处理：及时发现并处理软件运行中的错误和异常，防止系统崩溃或数据丢失。

安全监控：监测潜在的安全威胁，保护系统免受攻击。

实现方法和技术手段

实现软件运行监控的方法和技术手段包括：

日志分析：通过记录和分析软件的运行日志，发现潜在的问题和异常。

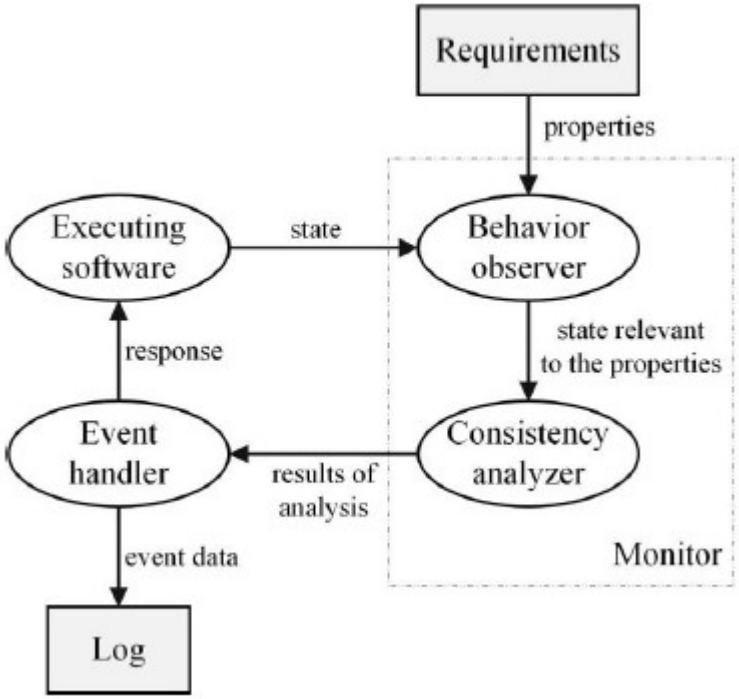
性能监控工具：使用专门的性能监控工具，如域智盾系统和网控堡垒系统，实时监测软件的运行状态和性能指标。

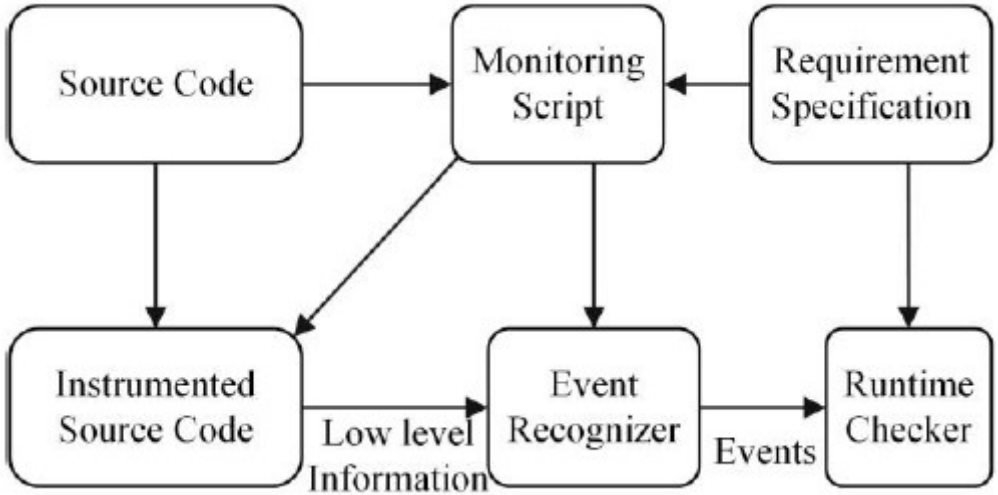
错误处理机制：建立错误处理机制，自动处理或报警异常情况。

安全监控系统：部署安全监控系统，检测和防范安全威胁。

通过这些方法和手段，可以有效地对软件运行进行监控，确保其稳定、高效和安全地运行。

- Runtime Monitoring：
  - Instrument
  - Tracing Program Execution
  - Aspect Programming
  - Logging
  - Observer模式



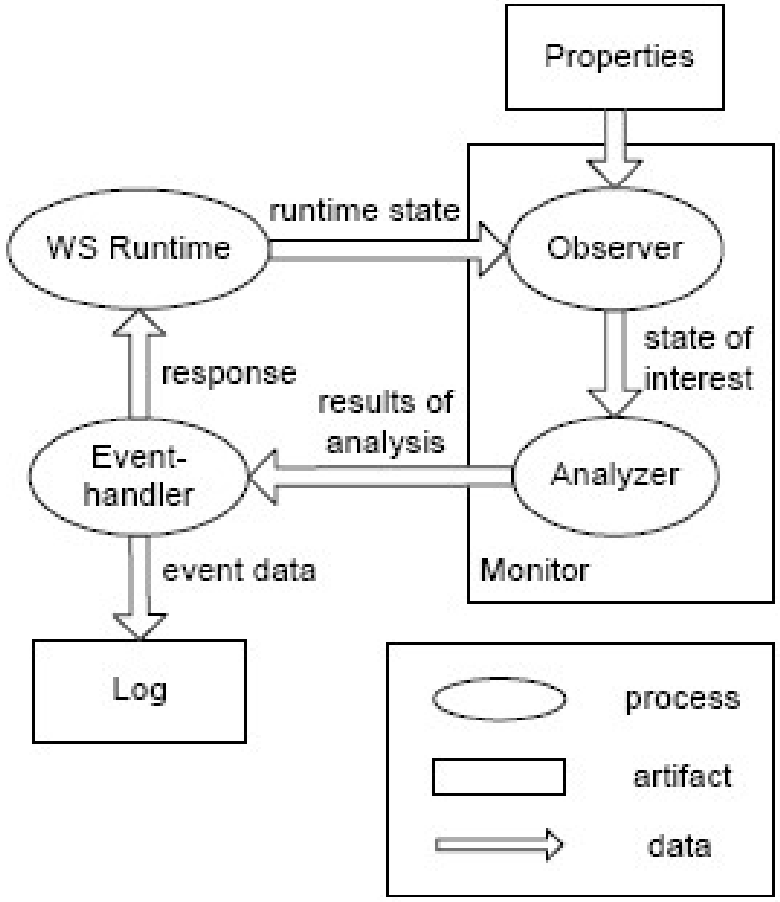


典型的监控流程

Monitoring Mode		Features	Monitoring Level	Typical Research	Language
Off-line runtime monitoring		1. Adopted in early works. 2. It cannot deal with monitoring information in real- time.	Function/class level	N/A	
			Module level	N/A	
			Architecture level	[4]The noninterference monitoring and replay mechanism	
			Subsystem level	[5]The monitoring and debugging for distributed real-time programs	C
Online runtime monitoring	Non-intrusive runtime monitoring	1. The monitor runs independently of the monitored software, which does not occupy the resources of the monitored system. 2. Needn't modify the monitored software system. 3. But it cannot obtain the internal state.	Function/class level	N/A	
			Module level	N/A	
			Architecture level	[6]Bro	Java
				[7]NER	Java
			Subsystem level	[8]BusMOP	C
	Intrusive runtime monitoring	1. It can effectively monitor the internal and external state information of the software at runtime. 2. But it can lead to time or resource overhead.	Function/class level	[9]Anna CCS	Ada
				[10]Runtime monitoring of real-time system	C
				[13]Design By Contract	Eiffel
				[15]Jass	Java
				[16]jContractor	Java

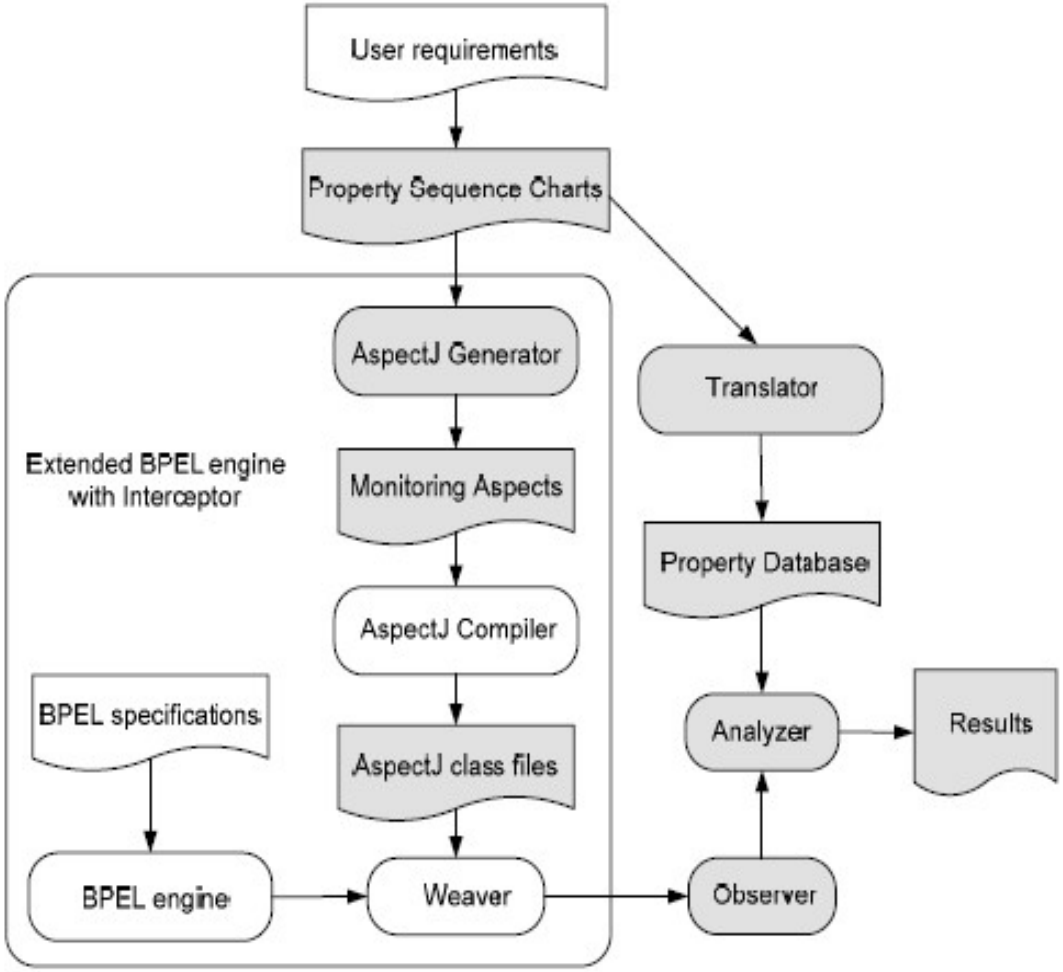
			Module level	N/A	
			Architecture level	N/A	
			Subsystem level	[11]The runtime monitoring for timing constraints in distributed real-time systems	C
	Runtime monitoring combined with intrusive and non-intrusive methods	1. Most current research focuses on this kind of combined method. 2. It can effectively monitor the internal and external state information of the software at runtime. 3. It can reduce the runtime overhead caused by monitoring, to some extent.	Function/class level	[17]-[22]MaC	Java
				[23]-[25]PaX	Java
				[26]RMOR	C
				[27]The concurrent runtime monitoring of formally specified programs	Ada
			Module level	[28]-[32]The CORBA-based distributed model for runtime monitoring	Java
			Architecture level	[33]-[37]JavaMOP	Java
				[38]MOVEC	C
				[42]The runtime monitoring for web services interaction	Java
			Subsystem level	[39]The runtime monitoring of distributed reactive systems	Java
				[40][41]Mega	Java
				[43]The trace-oriented monitoring framework for cloud systems	Java

- 传统的静态验证的方法已不再适用于运行时验证Web 服务组合的正确性，需要采用运行时监控技术作为静态验证技术的有益补充，来动态地验证Web 服务组合过程的执行轨迹是否与规定的属性一致。



早期WEB监控

ISEU成果：基于PSC的时态属性监控方法



➢首先分析用户需求，用PSC 来表示待监控属性，然后通过AspectJ 生成器为每个待监控属性生成对应的待监控aspect，再由Aspect 编译器编译生成AspectJ 类文件。最后将待监控属性的AspectJ 类文件和BPEL 执行引擎通过编织器(Weaver)一起运行。编织器的主要工作是把关注的PSC 方面逻辑与其相关的组合服务过程中实体(如消息、服务端口和操作等实际执行代码)绑定。

➢当PSC 方面逻辑和组合服务一起执行时，编织在一起的相关横切关注点的方面就会自动在服务业务逻辑的切入点插入监控逻辑。监控逻辑执行的信息进一步被观察器捕获，这些捕获的信息是相关的服务业务逻辑执行时用户关注的一些状态信息，如服务、活动、操作、消息事件的间隔时间和执行日志等，运行时由各aspect 方面捕获信息并传递给观察器。

ISEU的工作

- Lulu Wang, Jingyue Li, Bixin Li: Tracking runtime concurrent dependences in java threads using thread control profiling. J. Syst. Softw. 148: 116-131 (2019)
- Bixin Li, Lulu Wang, Hareton Leung: Profiling selected paths with loops. Sci. China Inf. Sci. 57(7): 1-15 (2014) instrument
- Bixin Li, Lulu Wang, Hareton Leung, Fei Liu: Profiling all paths: A new profiling technique for both cyclic and acyclic paths. J. Syst. Softw. 85(7): 1558-1576 (2012) instrument
- Bixin Li, Shunhui Ji, Li Liao, Dong Qiu, Mingjie Sun: Monitoring Web Services for Conformance. SOSE 2013: 92-102

Continuous Monitoring (DE.CM)

Jason Edwards

A Comprehensive Guide to the NIST Cybersecurity Framework 2.0: Strategies, Implementation, and Best Practice

Year: 2025 | Book Chapter | Publisher: Wiley

SMARLA: A Safety Monitoring Approach for Deep Reinforcement Learning Agents

Amirhossein Zolfagharian; Manel Abdellatif; Lionel C. Briand; Ramesh S

IEEE Transactions on Software Engineering

Year: 2025 | Volume: 51, Issue: 1 | Journal Article | Publisher: IEEE

HighGuard: Cross-Chain Business Logic Monitoring of Smart Contracts

Mojtaba Eshghie; Cyrille Artho; Hans Stammler; Wolfgang Ahrendt; Thomas T. Hildebrandt; Gerardo Schneider

2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)

Year: 2024 | Conference Paper | Publisher: IEEE

Space-Time-Aware Proactive QoS Monitoring for Mobile Edge Computing

Shunhui Ji; Jiajia Li; Huiying Jin; Ting Wei; Hai Dong; Pengcheng Zhang; Athman Bouguettaya

IEEE Transactions on Network and Service Management

Year: 2024 | Volume: 21, Issue: 5 | Journal Article | Publisher: IEEE

A survey of software runtime monitoring

Lihua Gao; Minyan Lu; Luyi Li; Cong Pan

2017 8th IEEE International Conference on Software Engineering and Service Science (ICSESS)

Year: 2017 | Conference Paper | Publisher: IEEE

JavaMOP: Efficient parametric runtime monitoring framework

Dongyun Jin; Patrick O'Neil Meredith; Choonghwan Lee; Grigore Roşu

2012 34th International Conference on Software Engineering (ICSE)

Year: 2012 | Conference Paper | Publisher: IEEE

Adaptive Service Composition Based on Runtime Requirements Monitoring

Zhanlei Ma; Lin Liu; Hongji Yang; John Mylopoulos

2011 IEEE International Conference on Web Services

Year: 2011 | Conference Paper | Publisher: IEEE

A Performance Guarantee Approach for Cloud Applications Based on Monitoring

Jin Shao; Qianxiang Wang

2011 IEEE 35th Annual Computer Software and Applications Conference Workshops

Year: 2011 | Conference Paper | Publisher: IEEE

A taxonomy and catalog of runtime software-fault monitoring tools

N. Delgado; A.Q. Gates; S. Roach

IEEE Transactions on Software Engineering

Year: 2004 | Volume: 30, Issue: 12 | Journal Article | Publisher: IEEE

Cited by: Papers (235) | Patents (2)

Monitoring Requirements in Systems of Systems

Michael Vierhauser; Rick Rabiser; Paul Grünbacher

IEEE Software

Year: 2016 | Volume: 33, Issue: 5 | Magazine Article | Publisher: IEEE

Runtime Monitoring of Web Service Conversations

Jocelyn Simmonds; Yuan Gan; Marsha Chechik; Shiva Nejati; Bill O'Farrell; Elena Litani; Julie Waterhouse

IEEE Transactions on Services Computing

Year: 2009 | Volume: 2, Issue: 3 | Journal Article | Publisher: IEEE

Cited by: Papers (54)

8 基于知识方法

- 核心思想：利用知识图谱描述领域知识及关系，获得领域知识图谱，再构建问题知识图谱[例如程序的知识图谱]，检测其中关系是否一致、实体是否一致…，不一致的话可能存在缺陷。
  - 例如，可以检查领域KG和问题KG的三元组（实体、关系、实体）之间是否存在不一致。
- 支持C、C++、Java、JavaScript、Python、Go、Solidity等主流程序设计语言

Static Defects Detection

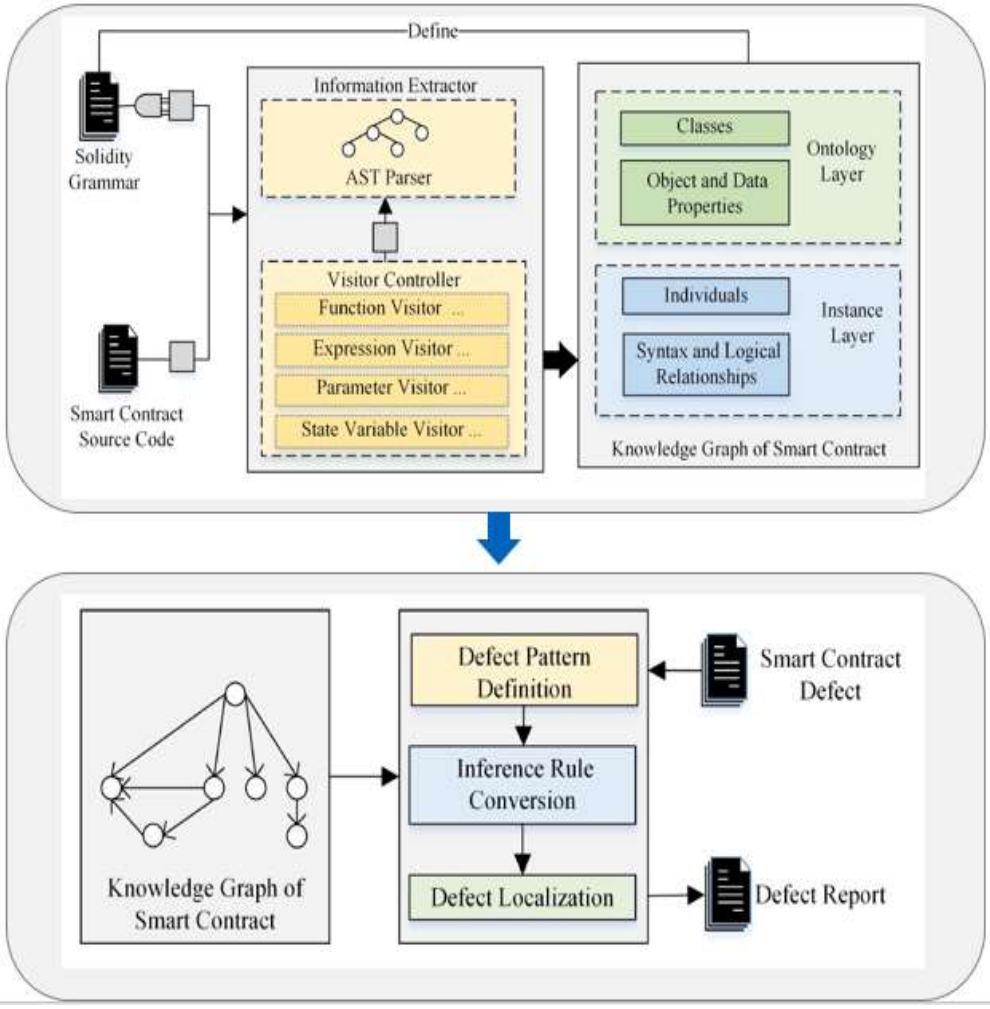
SoliDetector: Detect Security Defects of Solidity Smart Contract Based on Knowledge Graph

Step1.

Knowledge graph construction for Solidity source code

Step2.

Defect localization

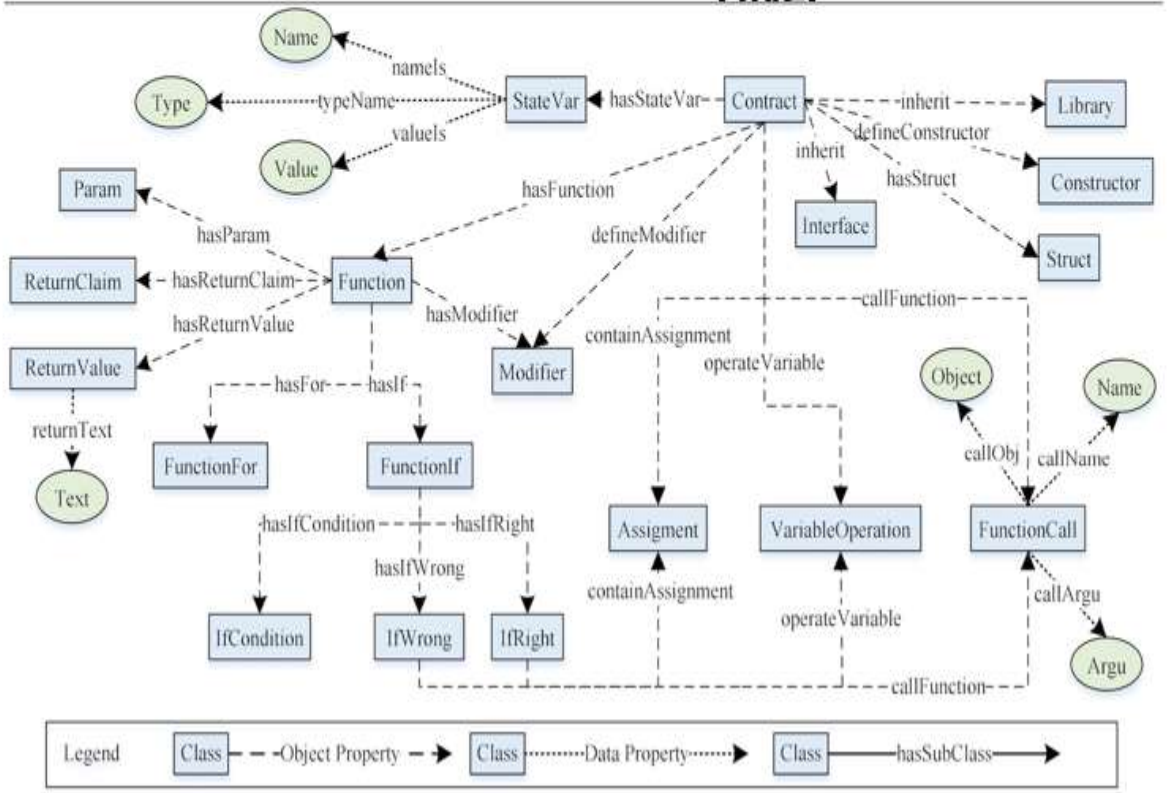


基于知识图谱代码检测流程图

SoliDetector: Detect Security Defects of Solidity Smart Contract Based on Knowledge Graph

Step1. **Knowledge graph construction for Solidity source code**

- **(1) Definition of Ontology Layer**
- **(2) Construction of Instance Layer**



- **Describe Solidity source code**
- **Compliant with the Solidity syntax rules and covers primary code elements and syntax relationships**
- **Defines abstract concepts including classes, object properties and data properties for Solidity**

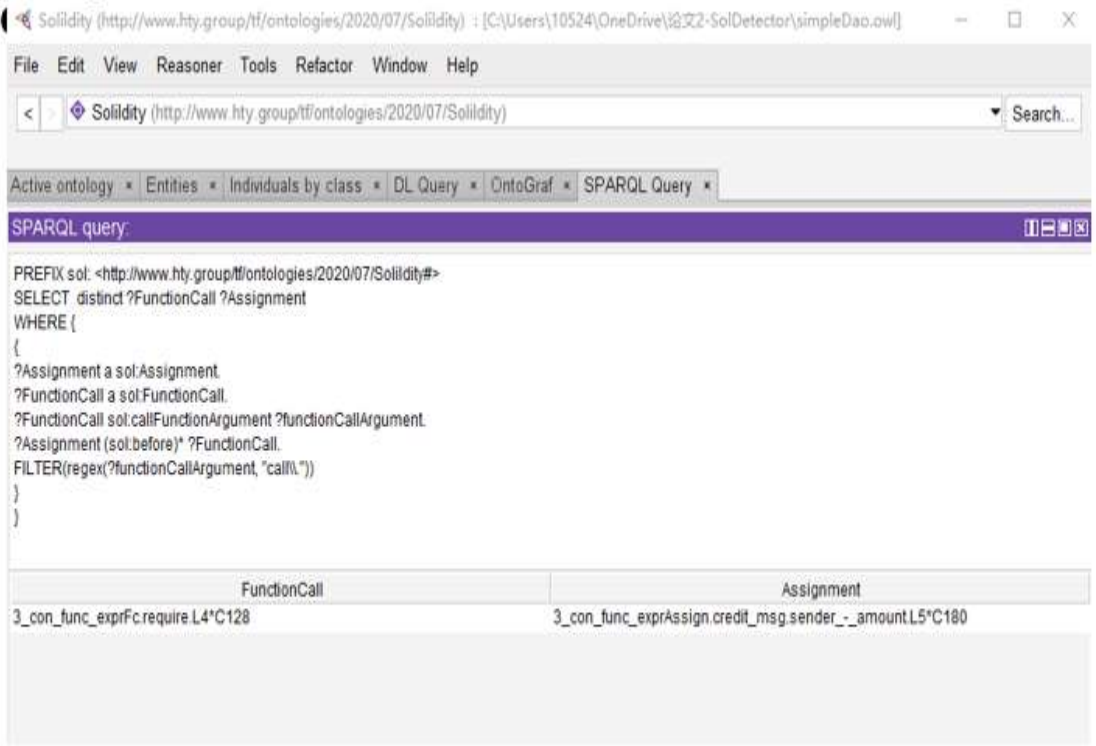
Figure. An Overall Description of Classes and Properties for Solidity in Ontology Layer

SoliDetector: Detect Security Defects of Solidity Smart Contract Based on Knowledge Graph

Step2. Defect localization

- The inference rules use SPARQL language to query and manipulate the knowledge graph data.
- Based on triple matching, SPARQL is conducted on knowledge graph. It enables flexible search strategies based on knowledge expressed in higher levels.
- Successful triple matching will return an individual name, which shows a defect is found; otherwise, no defect is f

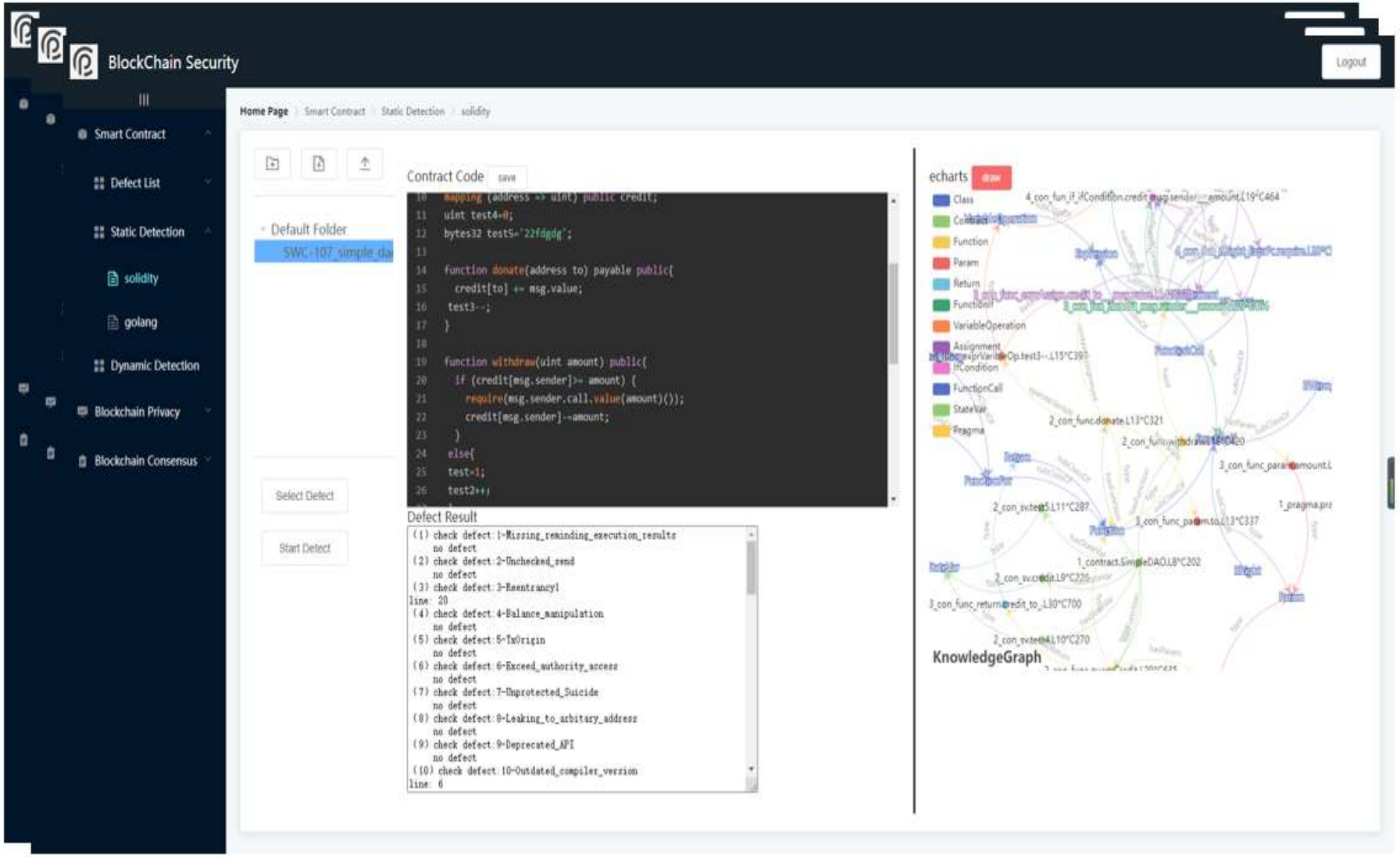
```
1.contract SimpleWithdraw {
2.   mapping (address => uint) public credit;
3.      function withdraw(uint amount) public{
4.        require(msg.sender.call.value(amount)());
5.        credit[msg.sender]-=amount;
6.      }
7.}
```



SoliDetector: Detect Security Defects of Solidity Smart Contract Based on Knowledge Graph

Table. Smart contract defect

Tool: SoliDetector



SoliDetector: Detect Security Defects of Solidity Smart Contract Based on Knowledge Graph

Comparison Experiment

- Dataset3 consists of 300 contracts with 9457 injected defects (50 contracts for 6 defect types). We collected 50 initial contracts from SolidiFI , on which injected RE,TD, IOU, UcC, TOD, and TO defects with defective code snippets.
- We inject each defect type separately and generate 300 contracts, totaling of 117,855 lines.
- We use Dataset3 as the benchmark contract to compare SoliDetector with SmartCheck, Slither and Mythril.

Defect	Injected	SoliDetector		SmartCheck		Slither		Mythril	
Name	Defects	$R_{bI}(TP)$	$R_{aI}(TP_I)(FN_I)$	$R_{bI}(TP)$	$R_{aI}(TP_I)(FN_I)$	$R_{bI}(TP)$	$R_{aI}(TP_I)(FN_I)$	$R_{bI}(TP)$	$R_{aI}(TP_I)(FN_I)$
RE	1343	2(2)	1284(1282)(61)	NA	NA	17(13)	1320(1282)(61)	6(5)	497(435)(908)
TD	1382	22(22)	1404(1382)(0)	0(0)	816(816)(566)	33(23)	1415(1382)(0)	13(13)	657(646)(736)
UcC	1346	10(10)	1335(1325)(21)	10(0)	1217(1207)(139)	0(0)	1132(1132)(214)	0(0)	(253)(224)(1122)
TOD	2001	33(33)	1363(1330)(**671**)	NA	NA	NA	NA	NA	NA
IOU	2049	121(121)	1909(1686)(**363**)	1(1)	359(358)(1691)	NA	NA	78(15)	699(634)(1415)
TO	1336	0(0)	1336(1336)(0)	1(0)	1337(1336)(0)	1(0)	1337(1336)(0)	2(0)	343(251)(1085)

Detect Defects of Solidity Smart Contract Based on the Knowledge Graph

Tianyuan Hu, Bixin Li, Zhenyu Pan, and Chen Qian

***Abstract*—Smart contract security is one of the core issues in any application based on blockchain. There are many techniques focusing on smart contract security, however, due to the diversity of Solidity versions and limitations of detection time, it is difficult for them to comprehensively localize defects in different versions of smart contracts. In this article, we propose a static defect detection method based on the knowledge graph of the Solidity language and present a defect detection tool called *SoliDetector*. First, we define the ontology layer of the knowledge graph and construct the instance layer in which syntactic and logical relationships are captured. Second, we introduce the defect pattern to describe each defect and design inference rules to infer complex relationships and judge whether a defect exists. Finally, we localize defects by executing SPARQL queries. *SoliDetector* can support the detection of 20 kinds of defects and the automatic SPARQL query generation. We conducted several experiments on multiple datasets. *SoliDetector* obtains a high *F-score* (i.e., 92.97% on Dataset1 and 91.54% on the SmartBug dataset). To compare *SoliDetector* with *SmartCheck*, *Slither*, and *Mythril*, we conducted experiments on a labeled benchmark Dataset3 and real-world contracts. *SoliDetector* has a high *F-score* of 94.04% and is faster than other tools with an average time of 0.37 s for each contract.**

***Index Terms*—Defect pattern, inference rule, knowledge graph, SPARQL query, smart contract.**

is compiled to Ethereum virtual machine (EVM) instructions before it is deployed on an Ethereum platform. Once a smart contract is deployed, it will be executed on all nodes in the network as a program and cannot be modified or replaced easily online. As smart contract involves certain information about private data and digital assets, any attacks may cause huge losses of money or assets and even divulge a secret [8]. Therefore, the security of the smart contract is always a prime problem to be solved in the blockchain.

To solve the security issues of the smart contract, researchers have proposed some feasible methods, these mainstream methods mainly borrow the following three kinds of techniques: pattern matching [9], [10], [11], symbolic execution [12], [13], [14], [15], [16], and fuzz testing [17], [18], [19], [20], [21]. *Pattern matching* requires the conversion from source code to an intermediate representation (IR) and predefined defect patterns. Unfortunately, the existing IRs can capture only partial syntax and semantic information of interest to their developers resulting in losses of crucial information during the conversion. The accuracy of predefined patterns is always unsatisfactory because of over-approximation with the superficial logic supported by

Detect Defects of Solidity Smart Contract Based on the Knowledge Graph

Tianyuan Hu; Bixin Li; Zhenyu Pan; Chen Qian

IEEE Transactions on Reliability

Year: 2024 | Volume: 73, Issue: 1 | Journal Article | Publisher: IEEE

A Defect Detection Method Based on Code Defect Knowledge Graph

He Yang; Wei Le; Li Yumin

2023 9th International Conference on Computer and Communications (ICCC)

Year: 2023 | Conference Paper | Publisher: IEEE

Review on Applications of Knowledge Graph in Software Development

Zhihua Chen; Ming Zhao; Luchang Yang; Fan Xu; Hongxia Ji; Peiya Wu; Kai Zhang; Xiaogang Dong; Xiaofeng Li

2023 6th International Symposium on Autonomous Systems (ISAS)

Year: 2023 | Conference Paper | Publisher: IEEE

Knowledge Graph Based on Defect Detection of Aerospace FPGA Software Code

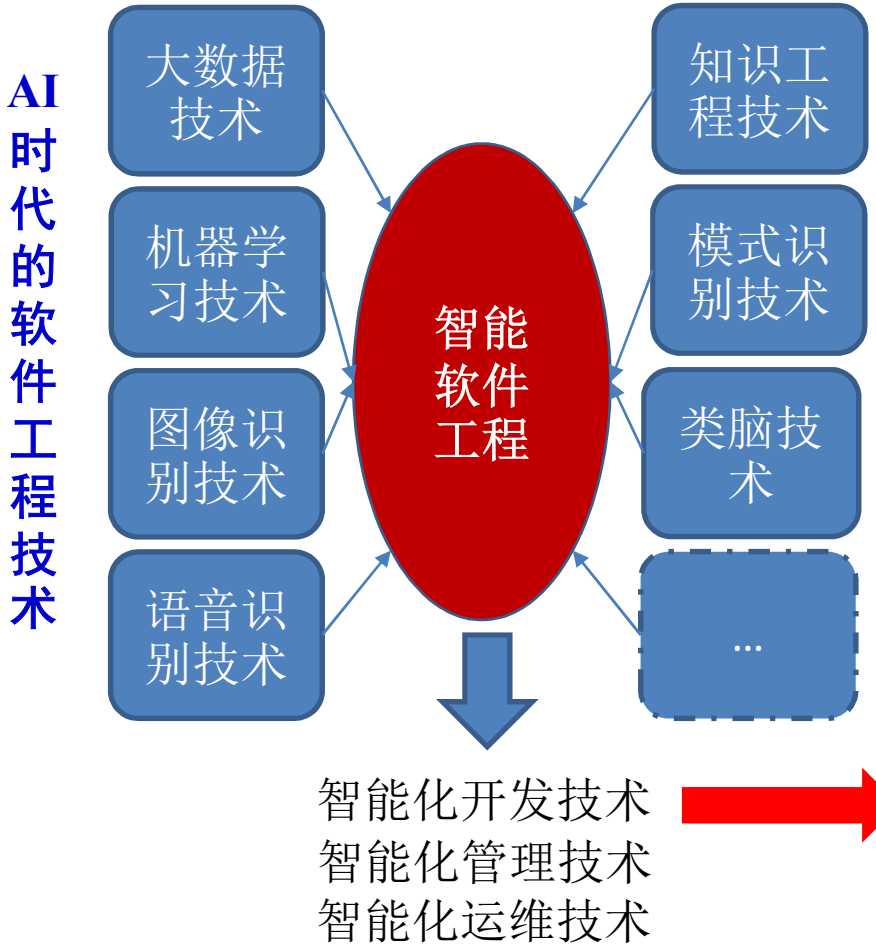
Hao Yan; DongFang Li

2021 2nd International Conference on Computer Science and Management Technology (ICCSMT)

Year: 2021 | Conference Paper | Publisher: IEEE

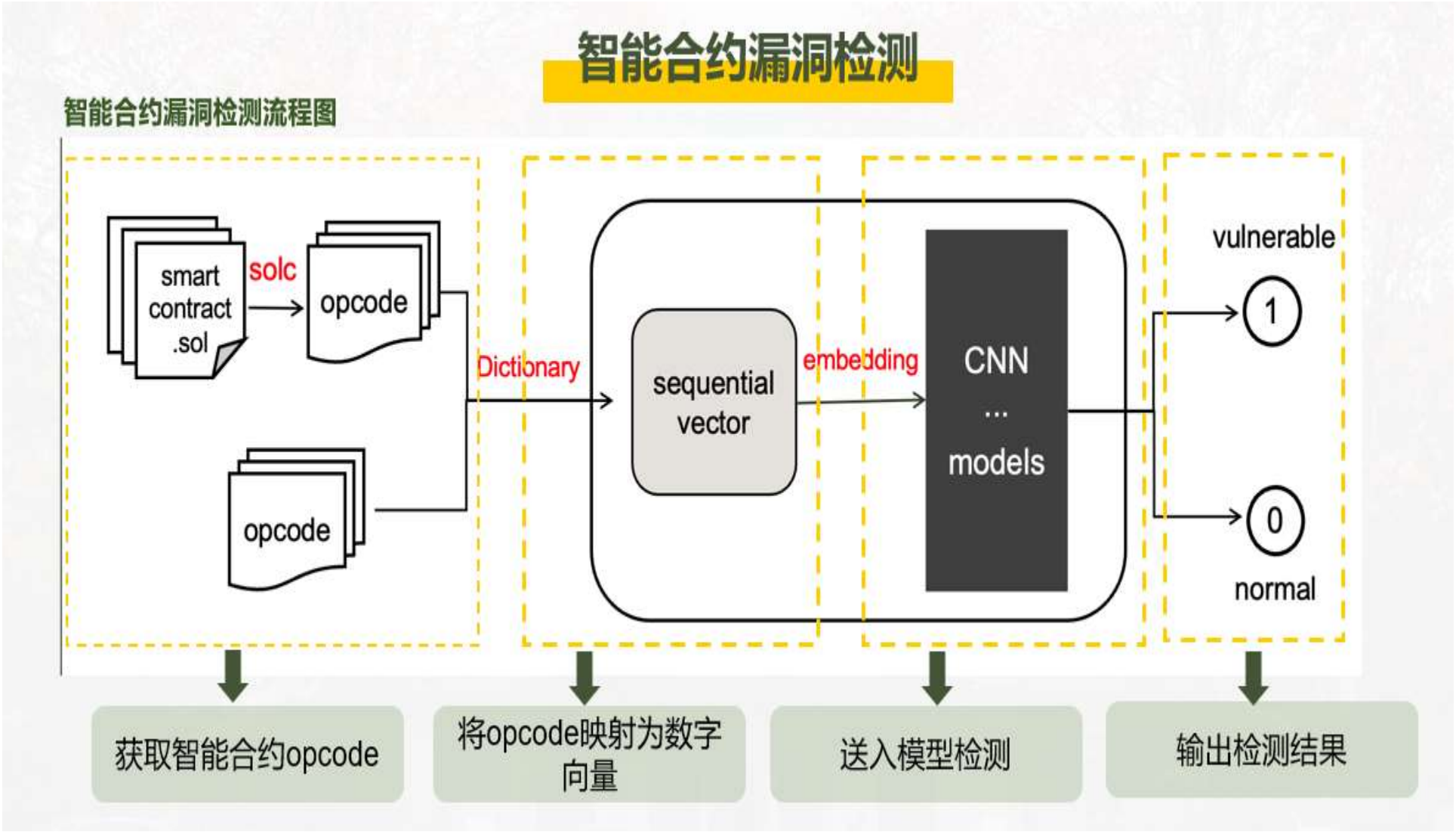
9 智能化方法

- 利用机器学习的各种模型
  - 数据集/训练模型：项目-测试需求，测试需求-TC，测试需求-Tscript，项目-测试报告…
  - 测试用例自动生成、选择和优先排序
  - 测试脚本自动生成
  - 缺陷预测：数量、出错倾向性、缺陷密度、缺陷严重性、缺陷分布情况



- 是否提高软件能力？
  - 能检测更多类型缺陷(语法的、语义的、逻辑的...)
  - 能检测更多地方的缺陷(文档的、模型的、代码的、数据的...)
- 是否提高软件质量？
  - 查全率
  - 查准率
  - F-value
- 是否提高软件性能？
  - 检测时间
  - 资源占用率
- 是否降低开发成本和运维成本？
  - 时间成本
  - 资源成本
  - 经济成本

哪些没有发生变化？哪些发生了变化？



基于深度学习的智能合约漏洞检测

PonziFinder: Attention-Based Edge-Enhanced Ponzi Contract Detection

Yingying Chen, Bixin Li, Yan Xiao, and Xiaoning Du

***Abstract*—*Ponzi contracts* are fraudulent investment scams that promise high returns with little risk to investors. However, existing methods for detecting Ponzi contracts have several limitations. For example, they struggle to deal with the class imbalance problem, and their analysis of function call transactions is inadequate, resulting in redundant features. To tackle the challenges of detecting Ponzi contracts, we present PonziFinder, a novel approach that leverages convolutional-based edge-enhanced graph neural network and attention mechanism for the classification of contract transaction graphs. In contrast to previous methods, we not only consider transaction value and timestamp but also analyze transaction input to standardize and sort transactions. We extract node and edge features that capture the unique characteristics of Ponzi contracts. The edge feature, reflecting interaccount correlation, enhances the propagation and updating of node features for effective Ponzi contract detection. To prevent oversmoothing of node embedding caused by the shallow transaction graph and extract important account node information, we introduce an attention-based global layerwise aggregation mechanism (ALGA) for generating the final contract graph representation for classification. Moreover, we optimize the node feature set and use an effective strategy based on undersampling and ensemble learning to address the issue of class imbalance. Experimental results show that PonziFinder can detect all types of Ponzi contracts (100%) with 97% accuracy when there is sufficient transaction data, outperforming other models. The analysis of input values and the ALGA mechanism are experimentally shown to improve accuracy by 4% and 2%, respectively. In summary, PonziFinder is a novel and effective method for detecting Ponzi contracts. Our approach addresses the limitations of existing methods and demonstrates significant improvements in accuracy and efficiency.**

cannot be modified. Therefore, any errors or pitfalls in deployed smart contracts can cause cryptocurrency loss. The security issues of smart contracts are primarily divided into code security and contract security. Code security focuses on smart contracts that are vulnerable to attackers due to code or other designs, such as reentrancy [2], timestamp dependence [3], and others [4]. On the other hand, contract security focuses on fraudulent events introduced into the blockchain domain, such as Ponzi schemes [5] and Honeypot [6]. Given the increasing prevalence of traditional scams, contract security is in urgent need of being addressed.

A Ponzi scheme is a classic form of fraud that has migrated to the blockchain ecosystem in smart contracts. Due to the close-source nature of most smart contracts, stakeholders find it challenging to comprehend the execution logic of smart contracts thoroughly. The unique execution mechanism of smart contracts offers numerous advantages to contract creators, including no maintenance costs after the implementation of smart contracts, operator anonymity, and stakeholder confidence in continuous repayment because smart contracts are automatically executed and cannot be terminated [7]. Shockingly, statistics reveal that Ponzi schemes operating through Bitcoin raised over $7 million between September 2013 and September 2014 [5]. The emergence of Ponzi contracts has resulted in significant losses to stakeholders and poses a severe threat to blockchain security.

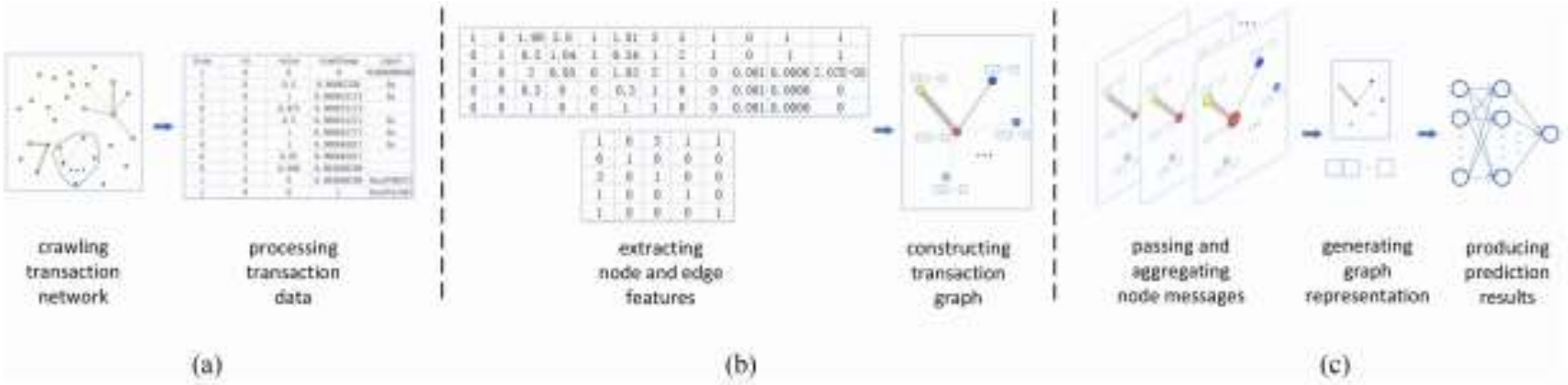


Fig. 3. Overview of PonziFinder. (a) Data preprocessing. (b) Feature extraction and transaction graph construction. (c) Ponzi contract detection.

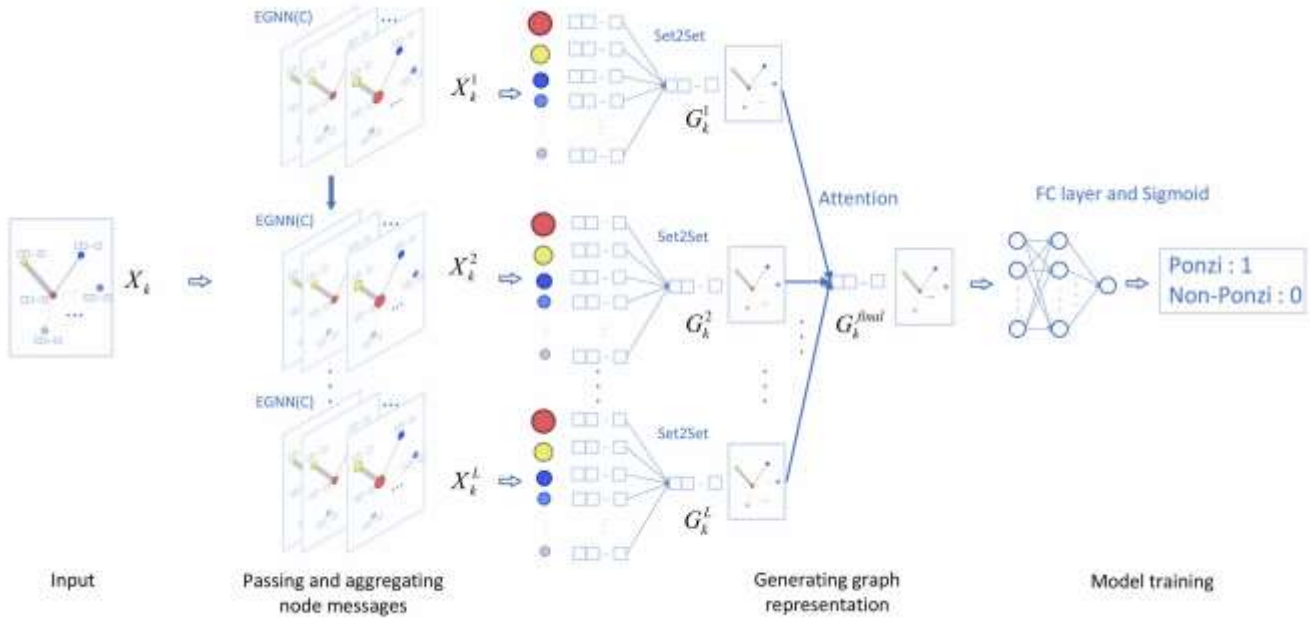


Fig. 5. Process of Ponzi contract detection.

五 软件全方位缺陷预测的主流方法

软件缺陷预测（Software Defect Prediction）是一种通过分析软件代码、开发过程数据或历史信息，提前识别软件中可能存在的缺陷（Bug）或高风险模块的技术。其核心目标是帮助开发团队优化测试资源分配、提高软件质量，并降低修复成本。

与缺陷检测的区别：

-缺陷检测：在代码中直接发现已存在的错误（如静态分析工具）。

-缺陷预测：在缺陷实际出现前，预测哪些代码模块或组件未来可能发生问题（如高风险文件预测）。

预测对象：

-模块级：预测某个文件、类或函数是否有缺陷。

-代码变更级：分析某次代码提交（Commit）是否可能引入错误。

-版本级：评估软件新版本的整体缺陷风险。

1. 版本内预测
2. 跨版本预测
3. 跨项目预测
4. 跨组织/企业预测
5. 跨领域/行业预测
6. 跨国家/地区预测
7. ……

预测技术的应用范围，或者说：待预测项目的范围。

缺陷预测的主流方法

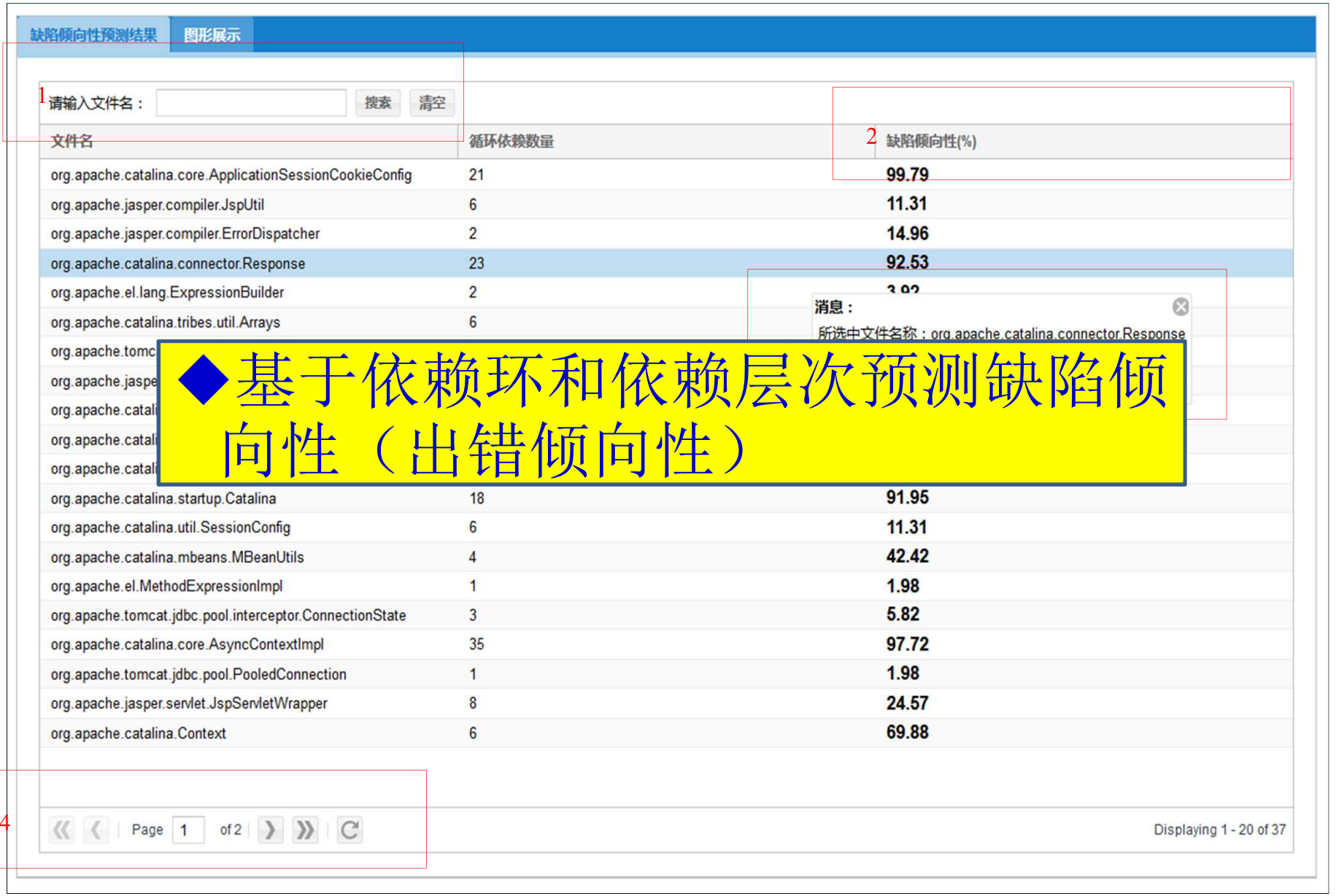
①基于统计：统计抽样
②基于软件度量：圈复杂度、Halstead、LOC、功能点…
③基于数据：大数据、演化数据、开发过程数据
④基于学习：集成、深度、半监督、标签
⑤基于特征：特征选择、特征迁移
⑥基于网络：神经、贝叶斯、复杂
⑦基于相似性：相似性/不相似性度量
⑧基于切片：静态、动态
⑨基于信息：信息熵、信号量
⑩基于向量机：SVM、向量回归
⑪基于XX算法：粒子群、蚁群、鱼群、
⑫……

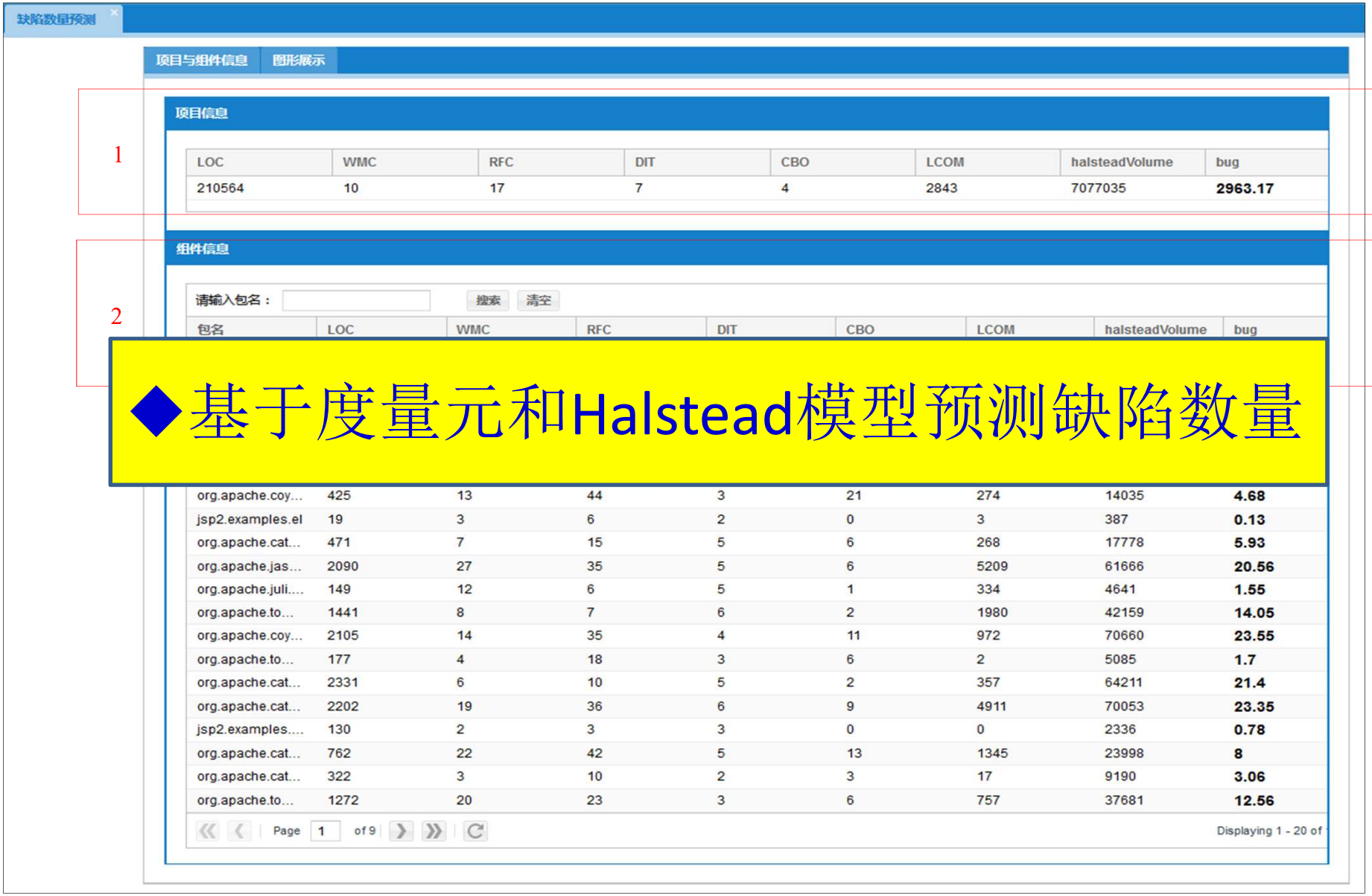
基于相似性度量
基于主题模型
基于贝叶斯模型
基于关联规则
基于 Logicboost
基于半监督学习
数据驱动的
深度信念网
基于集成学习
基于特征筛选
基于无标签数据
基于（多核）向量机
基于 KL 散度
基于最大信息系数
基于组织协同进化
基于组合机器学习算法
即时软件缺陷预测
基于特征迁移【CP】
基于随机森林和 SVM
基于搜索的半监督集成【CP】
基于数据过采样和集成学习
基于属性离散化
基于似然关系模型
基于实例选择的集成跨项目缺陷预测
基于实例迁移的跨项目软件缺陷预测
基于实例过滤和迁移的跨项目缺陷预测

基于自动特征工程
基于主动学习
基于主成分分析
基于支持向量回归
基于遗传优化支持向量机
基于早期数据
基于演化数据
基于域自适应神经网络
基于优化 PSO-BP 算法
基于新型 PSO 算法优化 BP 神经网络
基于训练数据选择【CP】
基于学习机制
基于相似度匹配
基于相关性分析【CP】
基于线性 LTSA 算法维数约减
基于统计抽样的非均衡分类方法
基于特征提取和代价敏感学习
基于失信因子的软件缺陷预测模型
基于生命周期的软件缺陷预测技术
基于深度自编码网络的软件缺陷预测
基于深度学习的细粒度软件缺陷预测
基于社会软件工程的软件缺陷预测
基于软件度量的软件缓存区溢出漏洞预测
基于软件测试的软件缺陷预测
基于权重方法的跨项目预测
基于迁移学习的软件缺陷预测算法

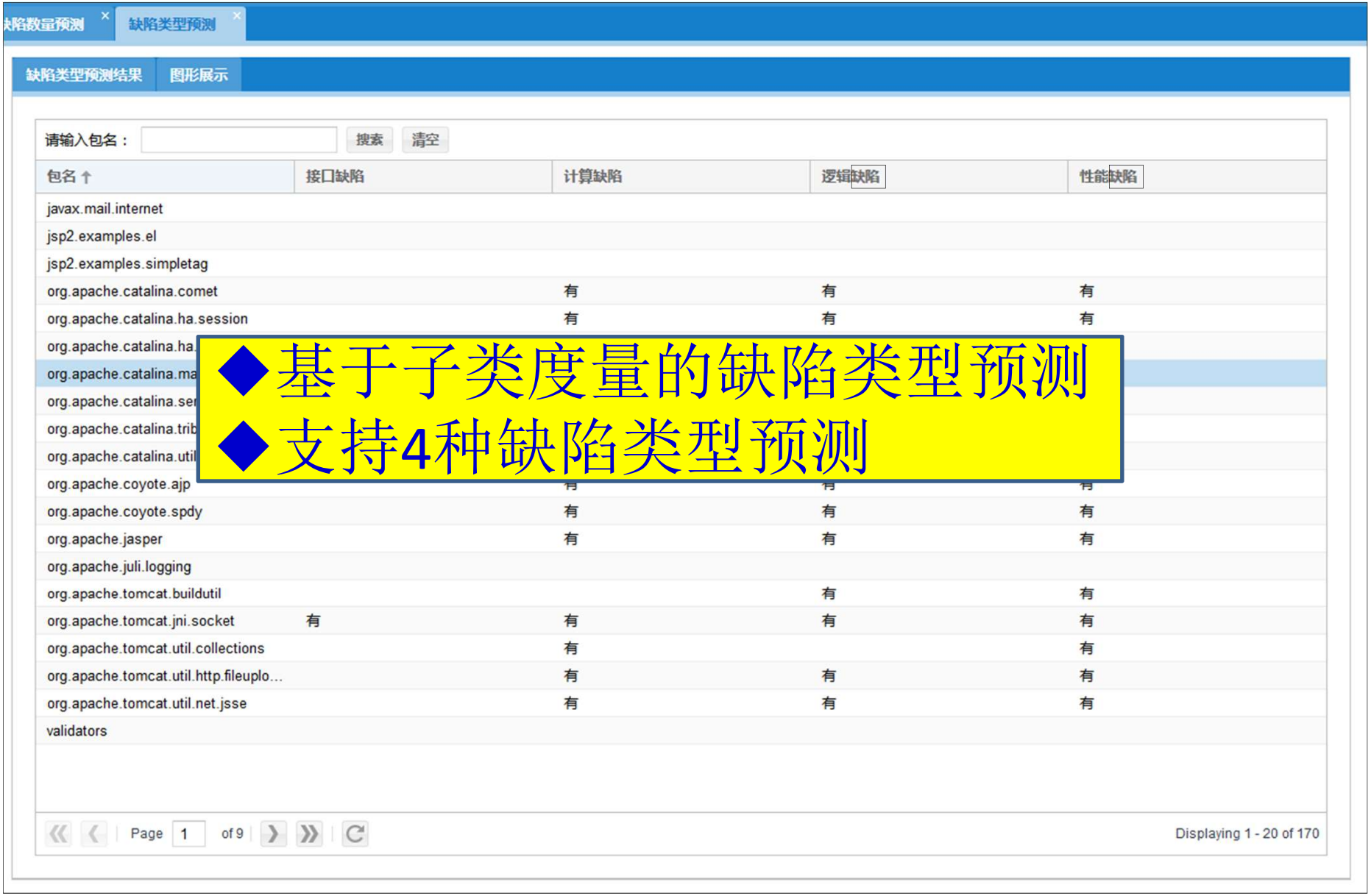
基于爬虫的软件缺陷预测
基于模糊支持向量机的软件缺陷预测
基于模糊支持向量回归的软件缺陷预测
基于模糊加权极限学习机的类不平衡软件缺陷预测
基于流形学习的面向对象的软件缺陷预测模型
基于邻域三支决策粗糙集模型的软件缺陷预测
基于两种统计模型的软件缺陷预测
基于联合表示的软件缺陷预测方法
基于粒子群优化 SVM 的面向对象软件缺陷预测
基于均衡有偏支持向量机的软件缺陷预测
基于卷积神经网络的代价敏感软件缺陷预测
基于聚类的软件模块缺陷预测方法
基于距离度量学习的软件缺陷预测方法
基于局部稀疏重构度量学习的软件缺陷预测
基于集成学习的软件缺陷预测方法
基于加权 RFE_Bayes 方法的软件缺陷预测
基于集成模型的软件缺陷预测
基于集成混合采样的软件缺陷预测
基于极限学习机的软件缺陷预测
基于灰色预测理论的软件缺陷预测
基于规则学习和朴素贝叶斯算法的软件缺陷预测
基于概率的无监督缺陷预测方法
基于改进蚁群算法在软件缺陷预测
基于改进稀疏自编码神经网络的软件缺陷预测
基于改进深度森林算法的软件缺陷预测
基于改进蝙蝠算法的软件缺陷预测

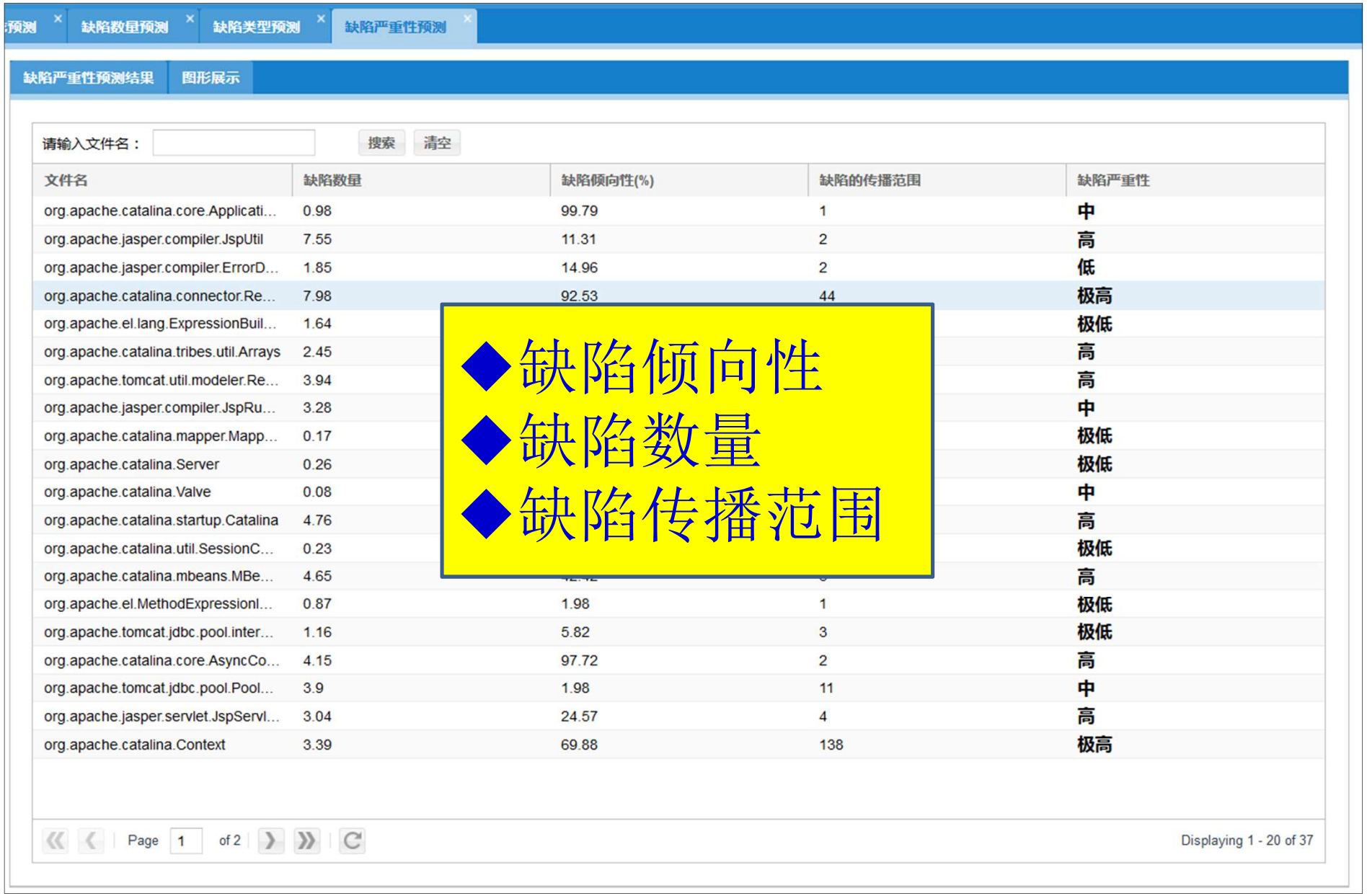
基于改进 PSO 与 NB 算法的软件缺陷预测
基于改进 ACO 优化 BP 神经网络的软件缺陷预测
基于改进 ACO 优化 BPNN 的软件缺陷预测
基于复杂性的软件缺陷预测
基于复杂网络的动态软件缺陷预测算法
基于分类器融合技术的软件缺陷预测
基于分类的软件缺陷严重性预测
基于非负稀疏图的协同训练软件缺陷预测
基于多源数据的跨项目软件缺陷预测
基于多维高斯分布概率模型的软件缺陷预测
基于多核字典学习的软件缺陷预测
基于多核集成学习的跨项目软件缺陷预测
基于度量元的静态软件缺陷预测
基于度量元的静态跨项目软件缺陷预测
基于度量的软件缺陷预测
基于动态代价敏感型贝叶斯网络的软件缺陷预测方法
基于代码修改的多目标有监督缺陷预测建模方法
基于代价敏感主动学习的软件缺陷预测方法
基于代价敏感支持向量机的软件缺陷预测
基于粗糙集_支持向量机的软件缺陷预测
基于程序语义的软件缺陷预测
基于程序语义的软件缺陷预测
基于程序聚类的软件缺陷预测方法
基于不相似性的软件缺陷预测算法
基于 PN 学习的软件模块缺陷预测
基于 GM_1_1_模型的软件缺陷数预测


缺陷倾向性预测结果
图形展示
1
请输入文件名：
搜索
清空
2
文件名
循环依赖数量
缺陷倾向性(%)
org.apache.catalina.core.ApplicationSessionCookieConfig 21 99.79
org.apache.jasper.compiler.JspUtil 6 11.31
org.apache.jasper.compiler.ErrorDispatcher 2 14.96
org.apache.catalina.connector.Response 23 92.53
org.apache.el.lang.ExpressionBuilder 2
org.apache.catalina.tribes.util.Arrays 6
消息：
所选中文件名称：org.apache.catalina.connector.Response
◆基于依赖环和依赖层次预测缺陷倾向性（出错倾向性）
org.apache.catalina.startup.Catalina 18 91.95
org.apache.catalina.util.SessionConfig 6 11.31
org.apache.catalina.mbeans.MBeanUtils 4 42.42
org.apache.el.MethodExpressionImpl 1 1.98
org.apache.tomcat.jdbc.pool.interceptor.ConnectionState 3 5.82
org.apache.catalina.core.AsyncContextImpl 35 97.72
org.apache.tomcat.jdbc.pool.PooledConnection 1 1.98
org.apache.jasper.servlet.JspServletWrapper 8 24.57
org.apache.catalina.Context 6 69.88
4
Page 1 of 2
Displaying 1 - 20 of 37



◆基于度量元和Halstead模型预测缺陷数量


缺陷数量预测
缺陷类型预测
缺陷类型预测结果
图形展示
请输入包名：
搜索
清空
包名
接口缺陷
计算缺陷
逻辑缺陷
性能缺陷
javax.mail.internet
jsp2.examples.el
jsp2.examples.simpletag
org.apache.catalina.comet 有 有 有
org.apache.catalina.ha.session 有 有 有
org.apache.coyote.spdy 有 有 有
org.apache.jasper 有 有 有
org.apache.juli.logging
org.apache.tomcat.buildutil 有 有
org.apache.tomcat.jni.socket 有 有 有 有
org.apache.tomcat.util.collections 有 有
org.apache.tomcat.util.http.fileuplo... 有 有 有
org.apache.tomcat.util.net.jsse 有 有 有
validators
◆基于子类度量的缺陷类型预测
◆支持4种缺陷类型预测
Page 1 of 9
Displaying 1 - 20 of 170


预测
缺陷数量预测
缺陷类型预测
缺陷严重性预测
缺陷严重性预测结果
图形展示
请输入文件名：
搜索
清空
文件名 缺陷数量 缺陷倾向性(%) 缺陷的传播范围 缺陷严重性
org.apache.catalina.core.Applicati... 0.98 99.79 1 中
org.apache.jasper.compiler.JspUtil 7.55 11.31 2 高
org.apache.jasper.compiler.ErrorD... 1.85 14.96 2 低
org.apache.catalina.connector.Re... 7.98 92.53 44 极高
org.apache.el.lang.ExpressionBuil... 1.64 极低
org.apache.catalina.tribes.util.Arrays 2.45 高
org.apache.tomcat.util.modeler.Re... 3.94 高
org.apache.jasper.compiler.JspRu... 3.28 中
org.apache.catalina.mapper.Mapp... 0.17 极低
org.apache.catalina.Server 0.26 极低
org.apache.catalina.Valve 0.08 中
org.apache.catalina.startup.Catalina 4.76 高
org.apache.catalina.util.SessionC... 0.23 极低
org.apache.catalina.mbeans.MBe... 4.65 高
org.apache.el.MethodExpressionI... 0.87 1.98 1 极低
org.apache.tomcat.jdbc.pool.inter... 1.16 5.82 3 极低
org.apache.catalina.core.AsyncCo... 4.15 97.72 2 高
org.apache.tomcat.jdbc.pool.Pool... 3.9 1.98 11 中
org.apache.jasper.servlet.JspServl... 3.04 24.57 4 高
org.apache.catalina.Context 3.39 69.88 138 极高
◆缺陷倾向性
◆缺陷数量
◆缺陷传播范围
Page 1 of 2
Displaying 1 - 20 of 37

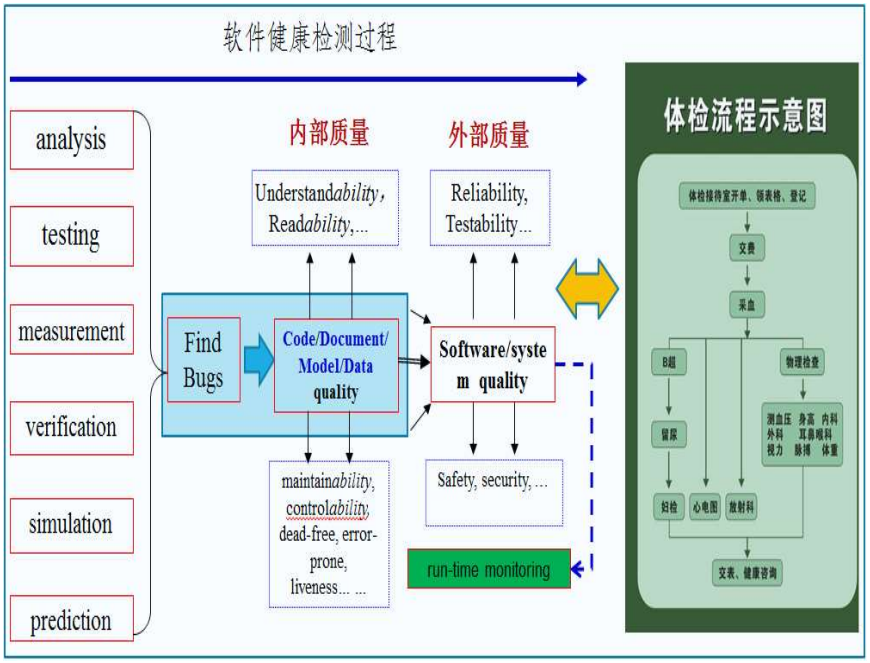
六 SHC研发进展汇报

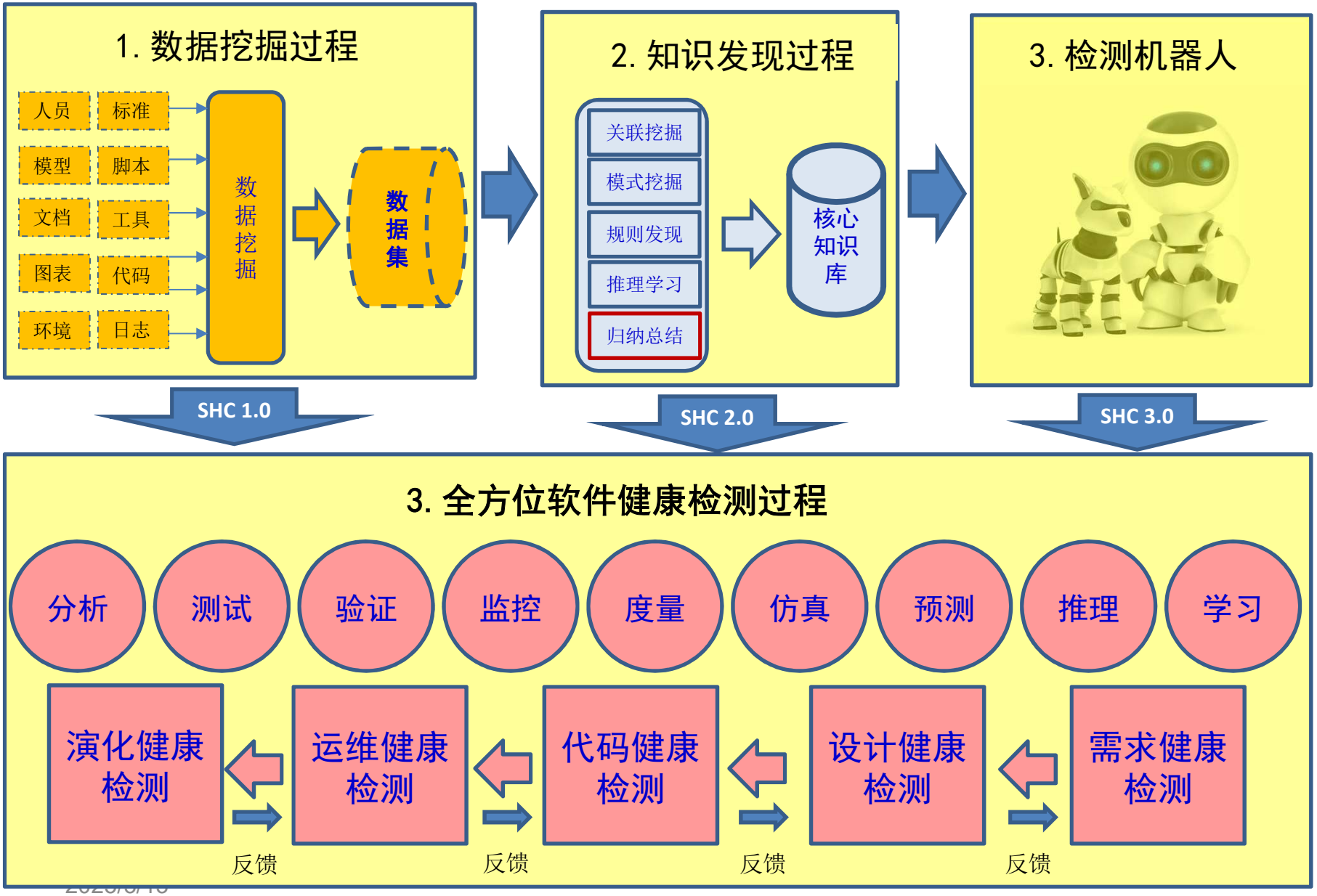
- 基于开发过程大数据SHC
- 基于知识库SHC
- 软件缺陷检测机器人

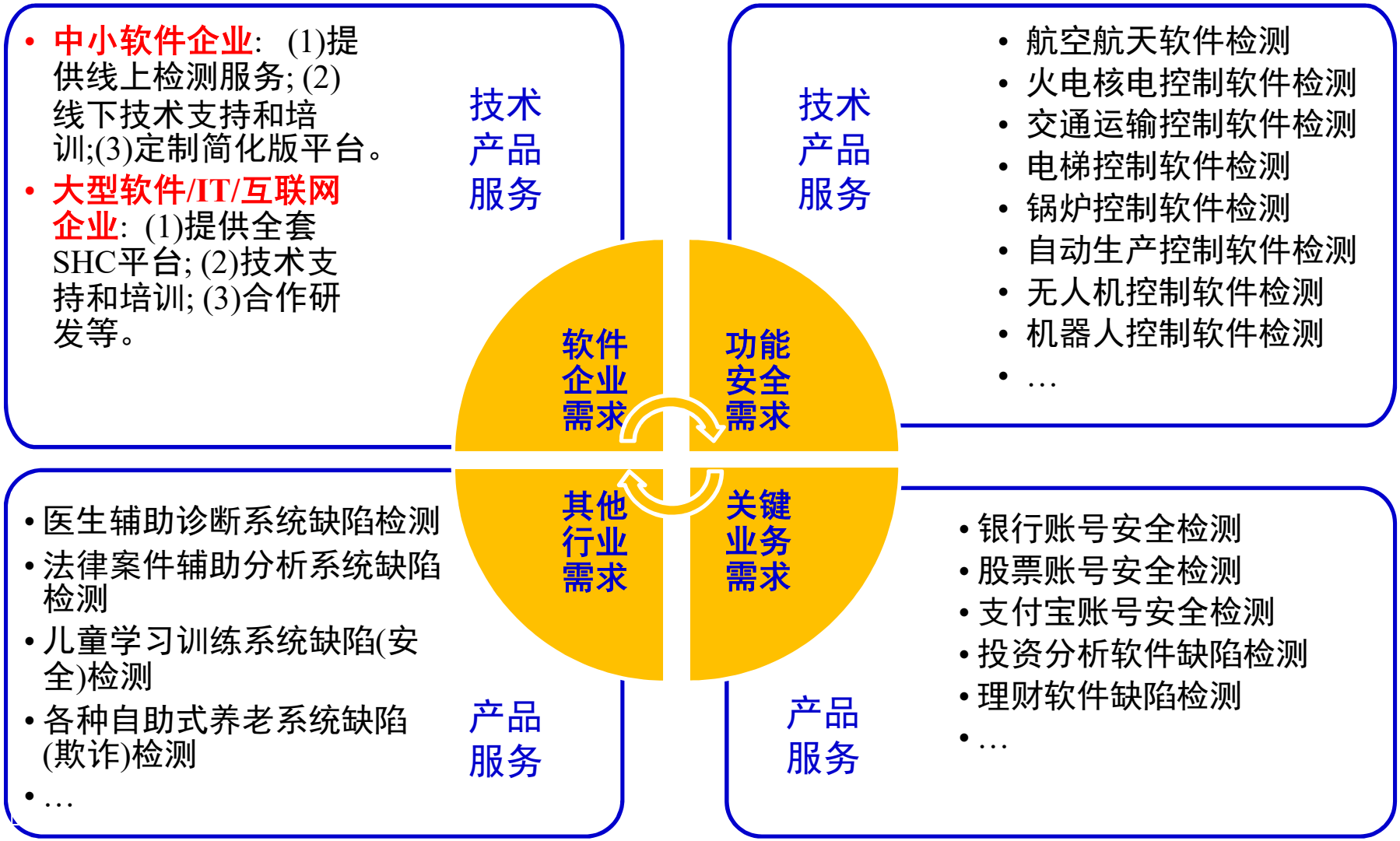
- 设计检测
- 代码检测

• 案例介绍：SHC 1.0 软件全方位健康检测平台

- 如何解决软件健康问题？
  - 第一步：软件健康检测！（如同人们需要体检、诊疗一样）
  - 第二步：软件缺陷定位！
  - 第三步：软件缺陷修复！
- SHC平台主要手段
  - 分析（望闻问切、专家会诊）
  - 测试（运动平板实验、抽血化验）
  - 监控（透视、B超、心电图）
  - 度量（量身高、体重、血压）
  - 验证（切片、肠镜、胃镜检查）
  - 仿真（核磁共振）
  - 推理（病例分析）




1. 数据挖掘过程
人员
标准
模型
脚本
文档
工具
图表
代码
环境
日志
数据挖掘
数据集
2. 知识发现过程
关联挖掘
模式挖掘
规则发现
推理学习
归纳总结
核心知识库
3. 检测机器人
SHC 1.0
SHC 2.0
SHC 3.0
3. 全方位软件健康检测过程
分析
测试
验证
监控
度量
仿真
预测
推理
学习
演化健康检测
运维健康检测
代码健康检测
设计健康检测
需求健康检测
反馈
反馈
反馈
反馈


• 中小软件企业: (1)提供线上检测服务; (2)线下技术支持和培训;(3)定制简化版平台。
• 大型软件/IT/互联网企业: (1)提供全套SHC平台; (2)技术支持和培训; (3)合作研发等。
技术
产品
服务
• 航空航天软件检测
• 火电核电控制软件检测
• 交通运输控制软件检测
• 电梯控制软件检测
• 锅炉控制软件检测
• 自动生产控制软件检测
• 无人机控制软件检测
• 机器人控制软件检测
• …
技术
产品
服务
软件企业需求
功能安全需求
其他行业需求
关键业务需求
• 医生辅助诊断系统缺陷检测
• 法律案件辅助分析系统缺陷检测
• 儿童学习训练系统缺陷(安全)检测
• 各种自助式养老系统缺陷(欺诈)检测
• …
产品
服务
• 银行账号安全检测
• 股票账号安全检测
• 支付宝账号安全检测
• 投资分析软件缺陷检测
• 理财软件缺陷检测
• …
产品
服务